00Δομημένος Προγραμματισμός με C

Ένα απλό πρόγραμμα

Παράδειγμα 1

```
1       /* A very simple C program.
2          Your first program. */
3       #include <stdio.h>
4
5       int main()
6       {
7           printf( "Welcome to C!\n" );
8
9           return 0;
10      }
```

ΕΞΟΔΟΣ:

```
Welcome to C!
```

Γραμμές **Εξηγήσεις**

1, 2 Ξεκινούν με **/*** και τελειώνουν με ***/** δείχνοντας ότι αυτές οι δύο γραμμές είναι σχόλια για αναγνωσιμότητα του προγράμματος. Τα σχόλια αγνοούνται από το μεταγλωττιστή.

3 Είναι ένας καθοδηγητής (directive) για το **C** προκάτοχο (predecessor). Όλες οι γραμμές που ακολουθούν το # υποβάλλονται σε επεξεργασία από τον προκάτοχο πριν το πρόγραμμα μεταγλωττιστεί. Αυτή η συγκεκριμένη γραμμή λέγει στον προκάτοχο να συμπεριλάβει το περιεχόμενο του πρότυπου αρχείου κεφαλίδας για εισόδου / εξόδου (**standard input/output header file (stdio)** στο πρόγραμμα. Αυτό το συγκεκριμένο αρχείο κεφαλίδας περιέχει πληροφορίες και δηλώσεις (declarations) που χρησιμοποιούνται από το μεταγλωττιστή όταν μεταφράζει πρότυπες συναρτήσεις της βιβλιοθήκης input/output όπως **printf()**. Επίσης συμπεριλαμβάνει πληροφορίες που βοηθούν τον μεταγλωττιστή να προσδιορίσει εάν οι κλήσεις αυτών των συναρτήσεων γράφτηκαν σωστά. Θα μιλήσουμε περισσότερο για αυτό το θέμα σε μετέπειτα διαλέξεις.

4 Κενή γραμμή. Μπορείτε να έχετε όσες κενές γραμμές θέλετε στο πρόγραμμα σας και να είναι οπουδήποτε.

5 Είναι μέρος κάθε προγράμματος της **C**. Οι παρενθέσεις δείχνουν ότι η main είναι μια συνάρτηση. Τα προγράμματα της **C** μπορούν να περιέχουν μια ή περισσότερες συναρτήσεις, μια από αυτές πρέπει να είναι η **main**. Κάθε πρόγραμμα της **C** ξεκινά να εκτελείται από τη συνάρτηση **main()**.

6 Το σώμα κάθε συνάρτησης πρέπει να ξεκινά με το αριστερό άγκιστρο (brace) (**{**). Με το δεξιό άγκιστρο κλείνουμε τη συνάρτηση δημιουργώντας το σώμα κάθε συνάρτησης. Με το δεξί άγκιστρο (**}**) δημιουργούμε ένα μπλοκ εντολών που το ονομάζουμε **compound statement (σύνθετη εντολή)**.

7 Καθοδηγεί τον υπολογιστή να εκτελέσει μια πράξη: σε αυτή την περίπτωση να τυπώσει στην οθόνη τη γραμματοσειρά (string) που βρίσκεται ανάμεσα στα εισαγωγικά.

8 Κενή γραμμή.

9 Περνάμε την τιμή μηδέν **(0)** στο περιβάλλον του λειτουργικού συστήματος για να δείξουμε ότι το πρόγραμμα εκτελέστηκε με επιτυχία. Θα δούμε πιο ύστερα το εγχειρίδιο για την αποτυχία ενός προγράμματος να εκτελεστεί.

10 Δείχνει το τέλος της **main()**.

Περισσότερα για το πρώτο σας πρόγραμμα

Ολόκληρη η γραμμή (**7**) συμπεριλαμβανομένου του **printf**, τα ορίσματα της μέσα στις παρενθέσεις και το ερωτηματικό (**;**) ονομάζεται εντολή. Κάθε εντολή στη C **πρέπει να τελειώνει** με το ερωτηματικό.

- **\n** δεν τυπώνεται Η ανάστροφη κάθετος (backslash (\)) ονομάζεται **escape character** (χαρακτήρας διαφυγής). Αυτός ο χαρακτήρας διαφυγής \n σημαίνει **νέα γραμμή** (**newline**). Μετακινεί τον κέρσορα στην αρχή της νέας γραμμής.
- Εάν θέλετε να τυπώσετε \ χρησιμοποιήστε \\.
- Εάν θέλετε να τυπώσετε **”** χρησιμοποιήστε **\”**.

Χαρακτήρας Διαφυγής	**Περιγραφή**
\n	Νέα γραμμή. Μετακινεί τον κέρσορα στην αρχή της νέας γραμμής.
\t	Οριζόντιος στηλοθέτης (tab). Μετακινεί τον κέρσορα στον επόμενο στηλοθέτη.
\a	Ειδοποίηση με ήχο.
\f	Αλλαγή σελίδας (Form feed).
\r	Επαναφορά γραμμής (Carriage return).
\v	Κάθετος στηλοθέτης.
\b	Backspace
\”	**Δ**ιπλά εισαγωγικά
\’	Απλά εισαγωγικά
\O	Κενό
\\	Ανάποδη κάθετος (backslash)
\N	Οκταδική σταθερά
\xN	Δεκαεξαδική σταθερά

- **printf** και **scanf** δεν είναι μέρη της γλώσσας προγραμματισμού **C** για αυτό δεν μπορεί ο μεταγλωττιστής να βρει ορθογραφικά λάθη σε αυτά. Το πρόγραμμα σύνδεσης (linker) θα βρει αυτού του είδους λάθη.

Παράδειγμα 2

```
1   /* Printing on one line with two printf statements. */
2
3   #include <stdio.h>
4
5   int main()
6   {
7       printf( "Welcome " );
8       printf( "to C!\n" );
9
10      return 0;
11  }
```

ΕΞΟΔΟΣ:

```
Welcome to C!
```

Παράδειγμα 3

```
1   /* Printing multible lines with a single printf. */
2
3   #include <stdio.h>
4
5   int main()
6   {
7       printf( "Welcome\nto\nC!\n" );
8
9       return 0;
10  }
```

ΕΞΟΔΟΣ:

```
Welcome
to
C!
```

Παράδειγμα 4

```
1   /* Additional program */
2
3   #include <stdio.h>
4
5   int main()
6   {
7       int integer1, integer2, sum;          /* declaration */
8
9       printf( "Enter first integer\n" );    /* prompt */
10      scanf( "%d", &integer1 );             /* read one integer */
11      printf( "Enter second integer\n" );   /* prompt */
12      scanf( "%d", &integer2 );             /* read one integer */
13      sum = integer1 + integer2;            /* assignment of sum */
14      printf( "Sum is %d\n", sum );         /* print sum */
15
16      return 0;           /* indicates that the program ended successfully */
17  }
```

ΕΞΟΔΟΣ:

```
Enter first integer
45
Enter second integer
72
Sum is 117
```

Η ΓΡΑΜΜΗ 7 είναι ένας ορισμός. **integer1**,**integer2** και **sum** είναι ονόματα αναγνωριστικών για μεταβλητές (**variables**). Μια μεταβλητή είναι μια τοποθεσία στη μνήμη όπου μια τιμή μπορεί να αποθηκευτεί και να χρησιμοποιηθεί από το πρόγραμμα. Σε αυτό το παράδειγμα και οι τρεις μεταβλητές είναι του τύπου ακεραίου - **integer (int)** – που σημαίνει ότι αυτές οι μεταβλητές μπορούν να κρατήσουν ακέραιες τιμές όπως **7, -11, 0, 31914** ……. .

- Όλες οι μεταβλητές στη **C** πρέπει να οριστούν αμέσως μετά το αριστερό άγκιστρο της συνάρτησης main().
- Υπάρχουν και άλλοι τύποι δεδομένων από **int** στη **C**.
- Πολλές μεταβλητές του ιδίου τύπου μπορούν να οριστούν μέσα σε ένα ορισμό χωρισμένα με κόμμα.
- Μπορείτε να ορίσετε τους τρεις ακέραιους σε τρεις διαφορετικούς ορισμούς.
- Έγκυρα ονόματα μεταβλητών – σειρές από χαρακτήρες που αποτελούνται από γράμματα, ψηφία και τον χαρακτήρα υπογράμμισης (underscore (_)) και δεν μπορούν να ξεκινούν με ψηφίο.
- Η **C** διακρίνει μικρά-κεφαλαία γράμματα, είναι (case sensitive) - έτσι ώστε το **a1** και το **A1** είναι δύο διαφορετικά αναγνωριστικά.

Η ΓΡΑΜΜΗ 10 χρησιμοποιεί την εντολή (συνάρτηση) **scanf** για να πάρει μια τιμή από το χρήστη. Scanf λαμβάνει στοιχεία από τη πρότυπη συσκευή εισόδου (standard input device) που είναι το πληκτρολόγιο. Αυτό το scanf έχει δύο ορίσματα (arguments): (α) **"%d"** το οποίο είναι μια τυπική συμβολοσειρά ελέγχου που δεικνύει το τύπο των δεδομένων που πρέπει να δοθούν από το χρήστη. Σε αυτό το παράδειγμα το **d** είναι για **ψηφιακό ακέραιο**(**digital integer**). **%** είναι ένας χαρακτήρας διαφυγής. (β) **&integer1** ξεκινά με το εμπορικό "και" (ampersand). Με το **integer1** λέγουμε στο scanf την περιοχή της μνήμης στην οποία η μεταβλητή integer1 θα αποθηκευτεί.

Η ΓΡΑΜΜΗ 13 **sum = integer1 + integer2** υπολογίζει το άθροισμα των μεταβλητών **integer1** και **integer2** και αντιστοιχίζει το αποτέλεσμα στη μεταβλητή **sum**. **=**, **+** είναι **δυαδικοί αριθμητικοί τελεστές** (**binary arithmetic operators**).

Η ΓΡΑΜΜΗ 14 καλεί τη συνάρτηση **printf** για να τυπώσει τη σταθερή "**Sum is**" ακολουθούμενη από τη αριθμητική τιμή της μεταβλητής **sum** στην οθόνη του Η/Υ. Το **%d** δεικνύει ότι ένας ακέραιος (**integer**) θα τυπωθεί.

Επίσης θέλω να ξέρετε ότι υπολογισμοί μπορούν να εκτελεσθούν μέσα σε μια **printf** εντολή, όπως:

printf("sum is %d\n", integer1 + integer2);

Αριθμητικά με τη C

C Πράξη	Αριθμητικός τελεστής	Αλγεβρική παράσταση	C expression
Πρόσθεση	+	f+7	f+7
Αφαίρεση	-	p – c	p–c
Πολλαπλασιασμός	*	b x m	b*m
Διαίρεση	/	x/m ή x÷m	x/m
Υπόλοιπο (Modulus)	%	r mod s	r%s

Οι παρενθέσεις χρησιμοποιούνται στις παραστάσεις της **C** όπως ακριβώς χρησιμοποιούνται στις αλγεβρικές παραστάσεις.

a * (b + c)

Αλγεβρική : $m = \frac{a + b + c + d + e}{5}$

C: *m = (a + b + c + d + e)/5;*

Προτεραιότητα (Precedence)

Τελεστής	Πράξη
()	Παρενθέσεις
***, /, %**	Πολλαπλασιασμός, διαίρεση, υπόλοιπο
+, -	Πρόσθεση, αφαίρεση

Παραδείγματα:

1. **z = p * r % q + w / x – y;**
   6 1 2 4 3 5

2. **y = a * x * x + b * x + c;**
   6 1 2 4 3 5

Εάν a =2, b = 3, c = 7, x = 5 τότε y = 72

γιατί

y = 2 * 5 * 5 + 3 * 5 +7;
1 2 4 3 5

Λήψη αποφάσεων

Σχεσιακοί τελεστές	Παράδειγμα	Έννοια
==	x == y	x είναι ίσον με το y
!=	x != y	x δεν είναι ίσον με το y
>	x > y	x είναι μεγαλύτερο από το y
<	x < y	x είναι μικρότερο από το y
>=	x >= y	x είναι μεγαλύτερο ή ίσον με το y
<=	x <= y	x είναι μικρότερο ή ίσον με το y

Παράδειγμα 5

```
1   /* Using if statements, relational operators
2      and equality operators */
3
4   #include <stdio.h>
5
6   int main()
7   {
8      int num1, num2;
9
10     printf ( "Enter two integers and I will tell you\n" );
11     printf( "the relationships they satisfy: " );
12     scanf( "%d%d", &num1, &num2 );          /* read two integer values */
13
14     if (num1 == num2)
15        printf( "%d is equal to %d\n", num1, num2 );
16
17     if (num1 != num2)
18        printf( "%d is not equal to %d\n", num1, num2 );
19
20     if ( num1 < num2 )
21        printf( "%d is less than %d\n", num1, num2 );
22
23     if ( num1 > num2 )
24        printf( "%d is greater than %d\n", num1, num2 );
25
26     if ( num1 <= num2 )
27        printf( "%d is less  than or equal to %d\n",
28                                          num1, num2 );
29
30     if (num1 >= num2)
31        printf( "%d is greater than or equal to %d\n",
32                                          num1, num2 );
33
34     return 0;         /* indicate program ended successfully */
35  }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter two integers, and I will tell you
the relationship they satisfy: 3 7
3 is not equal to 7
3 is less than 7
3 is less than or equal to 7
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter two integers, and I will tell you
the relationship they satisfy: 22 12
22 is not equal to 12
22 is greater than 12
22 is greater than or equal to 12
```

3ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter two integers, and I will tell you
the relationship they satisfy: 7 7
7 is  equal to 7
7 is less than or equal to7
7 is greater than or equal to 7
```

Τελεστές Bit

Οι τελεστές bit χρησιμοποιούνται για τον χειρισμό των bits μιας ακέραιας μεταβλητής ή σταθεράς. Η τιμή ενός bit μπορεί να είναι 0 ή 1. Ο υπολογισμός της τιμής μιας έκφρασης που περιέχει τελεστές bit γίνεται με την εφαρμογή τους στα αντίστοιχα bits των τελεστέων.

Οι τελεστές bit είναι ιδιαίτερα χρήσιμοι σε διάφορες εφαρμογές, όπως κωδικοποίηση πληροφορίας και πρωτόκολλα επικοινωνιών, κρυπτογράφηση δεδομένων και σε εφαρμογές που επικοινωνούν με μονάδες υλικού.

Ο τελεστής &

Ο τελεστής & εφαρμόζει τη λογική πράξη AND στα bits των δύο τελεστέων και θέτει το bit εξόδου στο 1 μόνο αν τα αντίστοιχα bits και στους δύο τελεστέους είναι 1. Αλλιώς, το bit εξόδου τίθεται στο 0. Για παράδειγμα, το αποτέλεσμα της πράξης 19 & 2 είναι 2:

```
        00010011    (19)
&       00000010    (2)
        ------------------------
        00000010    (2)
```

Ο τελεστής |

Ο τελεστής | εφαρμόζει τη λογική πράξη OR στα bits των δύο τελεστέων και θέτει το bit εξόδου στο 0 μόνο αν τα αντίστοιχα bits και στους δύο τελεστέους είναι 0. Αλλιώς, το bit εξόδου τίθεται στο 1. Για παράδειγμα, το αποτέλεσμα της πράξης 19 | 6 είναι 23:

```
        00010011    (19)
|       00000110    (6)
        ------------------------
        00010111    (23)
```

Ο τελεστής ^

Ο τελεστής ^ εφαρμόζει τη λογική πράξη XOR(αποκλειστικό OR) στα bits των δύο τελεστέων και θέτει το bit εξόδου στο 1 μόνο αν τα αντίστοιχα bits και στους δύο τελεστέους είναι διαφορετικά μεταξύ τους. Αλλιώς, το bit εξόδου τίθεται στο 0. Για παράδειγμα, το αποτέλεσμα της πράξης 19 ^ 6 είναι 21:

```
        00010011    (19)
^       00000110    (6)
        ------------------------
        00010101    (21)
```

Ο τελεστής ^ χρησιμοποιείται συχνά σε εφαρμογές κρυπτογράφησης δεδομένων.

Tip Όταν εκτελούνται πράξεις με τελεστές bit είναι ασφαλέστερο οι αντίστοιχες μεταβλητές να δηλώνονται **unsigned**. Αν δεν τις δηλώσετε, να λαμβάνετε υπόψη σας το bit προσήμου όταν κάνετε τους υπολογισμούς σας.

Παράδειγμα 6

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει έναν ακέραιο που θα αποτελεί το κλειδί της κρυπτογράφησης και τον ακέραιο που θα κρυπτογραφηθεί. Η κρυπτογράφηση να γίνει εφαρμόζοντας τον τελεστή ^ μεταξύ τους. Στη συνέχεια το πρόγραμμα να χρησιμοποιεί πάλι τον τελεστή ^ για να αποκρυπτογραφήσει το κρυπτογραφημένο αποτέλεσμα και να το εμφανίσει στην οθόνη.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int num, key;
5.
6.      printf("Enter key: ");
7.      scanf("%d", &key);
8.
9.      printf("Enter number: ");
10.     scanf("%d", &num);
11.
12.     num = num ^ key;
13.     printf("Encrypted : %d\n", num);
14.     num = num ^ key;
15.     printf("Original: %d\n", num);
16.     return 0;
17. }
```

Σχόλια: Το αποτέλεσμα της αποκτυπτογράφησης είναι ο αριθμός που εισήγαγε ο χρήστης.

Παράδειγμα 7

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει δύο ακέραιους, να τους αποθηκεύει σε δύο μεταβλητές και να αντιμεταθέτει τις τιμές τους. Χρησιμοποιήστε τον τελεστή ^.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int num1, num2;
5.
6.      printf("Enter numbers: ");
7.      scanf("%d%d", &num1, &num2);
8.
9.      num1 = num1 ^ num2; /* Éóïäýíáìï ìå num1 ^= num2; */
10.     num2 = num1 ^ num2; /* Éóïäýíáìï ìå num2 ^= num1; */
11.     num1 = num1 ^ num2; /* Éóïäýíáìï ìå num1 ^= num2; */
12.     printf("Num1 = %d Num2 = %d\n", num1, num2);
13.     return 0;
14. }
```

Σχόλια: Ο πιο απλός τρόπος για να λύσουμε το πιο πάνω πρόβλημα είναι να χρησιμοποιήσουμε τρίτη μεταβλητή όπως κάναμε σε άλλα παραδείγματα. Ωστόσο, με τη χρήση του τελεστή ^ δεν χρειάζεται η τρίτη μεταβλητή. Η πάραπανω λύση βασίζεται στις ισότητες:

(a ^ b) ^ a = b και (a ^ b) ^ b =a



Επίσης, αν θέλετε να γράψετε πραγματικά δυσνόητο κώδηκα που ούτε εσείς οι ίδιοι δεν θα καταλαβαίνετε μετά από λίγο καιρό, πόσο μάλλον άλλοι αναγνώστες, θα μπορούσατε να χρησιμοποιήσετε τον συνδυαστικό τελεστή ^= και να γράψετε τις τρεις εντολές σε μια γραμμή ως εξής:

```
num1 ^= num2 ^= num1 ^= num2;
```

Αυτό είναι ένα παράδειγμα απαράδεκτου τρόπου γραφής κώδηκα.

Ο τελεστής Συμπληρώματος ~

Ο τελεστής Συμπληρώματος ~ είναι μοναδιαίος και εφαρμόζει τη λογική πράξη NOT στα bits του τελεστέου του. Συγκεκριμένα, αντιστρέφει κάθε bit του από 0 σε 1 και αντίστροφα. Για παράδειγμα το αποτέλεσμα της πράξης ~19 σε ένα 16-bit σύστημα είναι:

```
~ 00000000 00000000 00000000 00010011  (19) =
  11111111 11111111 11111111 11101100
```

Τελεστές Ολίσθησης
Οι τελεστές ολίσθησης μετατοπίζουν τα bits μιας ακέραιας μεταβλητής ή σταθεράς κατά έναν συγκεκριμένο αριθμό θέσεων. Ο τελεστής >> μετατοπίζει τα bits προς τα δεξιά και ο τελεστής << μετατοπίζει τα bits προς τα αριστερά, όπως δηλαδή δείχνουν τα «νοητά βέλη».

Tip: Επειδή η εφαρμογή των τελεστών ολίσθησης σε αρνητικούς αριθμούς εξαρτάται από τον μεταγλωττιστή, είναι ασφαλέστερο να τους χρησιμοποιείτε σε θετικούς ακέραιους ή **unsigned** μεταβλητές.

Ο τελεστής >>

Η έκφραση **i >> n** μετατοπίζει τα bits της μεταβλητής **i** κατά **n** θέσεις δεξιά και τοποθετεί **μηδενικά** στα **n** υψηλότερης τάξης bits της μεταβλητής. Για παράδειγμα, ποια θα είναι η τιμή της μεταβλητής **a** στον ακόλουθο κώδικα:

```
unsigned int a, b = 35;
a = b >> 2;
```

Αφού ο αριθμός **35** γράφεται ως **00100011** στο δυαδικό σύστημα, το αποτέλεσμα της πράξης 00100**11** >> 2 είναι **00**001000. Συγκεκριμένα χάθηκαν τα τελευταία bits **1** και **1** του αρχικού αριθμού και τοποθετήθηκαν τα bits **0** και **0** στην έβδομη και όγδοη θέση, αντίστοιχα.
Επομένως, η τιμή του **a** γίνεται **8**. Σημειώστε ότι η τιμή του **b** παραμένει η **35**.

Tip: Επειδή η θέση ενός bit αντιστοιχεί σε μια δύναμη του **2**, η ολίσθηση ενός θετικού ακεραίου **n** θέσεις δεξιά ισοδυναμεί με τη διαίρεση της τιμής του με 2^n.

Ο τελεστής <<

Η έκφραση **i << n** μετατοπίζει τα bits της μεταβλητής **i** κατά **n** θέσεις αριστερά και τοποθετεί **μηδενικά** στα **n** χαμηλότερης τάξης bits της μεταβλητής. Για παράδειγμα, ποια θα είναι η τιομή της μεταβλητής **a** στον ακόλουθο κώδικα:

```
unsigned int a, b = 35;
a = b << 2;
```

Αφού ο αριθμός **35** γράφεται ως **00100011** στο δυαδικό σύστημα, το αποτέλεσμα της πράξης 00100011 << 2 είναι 0010001**100**. Συγκεκριμένα τα bits του αρχικού αριθμού ολίσθησαν δύο θέσεις αριστερά και τοποθετήθηκαν τα bits **0** και **0** στην πρώτη και στη δεύτερη θέση, αντίστοιχα.

Επομένως, η τιμή του **a** γίνεται **140**. Σημειώστε ότι η τιμή του **b** παραμένει η **35**.

Tip: Η ολίσθηση ενός ακεραίου **n** θέσεις αριστερά ισοδυναμεί με τον πολλαπλασιασμό της τιμής του με 2^n.

Όταν χρησιμοποιείτε τον τελεστή << για να αποθηκεύσετε το αποτέλεσμα σε μια μεταβλητή, πρέπει να έχετε εξασφαλίσει ότι το αποτέλεσμα μπορεί να αποθηκευτεί στη μεταβλητή. Για παράδειγμα, δείτε το παρακάτω πρόγραμμα.

```
#include <stdio.h>
int main ()
{
        unsigned char a = 1;
        a <<= 8;          /* Ισοδύναμη με a = a << 8; */
        printf("Val = %d\n", a);
        return 0;
}
```

Η έκφραση **a <<= 8** ολισθαίνει την τιμή της μεταβλητής **a** οκτώ θέσεις αριστερά και τοποθετεί μηδενικά στα οκτώ χαμηλότερης τάξης bits. Δηλαδή, το αποτέλεσμα της ολίσθησης είναι **100000000**.

Αφού η μεταβλητή **a** έχει δηλωθεί σαν **unsigned char**, αποθηκεύεται σε αυτήν η τιμή που υπάρχει στα οκτώ τελευταία bits. Άρα, το πρόγραμμα θα εμφανίσει **Val = 0** και όχι **Val = 256**, όπως ίσως το περιμένατε.

Αν είχε δηλωθεί σαν **int**, το πρόγραμμα θα εμφάνιζε **Val = 256**.

Παράδειγμα 8

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
#include <stdio.h>
int main ()
{
        printf("%d\n", ~(~0 << 4));
        return 0;
}
```

Απάντηση: Σε ένα **32-bit** σύστημα η έκφραση **~0** αντιστοιχεί στον αριθμό που έχει **1** και τα 32 bits. Άρα, η τιμή της έκφρασης **(~0 << 4)** είναι ο αριθμός:

11111111 11111111 11111111 11110000

Αφού ο τλεστής ~ αντιστρέφει τα bits του αριθμού, το αποτέλεσμα της συνολικής έκφρασης γίνεται:

00000000 00000000 00000000 00001111

και το πρόγραμμα εμφανίζει **15**.

Παράδειγμα 9

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      unsigned int i = 10;
5.
6.      if  (( i >> 4 ) != 0 )
7.          printf((“One\n”);
8.      else
9.          printf((“Two\n”);
10.
11.     printf((“Val = %d\n”, i);
12.     i = 1;
13.     if  (( i << 3 ) == 8 )
14.         printf((“One\n”);
15.     else
16.         printf((“Two\n”);
17.
18.     printf((“Val = %d\n”, i);
19.     return 0;
20. }
```

Απάντηση: Αφού η τιμή του i (10_{10} = 00001010_2) ολισθημένη τέσσερις θέσεις δεξιά γίνεται 00000000_2, η πρώτη if εντολή εμφανίζει **Two**.

Σημειώστε ότι η **if** συνθήκη **δεν αλλάζει** την τιμή του **i** σε **0**, απλά ελέγχει αν η τιμή του **i** μετατοπισμένη τέσσερις θέσεις δεξιά είναι **0** ή όχι. Αφού η τιμή του **i** παραμένει **10**, το πρόγραμμα εμφανίζει στη συνέχεια **Val = 10**.

Η δεύτερη **if** εντολή ελέγχει αν η τιμή του **i** ολισθημένη τρεις θέσεις αριστερά είναι ίση με **8**. Αφού η τιμή του 1_{10} = 00000001_2 ολισθημένη τρεις θέσεις αριστερά γίνεται 00001000_2 = 8_{10} το πρόγραμμα εμφανίζει **One**.

Η τιμή του **i** παραμένει αμετάβλητη άρα το πρόγραμμα εμφανίζει **Val = 1**.

Παράδειγμα 10

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.       char a = 8;
5.
6.      a <<= 4;
7.      printf("Val = %d\n", a);
8.      return 0;
9.  }
```

Απάντηση: Η έκφραση a <<= 4 ολισθαίνει την τιμή της μεταβλητής **a** τέσσερις θέσεις αριστερά και την κάνει 10000000.

Η μεταβλητή **a** έχει δηλωθεί σαν **char**, επομένως επομένως το bit στη θέση 8 δηλώνει το πρόσημο του αριθμού. Αν η τιμή του είναι 1 ο αριθμός είναι αρνητικός, αλλιώς είναι θετικός. Επομένως, το πρόγραμμα θα εμφανίσει Val = -128. Αν αντί για **char a;** δηλώναμε τη μεταβλητή σαν **unsigned char**, τότε το bit 8 δεν θα δήλωνε το πρόσημο και το πρόγραμμα θα εμφάνιζε Val = 128.

Να θυμάστε ότι είναι ασφαλέστερο να χρησιμοποιείτε τους bit τελεστές σε **unsigned** μεταβλητές.

Παράδειγμα 11

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει έναν ακέραιο και να εμφανίζει ένα μήνυμα για το αν είναι άρτιος ή περιττός.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int num;
5.
6.      printf("Enter number: ");
7.      scanf("%d", &num);
8.
9.      if (num & 1)
10.         printf("The number %d is odd\n", num);
11.     else
12.         printf("The number %d is even\n", num);
13.     return 0;
14. }
```

Σχόλια: Για να διαπιστώσουμε αν ένας αριθμός είναι άρτιος ή περιττός ελέγχουμε το τελευταίο bit του αριθμού. Αν είναι **0**, ο αριθμός είναι άρτιος, αλλιώς είναι περιττός. Αυτός ο έλεγχος γίνεται με την εντολή **if (num & 1)**. Θεωρείστε ότι ο ακέραιος **num** αναπαρίσταται στο δυαδικό σύστημα ως εξής:

xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx

όπου το κάθε 'x' αντιστοιχεί σε ένα bit με τιμή 0 ή 1. Άρα, το αποτέλεσμα της πράξης **num & 1** είναι:

```
        xxxxxxxx xxxxxxxx xxxxxxxx xxxxxxxx
&       00000000 00000000 00000000 00000001
        -----------------------------------------------
        00000000 00000000 00000000 0000000x
```

Δηλαδή, είναι ίσο με την τιμή που έχει το τελευταίο bit του ακεραίου num. Αν είναι 1, η συνθήκη στην **if** εντολή είναι αληθής και εμφανίζεται το μήνυμα ότι ο αριθμός είναι περιττός. ΑΝ είναι 0, η συνθήκη είναι ψευδής και εμφανίζεται το μήνυμα ότι ο αριθμός είναι άρτιος.

Παράδειγμα 12

Οι τελεστές bit και οι τελεστές ολίσθησης χρησιμοποιούνται πολύ συχνά σε εφαρμογές κωδικοποίησης δεδομένων σε δίκτυα επικοινωνιών. Για παράδειγμα, η επικεφαλίδα ενός TCP πακέτου περιέχει μια οκτάδα από bits, όπου κάποια συγκεκριμένα bits κωδικοποιούνται ως εξής:

bit8	bit7	bit6	bit5	bit4	bit3	bit2	bit1
		URG	ACK			SYN	FIN

FIN (bit1): Όταν είναι 1 δηλώνει την αποδέσμευση της σύνδεσης.
SYN (bit2): Όταν είναι 1 δηλώνει την εγκατάσταση της σύνδεσης.
ACK (bit5): Όταν είναι 1 επιβεβαιώνει τη λήψη δεδομένων.
URG (bit6): Όταν είναι 1 δηλώνει τη μεταφορά επειγόντων δεδομένων.

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει τις τιμές των URG, ACK, SYN και FIN bits και να κωδικοποιεί αυτή την πληροφορία σε μια μεταβλητή του προγράμματος. Στη συνέχεια, να αποδικωποιείται το αποτέλεσμα και να εμφανίζονται οι τιμές των αντίστοιχων bits.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      unsigned int temp, urg, ack, syn, fin;
5.      printf("Enter FIN bit: ");
6.      scanf("%d", &fin);
7.      printf("Enter SYN bit: ");
8.      scanf("%d", &syn);
9.
10.     printf("Enter ACK bit: ");
11.     scanf("%d", &ack);
12.
13.     printf("Enter URG bit: ");
14.     scanf("%d", &urg);
15.
16.     temp = fin + (syn << 1) + (ack << 4) + (urg << 5);
17.     printf("\nEncoding: %d\n", temp);
18.
19.     fin = temp & 1;
20.     syn = (temp >> 1) & 1;
21.     ack = (temp >> 4) & 1;
22.     urg = (temp >> 5) & 1;
23.     printf("FIN = %d, SYN = %d, ACK = %d, URG = %d\n", fin, syn, ack, urg);
24.     return 0;
25. }
```

Παράδειγμα 13

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει έναν ακέραιο στο [0, 255], να αντιμεταθέτει τις δύο τετράδες των δυαδικών ψηφίων του και να εμφανίζει τον νέο αριθμό. Για παράδειγμα, αν ο χρήστης εισάγει τον αριθμό $10_{10} = 00001010_2$, το πρόγραμμα να εμφανίζει τον αριθμό $10100000_2 = 160_{10}$.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      unsigned int num, temp;
5.
6.      printf("Enter number [0-255]: ");
7.      scanf("%d", &num);
8.
9.      if (num >= 0 && num <= 255)
10.     {
11.         temp = num & 0xF; /* Στη μεταβλητή temp αποθηκεύεται η τιμή των
12.                              τεσσάρων χαμηλότερων δυαδικών ψηφίων του
13.                              ακέραιου num.  Το 0 X F αντιστοιχεί στο δυαδικό
14.                              00001111. */
15.         temp <<= 4;         /* Ολίσθηση της τιμής τέσσερις θέσεις αριστερά. */
16.         temp += num >> 4; /* Ολίσθηση του ακέραιου num τέσσερις θέσεις δεξιά
17.                              και πρόσθεση αυτής της τιμής στη temp. */
18.         printf("Num_1 = %d  Num_2 = %d\n", num, temp);
19.     }
20.     else
21.         printf("Number should be in [0-255]\n");
22.     return 0;
23. }
```

Παράδειγμα 14

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει έναν ακέραιο και τη θέση ενός bit στον αριθμό και να εμφανίζει την τιμή του συγκεκριμένου bit.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      unsigned int num, pos;
5.
6.      printf("Enter number: ");
7.      scanf("%d", &num);
8.
9.      printf("Enter bit position [1-32]: ");
10.     scanf("%d", &pos);
11.     if (pos >= 1 && pos <= 32)
12.         printf("The value of bit%d is %d\n", pos, (num >> (pos-1)) & 1);
13.     else
14.         printf("Bit position should be in [1-32]\n");
15.     return 0;
16. }
```

Παράδειγμα 15

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      unsigned char ch = 3;
5.      ch = (( ch & 1 ) << 7 ) | (( ch & 2 ) << 5 ) | (( ch & 4 ) << 3 ) |
6.           (( ch & 8 ) << 1 ) | (( ch & 16 ) >> 1 ) | (( ch & 32 ) >> 3 ) |
7.           (( ch & 64 ) >> 5 ) | (( ch & 128 ) >> 7 );
8.      printf("%d\n", ch);
9.      return 0;
10. }
```

Απάντηση: Ας υποθέσουμε ότι η τιμή της **ch** αναπαρίσταται στο δυαδικό σύστημα σαν $X_8\ X_7\ X_6\ X_5\ X_4\ X_3\ X_2\ X_1$, όπου η τιμή του κάθε bit X_i είναι **1** ή **0**. Άρα, η τιμή της έκφρασης (ch & 1) << 7 είναι:

$$
\begin{array}{ll}
 & X_8\ X_7\ X_6\ X_5\ X_4\ X_3\ X_2\ X_1 \\
\& & 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1 \\
 & \text{---------------------------------} \\
 & 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ X_1 << 7 = X_1\ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0
\end{array}
$$

Παρόμοια, η τιμή της έκφρασης (ch & 2) << 5 είναι:

$$
\begin{array}{ll}
 & X_8\ X_7\ X_6\ X_5\ X_4\ X_3\ X_2\ X_1 \\
\& & 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 1\ \ 0 \\
 & \text{---------------------------------} \\
 & 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ X_2\ 0 << 5 = 0\ \ X_2\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0
\end{array}
$$

Η τιμή της έκφρασης (ch & 4) << 3 είναι: $0\ \ 0\ \ X_3\ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ \ 0$
Η τιμή της έκφρασης (ch & 8) << 1 είναι: $0\ \ 0\ \ \ 0\ \ X_4\ \ 0\ \ \ 0\ \ \ 0\ \ \ 0$
Η τιμή της έκφρασης (ch & 16) >> 1 είναι: $0\ \ 0\ \ \ 0\ \ \ 0\ \ X_5\ \ 0\ \ \ 0\ \ \ 0$
Η τιμή της έκφρασης (ch & 32) >> 3 είναι: $0\ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ X_6\ \ 0\ \ \ 0$
Η τιμή της έκφρασης (ch & 64) >> 5 είναι: $0\ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ X_7\ \ 0$
Η τιμή της έκφρασης (ch & 128) >> 7 είναι: $0\ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ \ 0\ \ X_8$

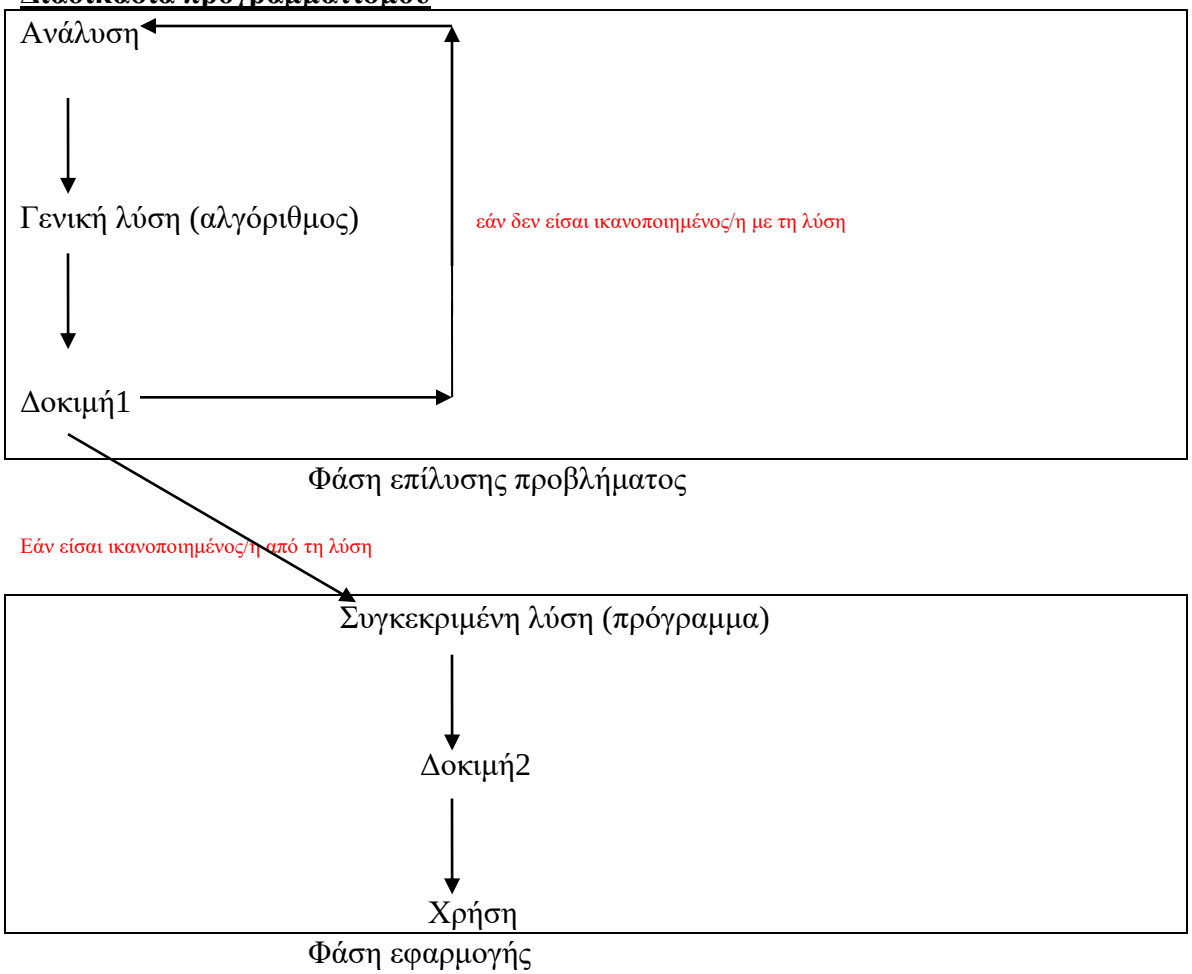
Επομένως, η συνολική έκφραση ισοδυναμή με:

$$
\begin{array}{l}
(X_1\ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0) \mid (0\ \ X_2\ \ 0\ 0\ \ 0\ \ 0\ \ 0\ \ 0) \mid (0\ 0\ \ X_3\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0) \mid \\
(0\ \ 0\ \ 0\ \ X_4\ \ 0\ \ 0\ \ 0\ \ 0) \mid (0\ \ 0\ \ 0\ \ 0\ \ X_5\ \ 0\ \ 0\ \ 0) \mid (0\ \ 0\ \ 0\ \ 0\ \ 0\ \ X_6\ \ 0\ 0) \mid \\
(0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ X_7\ \ 0) \mid (0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ 0\ \ X_8) = X_1\ X_2\ X_3\ X_4\ X_5\ X_6\ X_7\ X_8
\end{array}
$$

Δηλαδή, το συγκεκριμένο πρόγραμμα αντιστρέφει τα bits της μεταβλητής **ch**, χωρίς να χρησιμοποιηθεί δεύτερη μεταβλητή. Άρα, η τιμή της **ch** (00000011_2) γίνεται 192 (11000000_2) και το πρόγραμμα εμφανίζει 192.

Αλγόριθμοι (Ψευδό-κώδικας)

Διαδικασία προγραμματισμού



Ο αλγόριθμος είναι μια βήμα-προς-βήμα διαδικασία για να λύνουμε προβλήματα.

Ανάλυση: Καταλαβαίνουμε το πρόβλημα.
Δοκιμή1: Ακολουθούμε τα βήματα του αλγορίθμου για να δούμε εάν αυτός μας δίνει σίγουρη λύση στο πρόβλημα.
Πρόγραμμα: Μεταφράζουμε τον αλγόριθμο σε γλώσσα προγραμματισμού (κώδικας).
Δοκιμή2: Αφήνουμε τον Η/Υ να τρέξει το πρόγραμμα. Κάνουμε διορθώσεις.
Χρήση: Χρησιμοποιούμε το πρόγραμμα.

Η λύση για κάθε πρόβλημα στον Η/Υ περιλαμβάνει την εκτέλεση μιας ομάδας ενεργειών με συγκεκριμένη σειρά. Η διαδικασία της λύσης του προβλήματος από την άποψη των:

1. Οι ενέργειες που θα εκτελεσθούν, και
2. Η σειρά που θα εκτελεσθούν

ονομάζεται αλγόριθμος.

Πρόβλημα "rise and shine"
Αλγόριθμος

Βήμα	Οδηγία
1	Σήκω από το κρεβάτι
2	Βγάλε τις πυτζάμες σου
3	Κάνε μπάνιο
4	Ντύσου
5	Φάγε το πρόγευμα σου
6	Οδήγα για τη δουλειά

Φανταστείτε εάν αλλάζετε τη σειρά των πράξεων σας. Θα σου έλυνε το πρόβλημα;

Πρόβλημα "start-a-car"
Αλγόριθμος

Βήμα	Οδηγία
1	Βάλε το κλειδί στη θέση του
2	Βεβαιώσου ότι η ταχύτητα είναι στο ουδέτερο
3	Πάτησε λίγο το πετάλι της βενζίνας
4	Γύρισε το κλειδί για ξεκίνημα του αυτοκινήτου
5	Εάν η μηχανή ξεκινήσει μέσα σε 6 δευτερόλεπτα, άφησε το κλειδί
6	Εάν η μηχανή δεν ξεκινήσει μέσα σε 6 δευτερόλεπτα, περίμενε για 10 δευτερόλεπτα και επανάλαβε τα βήματα 3 μέχρι 6
7	Εάν το αυτοκίνητο δεν ξεκινά τηλεφώνησε στο μηχανικό

Πρόβλημα "Determine wages for the week"
Αλγόριθμος

1	Βρες το επίδομα του υπαλλήλου ανά ώρα
2	Προσδιόρισε τις ώρες που εργάστηκε αυτή την εβδομάδα
3	Εάν ο αριθμός των ωρών είναι <= 40 πολλαπλασίασε τον αριθμό των ωρών επί το επίδομα ανά ώρα για να πάρεις τον κανονικό μισθό
4	Εάν ο αριθμός των ωρών είναι > 40 πολλαπλασίασε το επίδομα ανά ώρα επί το 40 για να πάρεις τον κανονικό μισθό και πολλαπλασίασε επί 1 ½ το επίδομα ανά ώρα επί τη διαφορά από τις ώρες που εργάστηκε και του 40 για να υπολογίσεις τις υπερωρίες
5	Πρόσθεσε τον κανονικό μισθό και τις υπερωρίες

Όταν ο προγραμματιστής είναι ικανοποιημένος με τον αλγόριθμο, ο αλγόριθμος μεταφράζεται σε γλώσσα προγραμματισμού, η οποία όπως έχουμε πει είναι ένα σύνολο κανόνων, συμβόλων, και ειδικών λέξεων που χρησιμοποιούνται για να μας δημιουργήσουν ένα πρόγραμμα.

Πρόβλημα "Find the weighted average"
Αλγόριθμος

Πρόβλημα: Βρείτε τον σταθμισμένο μέσο όρο αποτελεσμάτων εξετάσεως. Τα δεδομένα θα είναι με τη μορφή ενός ακεραίου (integer) βαθμού ακολουθούμενο από το αντίστοιχο βάρος, με το κάθε ζευγάρι σε διαφορετική γραμμή.

Συζήτηση: Είναι κοινός γνωστό να δίνονται διαφορετικά βάρη σε διαγωνίσματα για να καταλήγουμε στο τελικό βαθμό του φοιτητή για κάθε μάθημα. Για παράδειγμα, εάν δύο διαγωνίσματα είχαν βάρος 30% το κάθε ένα και η τελική εξέταση άξιζε 40%, θα παίρναμε το βαθμό του πρώτου διαγωνίσματος και θα το πολλαπλασιάζαμε επί 0.30, θα παίρναμε το βαθμό του δεύτερου διαγωνίσματος και θα το πολλαπλασιάζαμε επί 0.30, και θα παίρναμε το βαθμό της τελικής εξέτασης και θα το πολλαπλασιάζαμε επί 0.40. Μετά θα προσθέταμε αυτές τις τρεις τιμές για να καταλήξουμε στον σταθμισμένο μέσο όρο. Θα γράψουμε τον αλγόριθμο για να λύσουμε το αυτό πρόβλημα.

Είσοδος (input): Τρεις ομάδες δεδομένων, κάθε στοιχείο του

exam score (integer) weight (double(real))

Έξοδος (output): Τυπώστε τα δεδομένα εισόδου με επικεφαλίδες (echo printing). Τυπώστε τον σταθμισμένο μέσο όρο με εξηγήσεις.
Θεωρήσεις: Το άθροισμα των τριών βαρών διαγωνισμάτων είναι 1.00, και τα δεδομένα εισόδου είναι σωστά (δεν χρειάζεται έλεγχος λαθών).

Η μέθοδος **top-down** (μέθοδος για γραφή ενός αλγορίθμου) μπορεί να σπάσει ως ακολούθως (χρησιμοποιούμε τη μέθοδο διαίρει και βασίλευε):

1. Ανάλυσε το πρόβλημα.
2. Γράψε τη κυρία ενότητα.
3. Γράψε τις υπόλοιπες ενότητες.
4. Άλλαξε τη σειρά των ενοτήτων και αναθεώρησε με τις ανάγκες.

ΚΥΡΙΑ ΕΝΟΤΗΤΑ Επίπεδο 0

```
Πάρε τα δεδομένα
Τύπωσε τα αποτελέσματα
Βρες το μέσο όρο
Τύπωσε το μέσο όρο
```

ΠΑΡΕ ΤΑ ΔΕΔΟΜΕΝΑ Επίπεδο 1

```
Τύπωσε "Enter first exam score and weight."
Διάβασε exam1, weight1
Τύπωσε "Enter second exam score and weight."
Διάβασε exam2, weight2
Τύπωσε "Enter third exam score and weight."
Διάβασε exam3, weight3
```

ΤΥΠΩΣΕ ΤΑ ΑΠΟΤΕΛΕΣΜΑΤΑ

Τύπωσε την επικεφαλίδα Τύπωσε exam1, weight1 Τύπωσε exam2, weight2 Τύπωσε exam3, weight3

ΒΡΕΣ ΤΟ ΜΕΣΟ ΟΡΟ

ave = exam1 * weight1 + exam2 * weight2 + exam3 * weight3

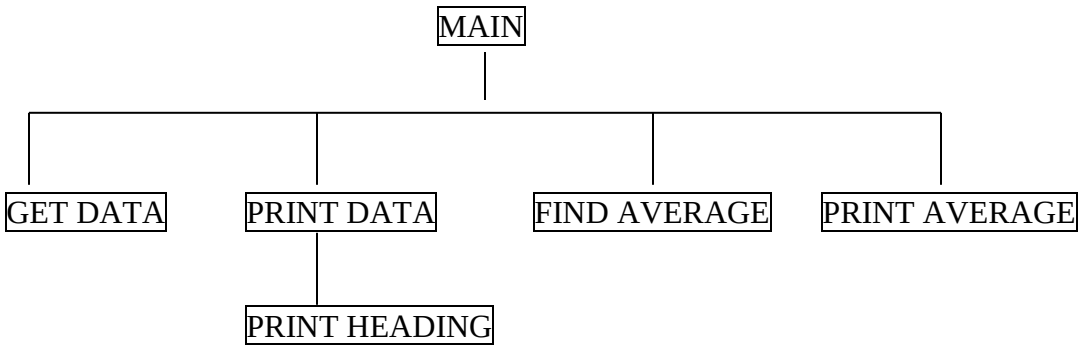
ΤΥΠΩΣΕ ΤΟ ΜΕΣΟ ΟΡΟ

Τύπωσε “Weighted average is “, ave

ΤΥΠΩΣΕ ΤΗΝ ΕΠΙΚΕΦΑΛΙΔΑ Επίπεδο 2

Τύπωσε “Exam score Weight”

MODULE STRUCTURE CHART



ΛΙΣΤΑ ΜΕΤΑΒΛΗΤΩΝ ΠΟΥ ΠΡΟΕΡΧΟΝΤΑΙ ΑΠΟ ΤΟΝ ΑΛΓΟΡΙΘΜΟ

Όνομα	Τύπος δεδομένων	Χρήση
Exam1	Integer	Βαθμός για πρώτο διαγώνισμα
Exam2	Integer	Βαθμός για δεύτερο διαγώνισμα
Exam3	Integer	Βαθμός για τρίτο διαγώνισμα
Weight1	Double	Βάρος για πρώτο διαγώνισμα
Weight2	Double	Βάρος για δεύτερο διαγώνισμα
Weight3	Double	Βάρος για τρίτο διαγώνισμα
Ave	Double	Σταθμισμένος μέσος όρος

Η εντολή δομής επιλογής IF

Ψευδό-κώδικας

If student's grade is greater than or equal to 60
    Print Passed

C κώδικας

```
If ( grade >= 60 )
    printf( "Passed\n" );
```

Η δομή if/else

Ψευδό-κώδικας

If student's grade is greater than or equal to 60
    Print "Passed"
Else
    Print "Failed"

C κώδικας

```
If ( grade >= 60 )
    printf( "Passed\n" );
else
    printf( "Failed\n" );
```

Η C μας παρέχει επίσης τον υποθετικό τελεστή (conditional operator) (?:) ο οποίος συνδέεται στενά με τη δομή if/else.

```
    printf( "%s\n",grade >= 60? "Passed":"Failed" );
```

Ένθετο (nested) if

Χρησιμοποιείτε σε περιπτώσεις πολλαπλών δομών if/else μέσα σε άλλες δομές if/else.

Ψευδό-κώδικας

```
If student's grade is greater than or equal to 90
        Print "A"
Else
        If student's grade is greater than or equal to 80
                Print "B"
        Else
                If student's grade is greater than or equal to 70
                        Print "C"
                Else
                        If student's grade is greater than or equal to 60
                                Print "D"
                        Else
                                Print "F"
```

C κώδικας

```
if ( grade >= 90 )
        printf( "A\n" );
else
        if ( grade >= 80 )
                printf( "B\n" );
        else
                if ( grade >= 70 )
                        printf( "C\n" );
                else
                        if ( grade >= 60 )
                                printf( "D\n" );
                        else
                                printf( "F\n" );
```

If με σύνθετη (compound) εντολή (Μπλοκ εντολών)

```
If ( grade >= 60 )
        printf( "Passed.\n" );
else  {
        printf( "Failed.\n" );
        printf( "You must take this course again.\n" );
}
```

Λογικοί τελεστές

Οι λογικοί τελεστές είναι **&&** (and), || (or) και **!** (not).

AND (&&)

```
if ( gender == 1 && age >= 65 )
        ++seniorFemales;
```

παράσταση1	παράσταση2	παράσταση1 && παράσταση2
0 (false)	0 (false)	0 (false)
0 (false)	nonzero (true)	0 (false)
nonzero (true)	0 (false)	0 (false)
nonzero (true)	nonzero (true)	1 (true)

Η C θέτει και δέχεται το **0** για το **false**. Αν και η C θέτει και δέχεται το **1** για το **true**, δέχεται επίσης οποιαδήποτε **nonzero** τιμή για το **true**.

OR (||)

```
if ( semesteraAverage >= 90 || finalExam >= 90 )
        printf( "Student grade is A\n" );
```

παράσταση1	παράσταση2	παράσταση1 || παράσταση2
0 (false)	0 (false)	0 (false)
0 (false)	nonzero (true)	1 (true)
nonzero (true)	0 (false)	1 (true)
nonzero (true)	nonzero (true)	1 (true)

NOT (!)

```
if ( !( grade == sentinelValue ) )
        printf( "The next grade is %f\n", grade );
```

Αυτό μπορεί επίσης να γραφτεί σαν :

```
if (  grade != sentinelValue )
        printf( "The next grade is %f\n", grade );
```

παράσταση	!παράσταση
0 (false)	1 (true)
nonzero (true)	0 (false)

Προαιρετική ιδιότητα τελεστών

Τελεστής	Προαιρετική ιδιότητα	Είδος
()	αριστερά στα δεξιά	παρενθέσεις
++ -- + - ! (type)	δεξιά στα αριστερά	μοναδιαίοι
* / %	αριστερά στα δεξιά	πολλαπλασιαστικά
+ -	αριστερά στα δεξιά	πρόσθετης ύλης
< <= > >=	αριστερά στα δεξιά	σχεσιακά
== !=	αριστερά στα δεξιά	ισότητας
&&	αριστερά στα δεξιά	λογικό AND
||	αριστερά στα δεξιά	λογικό OR
?:	δεξιά στα αριστερά	συνθήκη
= += -= *= /= %=	δεξιά στα αριστερά	ανάθεση
,	αριστερά στα δεξιά	κόμμα

Προβλήματα που μπορείτε να αντιμετωπίσετε

1.
Κάθε παράσταση στη C που παράγει μια τιμή μπορεί να χρησιμοποιηθεί στη συνθήκη κάθε δομής ελέγχου. Εάν αυτή η τιμή είναι 0, αντιμετωπίζεται ως false και εάν η τιμή δεν είναι 0 αντιμετωπίζεται σαν true. Επίσης, η ανάθεση στη C παράγει μια τιμή, δηλαδή η τιμή που αντιστοιχίζεται στη μεταβλητή που βρίσκεται στα αριστερά του τελεστή ανάθεσης. Για παράδειγμα, ας υποθέσουμε ότι έχουμε την πρόθεση να γράψουμε

```
if ( payCode == 4 )
        printf( "You get a bonus! " );
```

αλλά κατά λάθος γράφουμε

```
if ( payCode = 4 )
        printf( "You get a bonus! " );
```

Το πρώτο είναι σωστό. Εάν το **payCode** είναι **4** τότε τυπώνουμε το μήνυμα.

Το δεύτερο είναι σωστό από την άποψη του μεταγλωττιστή έτσι ώστε έχουμε λάθος αποτελέσματα για τον λόγο ότι η εντολή ανάθεσης **payCode = 4** όταν εκτελεσθεί η μεταβλητή **payCode** παίρνει τη τιμή **4** η οποία είναι **nonzero** και ως εκ η συνθήκη θα είναι πάντα **true**.

2.
Ας υποθέσουμε ότι ο προγραμματιστής θέλει να αντιστοιχίσει μια τιμή σε μια μεταβλητή με μια απλή εντολή όπως

x = 1;

αλλά αντί αυτού γράφει

x == 1;

Εδώ, επίσης, δεν είναι συντακτικό λάθος. Ο μεταγλωττιστής αξιολογεί τη συνθήκη. Είτε η τιμή **0** ή τιμή **1** επιστρέφεται ανάλογα με τη τιμή του **x**. Δεν υπάρχει καμία ανάθεση τιμής που αυτή χάνεται και μας οδηγεί σε λογικό λάθος.

Η επαναληπτική εντολή (βρόχος-loop) while

Ψευδό-κώδικας

While there are more items on my shopping list
    Purchase next item and cross it off my list

C κώδικας

```
product = 2;
while ( product <= 1000 )
    product = 2 * product;
```

Ψευδό-κώδικας

Initialize total to zero
Initialize counter to zero

Input the first grade
While the user has not as yet entered the sentinel
    Add this grade into the running total
    Add one to grade counter
    Input the next grade (possibly the sentinel)
If the counter is not equal to zero
    Set the average to the total divided by the counter
    Print the average
Else
    Print “No grades were entered”

Παράδειγμα 16

```
1   /* Class average program with
2      sentinel controlled repetition */
3
4   #include <stdio.h>
5
6   int main()
7   {
8       float average;          /* new data type */
9       int counter, grade, total; /*integer: -32000 mexri 32000*/
10
11      /* initialization phase */
12      total = 0;
13      counter = 0;
14
15      /* processing phase */
16      printf( "Enter grade, -1 to end: " );
17      scanf( "%d", &grade );  /*δοσμου 1 ακεραιο& αποθηκευω το στο grade*/
18
19      while (grade != -1)  {
20          total = total + grade;
21          counter =counter + 1;
22          printf( "Enter grade, -1 to end: " );
23          scanf( "%d", &grade );
24      }
25      /* termination phase */
26      if ( counter != 0 )  {
27          average = ( float ) total / counter;
28          printf( "Class average is %.2f", average );
29      }
30      else
31          printf( "No grades were entered\n" );
32      return 0;       /* indicate program ended successfully */
33  }
```

ΕΞΟΔΟΣ:

```
Enter grade, -1 to end: 75
Enter grade, -1 to end: 94
Enter grade, -1 to end: 97
Enter grade, -1 to end: 88
Enter grade, -1 to end: 70
Enter grade, -1 to end: 64
Enter grade, -1 to end: 83
Enter grade, -1 to end: 89
Enter grade, -1 to end: -1
Class average is 82.50
```

ΓΡΑΜΜΗ 27: Για την παραγωγή ενός κινητής υποδιαστολής (floating-point) υπολογισμού με ακέραιες τιμές, πρέπει να δημιουργήσουμε προσωρινές τιμές που είναι κινητής υποδιαστολής αριθμοί για τον υπολογισμό. Η C παρέχει το μοναδιαίο τελεστή cast για να εκπληρώσει την αποστολή αυτή. *average = (float) total / counter;*
ΓΡΑΜΜΗ 28: Το **f** καθορίζει ότι μια δεκαδική τιμή θα εκτυπωθεί. Το **.2** είναι η ακρίβεια με την οποία αυτή η τιμή θα εμφανίζεται.

```
printf( "%.2f\n",3.446 );        /* prints 3.45 */
printf( "%.1f\n",3.446 );        /* prints 3.4 */
```

Εάν χρησιμοποιήσουμε μόνο το %f – χρησιμοποιείται το προεπιλεγμένο 6 δεκαδικών.

Τελεστές ανάθεσης (Assignment operators)

c = c + 3;
μπορεί να γραφτεί συντετμημένο με την προσθήκη του τελεστή ανάθεσης της πρόσθεσης **+=**, όπως

c += 3;

στη θέση του + μπορεί να είναι **+**, **-**, *****, / ή **%**

Παράδειγμα 17

```
1.  /* Analysis of examination results.
2.     Use of assignment      statement */
3.  #include <stdio.h>
4.  int main()
5.  {
6.       /* initializing variables in declarations */
7.       int passes = 0, failures = 0, student =1, result;
8.
9.       /* process 10 students; counter-controlled loop */
10.     while ( student <= 10)  {
11.          printf( "Enter result (1 = pass, 2 = fail): " );
12.          scanf( "%d", &result );
13.
14.          if ( result == 1 )  /* if/else nested in while */
15.             passes = passes + 1;
16.          else
17.            failures = failures + 1;
18.
19.          student = student + 1;
20.       }
21.    printf( "Passed %d\n", passes );
22.    printf( "Failed %d\n", failures );
23.    if ( passes >8 )
24.          printf( "Raise tuition\n" );
25.    return 0;          /* indicate program ended successfully */
26. }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 2
Enter Result (1 = pass, 2 = fail): 2
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 2
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 2
Passed 6
Failed 4
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 2
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Enter Result (1 = pass, 2 = fail): 1
Passed 9
Failed 1
Raise tuition
```

Τελεστής Ανάθεσης	Δείγμα παράστασης	Εξήγηση	Ανάθεση

Assume: *int c = 3, d = 5, e = 4, f = 6, g = 12;*

+=	c += 7	c = c + 7	10 στο c
-=	d -= 4	d = d – 4	1 στο d
*=	e *= 5	e = e * 5	20 στο e
/=	f /= 3	f = f / 3	2 στο f
%=	g %= 9	g = g % 9	3 στο g

Τελεστές αύξησης και μείωσης

Τελεστής	Δείγμα παράστασης	Εξήγηση
++	++a	Αυξάνει το **a** κατά **1** και μετά χρησιμοποιεί τη νέα τιμή του **a** μέσα στη παράσταση που βρίσκεται το **a**.
++	a++	Χρησιμοποιεί τη τρέχουσα τιμή του **a** μέσα στη παράσταση που βρίσκεται το **a**, και μετά αυξάνει το **a** κατά **1**.
--	--b	Μειώνει το **b** κατά **1** και μετά χρησιμοποιεί τη νέα τιμή του **b** μέσα στη παράσταση που βρίσκεται το **b**.
--	b--	Χρησιμοποιεί τη τρέχουσα τιμή του **b** μέσα στη παράσταση που βρίσκεται το **b**, και μετά μειώνει το **b** κατά **1**.

Παράδειγμα 18

```
1.  /* Preincrementing and postincrementing */
2.  #include <stdio.h>
3.
4.  int main()
5.  {
6.      int c = 5;
7.
8.      printf( "%d\n", c );
9.      printf( "%d\n", c++ );          /* postincrement */
10.     printf( "%d\n\n", c );
11.
12.     c = 5;
13.     printf( "%d\n", c );
14.     printf( "%d\n", ++c );          /* preincrement */
15.     printf( "%d\n", c );
16.
17.     return 0;                       /* successful termination */
18. }
```

ΕΞΟΔΟΣ:

```
5
5
6

5
6
6
```

Έλεγχος προγράμματος με τη C

Τα περισσότερα προγράμματα περιλαμβάνουν επανάληψη ή με διαφορετικά λόγια το looping. Το loop (βρόχος) είναι μια ομάδα από εντολές που ο υπολογιστής εκτελεί επανειλημμένα ωσότου εξακολουθεί να ισχύει κάποια συνθήκη.

Επανάληψη με έλεγχο μετρητή

Αυτό απαιτεί:

1. Το **όνομα** της μεταβλητής-μετρητής (ή loop counter).
2. Την **αρχική τιμή** της μεταβλητής-μετρητής.
3. Την **αύξηση** ή **μείωση** από την οποία η μεταβλητή ελέγχου τροποποιείται κάθε φορά μέσα από το βρόγχο.
4. Την συνθήκη που ελέγχει για την τελική τιμή της μεταβλητής ελέγχου (δηλαδή, αν ο βρόχος πρέπει να συνεχίσει).

Παράδειγμα 19

```
1.  /* Counter-controlled repetition */
2.  #include <stdio.h>
3.
4.  int main()
5.  {
6.      int counter = 1;              /* initialization */
7.
8.      while ( counter <= 10 )   {   /* repetition condition */
9.          printf( "%d\n", counter );
10.         ++counter;                /* increment */
11.      }
12. return 0;
13. }
```

ΕΞΟΔΟΣ:

```
1
2
3
4
5
6
7
8
9
10
```

Σε αυτό το παράδειγμα θα μπορούσατε κανονικά να κάνετε αυτό το πρόγραμμα πιο συνοπτικό δίνοντας στο **counter** το **0** και αλλάζοντας τη δομή **while** με

```
while ( ++counter <= 10 )
    printf("%d\n", counter);
```

Η δομή επανάληψης for

Παράδειγμα 20

```
1.  /* Counter-controlled repetition
2.     with the for structures */
3.  #include<sdtio.h>
4.
5.  int main()
6.  {
7.        int counter;
8.
9.        /* initialization, repetition condition, and increment
10.          are all included in the for structure header */
11.       for ( counter = 1; counter <= 10; counter++ )
12.             printf( "%d\n", counter );
13.
14.       return 0;
15.   }
```

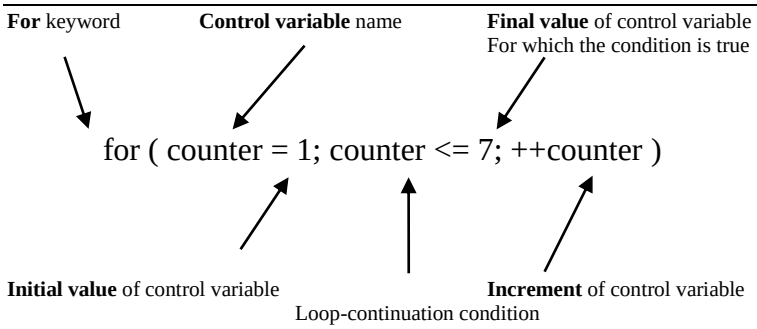
Όταν η δομή **for** ξεκινά να εκτελεί, δίνεται η τιμή 1στον μετρητή (**controlled variable counter**). Μετά, η **συνθήκη του βρόχου** (**loop-continuation condition**) **counter <= 10** ελέγχεται. Για το λόγο ότι η αρχική τιμή είναι **1** η συνθήκη ικανοποιείτε, έτσι η εντολή **printf** εκτυπώνει τη τιμή του **counter**, δηλαδή **1**.Μετά η τιμή του **counter** αυξάνεται κατά **1** από τη παράσταση **counter++**, και ο βρόχος ξεκινά ξανά να εκτελεί με τον έλεγχο της συνθήκης. Αυτή η διαδικασία συνεχίζεται μέχρι η ελεγχόμενη μεταβλητή αυξάνεται στην τελική τιμή **11**.

Η γενική μορφή του **for** είναι:

```
for ( expression1; expression2; expression3 )
        statement;
```

με τη δομή **while** μπορεί να γραφτεί σαν:

```
expression1;
while ( expression2 )  {
        statement;
        expression3;
}
```



Η αύξηση της μεταβλητής ελέγχου μπορεί να γίνει με

```
counter = counter + 1
counter += 1
++counter
counter++
```

Παραδείγματα της εντολής for:

Εάν **x = 2** and **y = 10** τότε το ακόλουθο

1. *for (j = x; j <= 4 * x * y; j += y / x)*

είναι το ίδιο με

for (j = 2; j <= 80; j +=5)

2. *for (i = 1; i <= 100; i++)*
3. *for (i = 100; i >= 1; i--)*
4. *for (i = 7; i <= 77; i += 7)*
5. *for (i=20; i >= 2; i -= 2)*
6. *for (j=2; j <= 20; j += 3)*
7. *for (j = 99; j >= 0; j -= 11)*

Παράδειγμα 21

```
1.      /* Summation with for */
2.
3.      #include <stdio.h>
4.
5.      int main()
6.      {
7.          int sum = 0, number;
8.
9.      for ( number = 2; number <= 100; number += 2 )
10.         sum += number;
11.
12.         printf( "Sum is %d\n", sum );
13.
14.         return 0;
15.     }
```

ΕΞΟΔΟΣ:

```
Sum is 2550
```

Σημειώστε ότι το σώμα της δομής **for** θα μπορούσε πραγματικά να συγχωνευθεί στο ακριανό δεξί τμήμα της κεφαλίδας του **for** χρησιμοποιώντας το κόμμα, όπως:

```
for ( number = 2; number <= 100;sum += number,  number += 2 )
    ;    /* empty body */
```

Η εντολή **sum = 0** θα μπορούσε επίσης να συγχωνευθεί στο πρώτο τμήμα του **for**.

Παράδειγμα 22

Το ακόλουθο παράδειγμα υπολογίζει τον ανατοκισμό (compound interest).

Κάποιος επενδύει $1000,00 σε ένα λογαριασμό ταμιευτηρίου που αποδίδει 5 τοις εκατό τόκο. Υποθέτοντας ότι όλος ο τόκος απομένει στην κατάθεση του λογαριασμού υπολογίστε και εκτυπώστε το ποσό των χρημάτων που θα υπάρχουν στο λογαριασμό στο τέλος κάθε χρόνου για 10 χρόνια. Χρησιμοποιήστε τον ακόλουθο τύπο για τον καθορισμό αυτών των ποσών:

$$a = p(1 + r)^n$$

όπου

p	is the original amount invested (i.e. the principal)
r	is the annual interest rate
n	is the number of years
a	is the amount on deposit at the end of the nth year.

```
1.  /* Calculating compound interest */
2.
3.  #include <stdio.h>
4.  #include <math.h>
5.
6.  int main()
7.  {
8.      int year;
9.      double amount, principal = 1000.0, rate = .05;
10.
11.     printf( "%4s%21s\n", "Year", "Amount of deposit " );
12.
13.     for ( year = 1; year <= 10; year++ )  {
14.         amount = principal * pow( 1.0 + rate, year );
15.         printf("%4d%21.2f\n", year, amount );
16.      }
17.
18. return 0;
19. }
```

ΕΞΟΔΟΣ:

Year	Amount on deposit
1	1050.00
2	1102.50
3	1157.62
4	1215.51
5	1276.28
6	1340.10
7	1407.10
8	1477.46
9	1551.33
10	1628.89

Η συνάρτηση **pow(x, y)** υπολογίζει τη τιμή του **x** στη δύναμη του **y**. Παίρνει δύο ορίσματα του **τύπου double** και επιστρέφει μια τιμή **double**. Ο **τύπος double** είναι τύπος υποδιαστολής όπως το **float**, αλλά η μεταβλητή του **τύπου double** μπορεί να αποθηκεύσει μια τιμή πολύ μεγαλύτερης εμβέλειας με μεγαλύτερη ακρίβεια από τη float μεταβλητή.

Σημειώστε ότι το αρχείο κεφαλίδας **math.h** πρέπει να συμπεριληφθεί όποτε μια συνάρτηση μαθηματικών, όπως **pow**, χρησιμοποιείτε. Για κάποιους μεταγλωττιστές αυτό δεν απαιτείται.

Σημειώστε ότι η μεταβλητή **year** είναι τύπου **int**. Η βιβλιοθήκη **math.h** καθοδηγεί τον μεταγλωττιστή να μετατρέψει προσωρινά τη τιμή της year σε τύπο double.

Τιμές και γραμματοσειρές (strings) είναι **right justified**. Για να έχουμε τιμές **left justified** χρησιμοποιούμε το σύμβολο του πλην (-) μεταξύ του % και του πλάτους του πεδίου όπως **%-6d** ή **%-8s**.

Η πολλαπλής-επιλογή δομή switch

Η εντολή **if** είναι **απλής-επιλογής** δομή και η δομή **if/else** είναι **διπλής-επιλογής** δομή. Η **C** μας παρέχει τη δομή **switch** που είναι **πολλαπλής-επιλογής** δομή που χειρίζεται καταστάσεις πολλαπλών επιλογών. Η δομή **switch structure** αποτελείται από μια σειρά **case labels**, και ένα προαιρετικό **default case**.

Παράδειγμα 23

```
1.  /* Counting letter grades */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.  int grade;
8.  int aCount = 0, bCount = 0, cCount = 0,
9.      dCount = 0, fCount = 0;
10.
11. printf( "Enter the letter grades.\n" );
12. printf( " Enter the EOF character to end input.\n" );
13.
14. while ( ( grade = getchar () ) != EOF )  {
15.
16. switch ( grade )   {            /* switch nested in while */
17.
18.         case 'A': case 'a':     /* grade was uppercase A*/
19.         ++aCount;               /* or lower case a */
20.         break;
21.
22.         case 'B': case 'b':     /* grade was uppercase B */
23.         ++bCount;               /* or lower case b */
24.          break;
25.
26.          case 'C': case 'c':    /* grade was uppercase C*/
27.          ++cCount;              /* or lower case c */
28.           break;
29.
30.           case 'D': case 'd ':  /* grade was uppercase D*/
31.           ++dCount;             /* or lower case d */
32.            break;
33.
```

```
34.                 case 'F': case 'f':     /* grade was uppercase F*/
35.                ++fCount;                /* or lower case f */
36.                 break;
37.
38.                 case '\n': case ' ':     /* ignore this in input */
39.                 break;
40.
41.                 default:                 /* catch all other characters */
42.                         printf( "Incorrect letter grade entered." );
43.                         printf( "Enter a new grade.\n" );
44.                         break;
45.             }
46.         }
47.         printf( "\nTotals for each letter grade are:\n" );
48.         printf( "A : %d\n", aCount );
49.         printf( "B : %d\n", bCount );
50.         printf( "C : %d\n", cCount );
51.         printf( "D : %d\n", dCount );
52.         printf( "F : %d\n", fCount );
53.         return 0;
54.         }
```

ΕΞΟΔΟΣ:

```
Enter the letter grades.
Enter the EOF character to end input.
A
B
C
C
A
D
F
C
E
Incorrect letter grade entered. Enter a new grade.
D
A
B

Totals for each letter grade are:
A: 3
B: 2
C: 3
D: 2
F: 1
```

ΓΡΑΜΜΗ 14: Η συνάρτηση **getchar** από τη πρότυπη βιβλιοθήκη εισόδου/εξόδου διαβάζει ένα χαρακτήρα από το πληκτρολόγιο και τον αποθηκεύει στην ακέραια (integer**)** μεταβλητή **grade**. Οι χαρακτήρες συνήθως αποθηκεύονται σε μεταβλητές του τύπου **char**. Εντούτοις, ένα σημαντικό χαρακτηριστικό της **C** είναι ότι οι χαρακτήρες μπορούν να αποθηκευτούν σε οποιονδήποτε τύπο **integer** για το λόγο ότι συνήθως αντιπροσωπεύονται σαν **1- byte integers** στον υπολογιστή. Ως εκτούτου, μπορούμε να μεταχειριζόμαστε ένα χαρακτήρα σαν είτε **integer** ή **character** ανάλογα της χρήσης του. Για παράδειγμα:

printf("The character (%c) has the value %d.\n", 'a', 'a');

Το αποτέλεσμα είναι

The character (a) has the value 97.

Το 97 είναι ο ASCII (**A**merican **S**tandard **C**ode for **I**nformation **I**nterchange) αριθμός του χαρακτήρα 'a'.

Το γεγονός ότι η εντολές ανάθεσης δίνουν τιμές σε μεταβλητές θα ήταν πολύ χρήσιμο να ξέρετε ότι με τη **C** μπορούμε να δώσουμε μια αρχική τιμή σε διαφορετικές μεταβλητές ταυτόχρονα, όπως:

a = b = c = 0;

EOF (End Of File)

Εξαρτάται από το σύστημα, μπορεί να είναι **-1,** ή **a special negative value,** ή **<return> <ctrl d>**, ή **<return> <ctrl z>**.

Break

Η εντολή **break** είναι η αιτία έτσι ώστε η ροή του προγράμματος να συνεχίσει με τη πρώτη εντολή μετά τη δομή **switch**.

Η εντολή break στο **default** είναι προαιρετική.

Στην εντολή **switch** τα άγκιστρα (**{}**) είναι προαιρετικά για σύνθετες εντολές (compound). Για παράδειγμα:

```
case 'A': case 'a ':        /* grade was uppercase A*/
   ++aCount;                /* or lower case a */
   break;
```

Η επαναληπτική εντολή do/while

Μορφή:

```
do {
        Statement(s)
} while ( condition ) ;
```

Τα άγκιστρα είναι προαιρετικά. Για καλύτερη πρακτική προγραμματισμού ως συνήθως τα χρησιμοποιούμε.

Παράδειγμα 24

```
1.  /* Using the do/while repetition structure */
2.  #include <stdio.h>
3.
4.  int main;
5.  {
6.      int counter = 1;
7.
8.      do   {
9.              printf( "%d   ", counter );
10.     }  while ( ++counter <= 10 );
11.
12.     return 0;
13. }
```

ΕΞΟΔΟΣ:

```
1  2  3  4  5  6  7  8  9  10
```

Η εντολή break

Η εντολή **break** χρησιμοποιείται μαζί με τις εντολές **while**, **for**, **do/while**, και **switch**. Προκαλεί αμέσως την έξοδο από τη δομή και η εκτέλεση του προγράμματος συνεχίζεται με την πρώτη εντολή μετά τη δομή.

Παράδειγμα 25

```
1.  /* Using the break statement in a for structure */
2.  #include <stdio.h>
3.
4.  int main()
5.  {
6.      int x;
7.
8.      for ( x = 1; x <= 10; x++ )   {
9.
10.         if ( x == 5 )
11.             break;            /* break loop only if x == 5 */
12.
13.         printf( "%d   ", x );
14.      }
15.
16.      printf( "\nBroke out of loop at x == %d\n", x );
17.      return 0;
18. }
```

ΕΞΟΔΟΣ:

```
1   2   3   4
Broke out of loop at x == 5
```

Η εντολή continue

Η εντολή **continue** χρησιμοποιείται μαζί με τις εντολές **while**, **for**, **do/while**, και **switch**. Παραλείπει τις υπόλοιπες εντολές του σώματος αυτής της δομής και εκτελεί την επόμενη επανάληψη του βρόχου. Στις δομές **while** και **do/while**, η συνέχιση του βρόχου αξιολογείται αμέσως μετά την εκτέλεση της εντολής continue. Στη δομή **for**, εκτελείται η παράσταση της προσαύξησης και μετά αξιολογείται η συνέχιση του βρόχου.

Παράδειγμα 26

```
1.  /* Using the break statement in a for structure */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int x;
8.
9.      for ( x = 1; x <= 10; x++ )   {
10.
11.         If ( x == 5 )
12.             continue;        /* skip remaining code in loop only
13.                                 if x == 5 */
14.
15.         printf( "%d   ", x );
16.      }
17.
18.     printf( "\nUsed continue to skip printing the value 5\n" );
19.     return 0;
20. }
```

ΕΞΟΔΟΣ:

```
1   2   3   4   6   7   8   9   10
Used continue to skip printing the value 5
```

ΕΝΟΤΗΤΕΣ ΠΡΟΓΡΑΜΜΑΤΟΣ ΜΕ C

Εμπειρία έχει δείξει ότι ο καλύτερος τρόπος για να αναπτύξουμε και να διατηρήσουμε ένα μεγάλο πρόγραμμα είναι να το κατασκευάσουμε από μικρότερα κομμάτια ή ενότητες, καθεμία από τις οποίες είναι πιο εύχρηστη από το αρχικό πρόγραμμα. Αυτή η τεχνική ονομάζεται **διαίρει και βασίλευε**.

Οι ενότητες στη **C** ονομάζονται **functions** (**συναρτήσεις**). Η **C** μας παρέχει μια **πρότυπη βιβλιοθήκη** με μια πλούσια συλλογή από συναρτήσεις για την εκτέλεση κοινών μαθηματικών υπολογισμών, χειρισμό γραμματοσειράς, χειρισμό χαρακτήρα, χειρισμό εισόδου/εξόδου, και για πολλές άλλες χρήσιμες λειτουργίες.

Παραδείγματα από έτοιμες συναρτήσεις που χρησιμοποιήσαμε μέχρι τώρα είναι **printf**, **scanf**, και **pow**.

Οι προγραμματιστές μπορούν να γράψουν συναρτήσεις για τον καθορισμό συγκεκριμένων εργασιών που μπορούν να χρησιμοποιηθούν σε πολλά σημεία μέσα στο πρόγραμμα. Αυτές, μερικές φορές ονομάζονται **programmer-defined functions**.

Οι συναρτήσεις επικαλούνται από μια **κλήση συνάρτησης**. Η κλήση συνάρτησης καθορίζει το **όνομα της συνάρτησης** και παρέχει πληροφορίες **σαν ορίσματα** (**arguments**) που η συνάρτηση που καλείται χρειάζεται για να εκτελέσει το καθορισμένο έργο της.

ΒΙΒΛΙΟΘΗΚΗ ΜΑΘΗΜΑΤΙΚΩΝ ΣΥΝΑΡΤΗΣΕΩΝ

```
printf( "%.2f", sqrt ( 900.0 ) );
```

Σε αυτό το παράδειγμα η συνάρτηση της βιβλιοθήκης μαθηματικών **sqrt** καλείται μαζί περνώντας το όρισμα **900.0**. αυτή η εντολή θα τυπώσει το **30.00**. Η συνάρτηση **sqrt** παίρνει αυτό το όρισμα που είναι τύπου **double** και επιστρέφει ένα **double** αποτέλεσμα.

Όλες οι συναρτήσεις της βιβλιοθήκης μαθηματικών επιστρέφουν δεδομένα τύπου **double**.

Τα ορίσματα μιας συνάρτησης μπορούν να είναι σταθερά (constant), μεταβλητές (variables), ή παραστάσεις (expressions). Εάν **c1 = 13.0, d = 3.0** και **f = 4.0**, τότε η εντολή

```
printf( "%.2f", sqrt ( c1 + d * f ) );
```

υπολογίζει και εκτυπώνει τη τετραγωνική ρίζα του **13.0 + 3.0 * 4.0 = 25.0** που είναι **5.00**.

Μερικές συναρτήσεις της βιβλιοθήκης μαθηματικών της C είναι:

Συνάρτηση	Περιγραφή	Παράδειγμα
sqrt(x)	τετραγωνική ρίζα του **x**	sqrt(900.0) είναι 30.0 sqrt(9.0) είναι 3.0
exp(x)	εκθετική συνάρτηση $\mathbf{e^x}$	exp(1.0) είναι 2.718282 exp(2.0) είναι 7.389056
log(x)	φυσικός λογάριθμος του **x (βάση e)**	log(2.718282) είναι 1.0 log(7.389056) είναι 2.0
log10(x)	λογάριθμος του **x (βάση 10)**	log10(1.0) είναι 0.0 log10(10.0) είναι 1.0 log10(100.0) είναι 2.0
fabs(x)	απόλυτη τιμή του **x**	fabs(5.0) είναι 5.0 fabs(0.0) είναι 0.0 fabs(-5.0) είναι 5.0
ceil(x)	στρογγυλοποιεί το **x** στο μικρότερο ακέραιο όχι μικρότερο του **x**	ceil(9.2) είναι 10.0 ceil(-9.8) είναι -9.0
floor(x)	στρογγυλοποιεί το **x** στο μικρότερο ακέραιο	floor(9.2) είναι 9.0 floor(-9.8) είναι -10.0
pow(x, y)	**x** υψωμένο σε δύναμη **y** ($\mathbf{x^y}$)	pow(2, 7) είναι 128.0 pow(9, .5) είναι 3.0

fmod(x, y)	υπόλοιπο του **x/y** σαν ένα δεκαδικό αριθμό (floating-point number)	fmod(13.657, 2.333) είναι 1.992
sin(x)	τριγωνομετρικό ημίτονο του **x** **(x σε ακτίνια)**	sin(0.0) είναι 0.0
cos(x)	τριγωνομετρικό συνημίτονο του **x** **(x σε ακτίνια)**	cos(0.0) είναι 1.0
tan(x)	τριγωνομετρικό εφαπτομένη του **x** **(x σε ακτίνια)**	tan(0.0) είναι 0.0

Σημείωση: Οι μεταβλητές **x** και **y** είναι τύπου **double**.

Οι **συναρτήσεις** επιτρέπουν στο προγραμματιστή να οργανώσει καλύτερα το πρόγραμμα του.

Όλες οι μεταβλητές που δηλώνονται μέσα στη συνάρτηση είναι **τοπικές-μεταβλητές** (**local-variables**) – αναγνωρίζονται μόνο μέσα στη συνάρτηση που ορίζονται.

Οι περισσότερες συναρτήσεις έχουν μια **λίστα** παραμέτρων **(list of parameters**). Οι παράμετροι παρέχουν τα μέσα επικοινωνίας και πληροφόρησης μεταξύ συναρτήσεων. **Οι παράμετροι μιας συνάρτησης θεωρούνται επίσης σαν τοπικές μεταβλητές**.

ΓΙΑΤΙ ΧΡΗΣΙΜΟΠΟΙΟΥΜΕ ΣΥΝΑΡΤΗΣΕΙΣ;

1. Το διαίρει και βασίλευε κάνει την ανάπτυξη ενός προγράμματος πιο εύχρηστη.
2. Δυνατότητα επαναχρησιμοποίησης λογισμικού (ένας σημαντικός παράγοντας στον προγραμματισμό).
3. Αποφεύγεται η επανάληψη κώδικα σε ένα πρόγραμμα.

Ορισμοί συναρτήσεων

```
return-value-type function-name ( parameter list )
{
        declarations

        statements
}
```

Η επιστρεφόμενη τιμή **void** δεικνύει ότι η συνάρτηση δεν επιστρέφει κάποια τιμή.
Μια απροσδιόριστη επιστρεφόμενη τιμή return-value-type πάντα θεωρείτε από το μεταγλωττιστή σαν **int**.
Η λίστα παραμέτρων είναι μια οριοθετημένη λίστα, οι παράμετροι χωρίζονται με κόμμα, που περιέχει τους ορισμούς των παραμέτρων που παίρνει η συνάρτηση από μια κλήση.

Εάν η συνάρτηση δεν παίρνει οποιανδήποτε τιμή η λίστα παραμέτρων είναι **void**. Το **void** σε αυτή την περίπτωση είναι προαιρετικό. Ο τύπος για κάθε παράμετρο πρέπει να δίνεται ρητά εκτός αν η παράμετρος είναι τύπου **int**. Εάν ο τύπος δεν δίνεται, ο τύπος **int** δίνεται από το μεταγλωττιστή.

Υπάρχουν τρεις τρόποι για να επαναφέρετε τον έλεγχο στο σημείο από όπου έγινε η επίκληση μιας συνάρτησης.

1. Εάν η συνάρτηση δεν επιστρέφει κάποιο αποτέλεσμα, ο έλεγχος απλά επιστρέφετε όταν φτάσουμε στο δεξί άγκιστρο της συνάρτησης.

2. Όταν εκτελείται η εντολή

   return;

3. Όταν εκτελείται η εντολή

   return παράσταση;

Ένα πρωτότυπο συνάρτησης λέει στο μεταγλωττιστή τον τύπο των δεδομένων που θα επιστραφούν από τη συνάρτηση, τον αριθμό των παραμέτρων που περιμένει να λάβει η συνάρτηση, ο τύπος των παραμέτρων και η σειρά με την οποία αυτές οι παράμετροι αναμένονται να ληφθούν.

Παράδειγμα 27

```
1.  /* A programmer defined square function */
2.
3.  #include<stdio.h>
4.
5.  int square ( int );   /* function prototype */
6.
7.  int main()
8.  {
9.      int x;
10.
11.     for ( x = 1; x <= 10; x++ )
12.         printf(  "%d  ", square( x ) );
13.
14.     printf( "\n" );
15.
16.     return 0;
17. }
18.
19. /* Function definition */
20. int square( int y )
21. {
22.     return y * y;
23. }
```

ΕΞΟΔΟΣ:

```
1 4  9  16  25  36  49  64  81  100
```

Παράδειγμα 28

```
1.  /* Finding the maximum of three integers */
2.
3.  #include<stdio.h>
4.
5.  int maximum( int, int, int );        /* function prototype */
6.
7.  int main()
8.  {
9.      int a, b, c;
10.
11.     printf( "Enter three integers: " );
12.     scanf( "%d%d%d", &a, &b, &c );
13.     printf( "Maximum is: %d\n", maximum( a, b, c ) );
14.
15.     return 0;
16. }
17.
18. /* Function maximum definition */
19. int maximum( int x, int y, int z )
20. {
21.     int max = x;
22.
23.    if ( y > max )
24.       max = y;
25.
26.   if ( z > max )
27.      max = z;
28.
29.  return max;
30. }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter three integers: 22 85 17
Maximum is: 85
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter three integers: 85 22 17
Maximum is: 85
```

3ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter three integers: 22 17 85
Maximum is: 85
```

Ιεραρχία προώθησης για τύπους δεδομένων

Οι τιμές των ορισμάτων που δεν αντιστοιχούν ακριβώς με τους τύπους των παραμέτρων της συνάρτησης στο πρωτότυπο μετατρέπονται στο κατάλληλο τύπο, πριν η συνάρτηση καλεστεί. Αυτές οι μετατροπές μπορούν να οδηγήσουν σε λανθασμένα αποτελέσματα εάν οι κανόνες προώθησης της C δεν ακολουθηθούν.

Ο βασικός κανόνας είναι να μετατρέπετε από τη χαμηλότερη ιεραρχία στη ψηλότερη. Αποφεύγεται το αντίθετο.

Τύποι δεδομένων	printf προδιαγραφές μετατροπής	scanf προδιαγραφές μετατροπής
long double	%Lf	%Lf
double	%f	%lf
float	%f	%f
unsigned long int	%lu	%lu
long int	%ld	%ld
unsigned int	%u	%u
int	%d	%d
short	%hd	%hd
char	%c	%c

Τύποι δεδομένων	Μέγεθος σε bytes	Όρια
long double	10	$3.4 * 10^{-4932}$ εώς $1.1 * 10^{+4932}$ (19 δεκαδικά)
double	8	$2.3 * 10^{-308}$ έως $1.7 * 10^{+308}$ (15 δεκαδικά)
float	4	$1.2 * 10^{-38}$ έως $3.4 * 10^{+38}$ (6 δεκαδικά)
unsigned long int	4	0 έως 4,294,967,295
long int	4	-2,147,483,648 έως 2,147,483,647
unsigned int	2 ή 4	0 έως 65,535 ή 0 έως 4,294,967,265
int	2 ή 4	-32,768 έως 32767 ή -2,147,483,648 έως 2,147,483,647
short	2	-32767 έως 32,767
unsigned short	2	0 έως 65,535
char	1	-128 έως 127 ή 0 255
unsigned char	1	0 έως 255
signed char	1	-128 έως 127

ΑΡΧΕΙΑ ΚΕΦΑΛΙΔΑΣ

Κάθε πρότυπη βιβλιοθήκη έχει ένα αντίστοιχο αρχείο κεφαλίδας που περιέχει τα πρωτότυπα για όλες τις συναρτήσεις αυτής της βιβλιοθήκης και τους ορισμούς διαφόρων τύπων δεδομένων και σταθερών που είναι χρήσιμες για αυτές τις συναρτήσεις.

Αρχείο κεφαλίδας πρότυπου βιβλιοθήκης	Εξήγηση
<assert.h>	Περιέχει μακρό-εντολές και πληροφορίες για την προσθήκη διαγνωστικών που βοηθούν στον εντοπισμό λαθών στο πρόγραμμα (debugging).
<ctype.h>	Περιέχει πρωτότυπα συναρτήσεων για συναρτήσεις που ελέγχου χαρακτήρες για ορισμένες ιδιότητες, και πρωτότυπα συναρτήσεων για συναρτήσεις που μπορούν να χρησιμοποιηθούν για μετατροπή μικρών χαρακτήρων σε κεφαλαία και το αντίστροφο.
<errno.h>	Ορίζει μακρό-εντολές που είναι χρήσιμες συνθήκες αναφοράς λάθους.
<float.h>	Περιέχει τα όρια μεγέθους των αριθμών με floating point του συστήματος.
<limits.h>	Περιέχει τα αναπόσπαστα όρια μεγέθους του συστήματος.
<locale.h>	Περιέχει πρωτότυπα συναρτήσεων και άλλες πληροφορίες που επιτρέπει σε ένα πρόγραμμα να τροποποιηθεί για το τρέχων σύνολο τοπικής προσαρμογής (locale) μέσα στο οποίο εκτελείται. Η έννοια του συνόλου τοπικής προσαρμογής επιτρέπει στο σύστημα του υπολογιστή να χειριστεί διάφορες συμβάσεις για την έκφραση δεδομένων, όπως, ημερομηνίες, ώρες, ποσά σε δολάρια και μεγάλους αριθμούς σε όλο το κόσμο.
<math.h>	Περιέχει πρωτότυπα συναρτήσεων για συναρτήσεις της βιβλιοθήκης μαθηματικών.
<setjmp.h>	Περιέχει πρωτότυπα συναρτήσεων για συναρτήσεις που επιτρέπουν παράκαμψη της συνήθης κλήσης μιας συνάρτησης και της ακολουθίας επιστροφής.
<signal.h>	Περιέχει πρωτότυπα συναρτήσεων και μακρό-εντολές για χειρισμό διαφόρων προβλημάτων που μπορούν να προκύψουν κατά τη διάρκεια της εκτέλεσης ενός προγράμματος.
<stdarg.h>	Ορίζει μακρό-εντολές για την αντιμετώπιση μιας λίστας ορισμάτων που ο αριθμός τους και ο τύπος τους είναι άγνωστος.
<stddef.h>	Περιέχει κοινούς ορισμούς τύπων που χρησιμοποιούνται από τη C για την εκτέλεση ορισμένων υπολογισμών.
<stdio.h>	Περιέχει πρωτότυπα συναρτήσεων για συναρτήσεις της πρότυπης εισόδου/εξόδου (input/output) βιβλιοθήκης, και πληροφορίες που χρησιμοποιούνται από αυτές.

Αρχείο κεφαλίδας πρότυπου βιβλιοθήκης	Εξήγηση
<stdlib.h>	Περιέχει πρωτότυπα συναρτήσεων για μετατροπές αριθμών σε κείμενο και κείμενο σε αριθμούς, εκχώρηση μνήμης, τυχαίων αριθμών και άλλες βοηθητικές συναρτήσεις.
<string.h>	Περιέχει πρωτότυπα συναρτήσεων για συναρτήσεις που χειρίζονται γραμματοσειρές (string).
<time.h>	Περιέχει πρωτότυπα συναρτήσεων και τύπους για το χειρισμό της ώρας και της ημερομηνίας.

Ο προγραμματιστής μπορεί να δημιουργήσει προσαρμοσμένο αρχείο κεφαλίδας-πρέπει να τελειώνουν με το περιεκτικό πρωτότυπο συνάρτησης **.h**-μπορούν να συμπεριληφθούν στα προγράμματα με την οδηγία του προ-επεξεργαστή **#include**.

Για παράδειγμα, το αρχείο κεφαλίδας **square.h** μπορεί να συμπεριληφθεί μέσα στο πρόγραμμα μας με το:

```
#include "square.h"
```

Δημιουργία τυχαίων αριθμών (Random Number)

Για να τυχαιοποιήσουμε με τη **C** χρησιμοποιούμε την πρότυπη συνάρτηση **rand()**. Θεωρείστε την ακόλουθη εντολή:

i = rand();

Αυτό θα μας δώσει ένα τυχαίο αριθμό μεταξύ **0..RAND_MAX** (συμβολική σταθερά που δηλώνεται στο αρχείο κεφαλίδας **<stdlib.h>**). Το πρότυπο του ANSI (American National Standards Institute) μας δηλώνει ότι η σταθερά RAND_MAX πρέπει να είναι τουλάχιστο 32767 που είναι η μεγαλύτερη τιμή για ένα 2-byte ακέραιο αριθμό (2^{16}-1).

Παράδειγμα 29

Αυτό το πρόγραμμα δημιουργεί τυχαίους αριθμούς για το ρίξιμο ζαριάς (1..6).

```
1.  /* Shifted, scaled integers produced by 1 + rand() % 6 */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.      int i;
9.
10.     for ( i = 1; i <= 20; i++ ) {
11.         printf( "%10d", (1 + rand() % 6 ) );
12.
13.         if ( i % 5 == 0 )
14.             printf( "\n" );
15.     }
16.
17.     return 0;
18. }
```

1ο Τρέξιμο

ΕΞΟΔΟΣ:

```
         6     6     5     5     6
         5     1     1     5     3
         6     6     2     4     2
         6     2     3     4     1
```

Εάν τρέξουμε το πρόγραμμα ξανά, ο υπολογιστής θα μας δώσει την ίδια σειρά αριθμών.

2ο Τρέξιμο

ΕΞΟΔΟΣ:

```
         6     6     5     5     6
         5     1     1     5     3
         6     6     2     4     2
         6     2     3     4     1
```

Παράδειγμα 30

Εάν θέλουμε διαφορετικούς τυχαίους αριθμούς κάθε φορά που τρέχουμε το πρόγραμμα μας χρησιμοποιούμε τη συνάρτηση **srand()**. Η συνάρτηση srand() παίρνει σαν όρισμα ένα **unsigned integer** και σπέρνει (seeds) τη συνάρτηση **srand()** για να παράγει διαφορετική ακολουθία τυχαίων αριθμών για κάθε εκτέλεση του προγράμματος.

```
1.  /* Randomizing die-rolling program */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.        int i;
9.        unsigned seed;
10.
11.       printf( "Enter seed: " );
12.       scanf( "%u", &seed );
13.       srand( seed );
14.
15.       for ( i = 1; i <= 10; i++ ) {
16.             printf( "%10d", 1 + (rand() % 6 ) );
17.
18.             if ( i % 5 == 0 )
19.                 printf( "\n" );
20.       }
21.
22.       return 0;
23. }
```

1ο Τρέξιμο

ΕΞΟΔΟΣ:

```
Enter seed: 67
         6         1         4         6         2
         1         6         1         6         4
```

2ο Τρέξιμο

ΕΞΟΔΟΣ:

```
Enter seed: 867
         2         4         6         1         6
         1         1         3         6         2
```

3ο Τρέξιμο

ΕΞΟΔΟΣ:

```
Enter seed: 67
         6         1         4         6         2
         1         6         1         6         4
```

Ο τύπος **unsigned** ο οποίος είναι short (**unsigned int**) αποθηκεύεται σε τουλάχιστον 2 bytes της μνήμης και έχει μόνο θετικές τιμές μεταξύ 0..65535. Ένα 4-byte unsigned μπορεί να έχει μόνο θετικές τιμές μεταξύ 0..4294967295.

***** ΣΗΜΕΙΩΣΗ *****
Εάν θέλουμε να δημιουργήσουμε τυχαίους αριθμούς χωρίς τη σπορά, μπορούμε να χρησιμοποιήσουμε μια εντολή όπως

srand(time (NULL));

Αυτό αναγκάζει τον υπολογιστή να διαβάσει το ρολόι για να λάβει τη τιμή της σποράς αυτόματα. Η συνάρτηση **time** επιστρέφει την τρέχουσα ώρα της ημέρας σε δευτερόλεπτα. Αυτή η τιμή μετατρέπεται σε unsigned integer και χρησιμοποιείται σαν σπορά για τον δημιουργό των τυχαίων αριθμό. Η συνάρτηση **time** παίρνει το **NULL** σαν όρισμα, σε αυτό το παράδειγμα, επιτρέποντας αυτής της συνάρτησης να μας επιστρέψει μια γραμματοσειρά που αντιπροσωπεύει την ώρα της ημέρας; Το πρωτότυπο της συνάρτησης **time** βρίσκεται στο αρχείο κεφαλίδας **<time.h>**.

Παράδειγμα 31

Για να δείξουμε ότι αυτοί οι αριθμοί εμφανίζονται περίπου με ίση πιθανότητα, ας προσομοιώσουμε 6000 ζαριές με το ακόλουθο παράδειγμα.

```
1.  /* Roll a six-sided die 6000 times */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.        int face, roll, frequency1 = 0, frequency2 = 0,
9.            frequency3 = 0, frequency4 = 0,
10.           frequency5 = 0, frequency6 = 0;
11.
12.       for ( roll = 1; roll <= 6000; roll++ ) {
13.             face = 1 + rand() % 6;
14.
15.             switch ( face )  {
16.                 case 1:
17.                       ++frequency1;
18.                       break;
19.                 case 2:
20.                       ++frequency2;
21.                       break;
22.                 case 3:
23.                       ++frequency3;
```

```
                    break;
                case 4:
                    ++frequency4;
                    break;
                case 5:
                    ++frequency5;
                    break;
                case 6:
                    ++frequency6;
                    break;
            }
        }

    printf( "%s%13s\n". "Face", "Frequency" );
    printf( "    1%13d\n", frequency1 );
    printf( "    2%13d\n", frequency2 );
    printf( "    3%13d\n", frequency3 );
    printf( "    4%13d\n", frequency4 );
    printf( "    5%13d\n", frequency5 );
    printf( "    6%13d\n", frequency6 );
    return 0;
}
```

ΕΞΟΔΟΣ:

Face	Frequency
1	1003
2	1017
3	983
4	994
5	1004
6	999

***** ΣΗΜΕΙΩΣΗ *****

Ο τύπος που χρησιμοποιούμε για οποιοδήποτε φάσμα αριθμών είναι:

ΠΡΩΤΗ ΤΙΜΗ + rand( ) % (ΔΙΑΦΟΡΑ ΤΩΝ ΔΥΟ ΤΙΜΩΝ + 1)

Παράδειγμα 32

```
1.  /* Craps-Game */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.  #include <time.h>
6.
7.  int rollDice ( void );
8.
9.  int main()
10. {
11.     int gameStatus, sum, myPoint;
12.
13.     srand( time( NULL ) );
14.     sum = rollDice();                    /* first roll of the dice */
15.
16.     switch ( sum )  {
17.         case 7:    case 11:          /* win on first roll */
18.             gameStatus = 1;
19.             break;
20.         case 2: case 3: case 12:      /* lose on first roll */
21.             gameStatus = 2;
22.             break;
23.         default:
24.             gameStatus = 0;
25.             myPoint = sum;
26.             printf( "Point is %d\n", myPoint );
27.             break;
28.     }
29.
30.     while ( gameStatus == 0 )  {   /* keep rolling */
31.             sum = rollDice();
32.
33.     if ( sum == myPoint )          /* win by making point */
34.             gameStatus = 1;
35.     else
36.             if ( sum == 7 )        /* lose by rolling 7 */
37.                 gameStatus = 2;
38.     }
39.
40.     if ( gameStatus == 1 )
41.             printf(  "Player wins\n" );
42.     else
43.             printf( "Player loses\n" );
44.
```

```
45.         return 0;
46.  }
47.
48.  int rollDice ( void )
49.  {
50.         int die1, die2, workSum;
51.
52.         die1 = 1 + ( rand() % 6 );
53.         die2 = 1 + ( rand() % 6 );
54.         workSum = die1 + die2;
55.         printf( "Player rolled %d + %d = %d\n", die1, die2, workSum );
56.         return workSum;
57.  }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Player rolled 6 + 5 = 11
Player wins
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Player rolled 6 + 6 = 12
Player wins
```

3ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Player rolled 4 + 6 = 10
Point is 10
Player rolled 2 + 4 = 6
Player rolled 6 + 5 = 11
Player rolled 3 + 3 = 6
Player rolled 6 + 4 = 10
Player wins
```

4ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Player rolled 1 + 3 = 4
Point is 4
Player rolled 1 + 4 = 5
Player rolled 5 + 4 = 9
Player rolled 4 + 6 = 10
Player rolled 6 + 3 = 9
Player rolled 1 + 2 = 3
Player rolled 5 + 2 = 7
Player loses
```

ΓΡΑΜΜΗ 48:
Η συνάρτηση **rollDice** δεν παίρνει οποιονδήποτε όρισμα για αυτό δεικνύουμε **void** στη λίστα παραμέτρων.

Κλάσεις αποθήκευσης

Τα χαρακτηριστικά των μεταβλητών περιλαμβάνουν όνομα, τύπο και τιμή. Σε αυτό το κεφάλαιο επίσης χρησιμοποιούμε αναγνωριστικά ως ονόματα για συναρτήσεις που ορίζονται από το χρήστη (user-defined functions). Στην πραγματικότητα, κάθε αναγνωριστικό έχει και άλλες ιδιότητες συμπεριλαμβανομένων τάξη αποθήκευσης, διάρκεια της αποθήκευσης, πεδίο εφαρμογής και σύνδεση.

Μια κλάση αποθήκευσης ενός αναγνωριστικού μας βοηθάει να καθορίσουμε τη διάρκεια της αποθήκευσης (storage duration), την εμβέλεια (scope) και τη σύνδεση (linkage) του.

- **Storage duration** είναι η περίοδος κατά την οποία το αναγνωριστικό υπάρχει στη μνήμη. Ορισμένα αναγνωριστικά υπάρχουν για σύντομη χρονική διάρκεια, μερικά δημιουργούνται και καταστρέφονται επανειλημμένα και άλλα υπάρχουν καθ' όλη τη διάρκεια της εκτέλεσης ενός προγράμματος.

- **An identifier's scope** είναι όπου το αναγνωριστικό μπορεί να αναγνωριστεί μέσα σε ένα πρόγραμμα. Ορισμένα αναγνωριστικά μπορούν να αναγνωριστούν σε ολόκληρο το πρόγραμμα και άλλα μόνο σε ορισμένα μέρη του προγράμματος.

- **An identifier's linkage** (κάποιο αρχείο με πηγαίο κώδικα για πολλαπλές χρήσεις (ένα θέμα που θα συζητηθεί σε μεταγενέστερο στάδιο)) καθορίζει εάν ένα αναγνωριστικό μπορεί να αναγνωριστεί, με ορθές δηλώσεις, σε διάφορα αρχεία με πηγαίο κώδικα ή μόνο στο τρέχον πρόγραμμα.

Η C μας παρέχει τέσσερις κλάσεις αποθήκευσης που δεικνύονται από τα προσδιοριστικά (specifiers) της αποθήκευσης: **auto**, **register**, **extern** και **static** που μπορούν να χωριστούν σε δύο διάρκειες αποθήκευσης: Αυτόματη (automatic) και στατική (static) διάρκεια αποθήκευσης.

1. **auto** και **register**

   auto και **register** χρησιμοποιούνται για να δηλώνουμε μεταβλητές με αποθήκευση αυτόματης διάρκειας που δημιουργούνται όταν μπαίνουμε στο μπλοκ μέσα στο οποίο αυτές οι μεταβλητές δηλώθηκαν. Υπάρχουν καθ' όλη τη διάρκεια που το μπλοκ είναι ενεργό και καταστρέφονται όταν βγαίνουμε από το μπλοκ. Μόνο μεταβλητές μπορούν να έχουν αποθήκευση αυτόματης διάρκειας. Οι τοπικές μεταβλητές μιας συνάρτησης (αυτές που δηλώνονται μέσα στη λίστα παραμέτρων ή μέσα στο σώμα της συνάρτησης) συνήθως έχουν αυτόματη διάρκεια αποθήκευσης.

   - Η λέξη-κλειδί **auto** ρητά δηλώνει μεταβλητές αυτόματης διάρκειας αποθήκευσης. Για τοπικές μεταβλητές η λέξη-κλειδί είναι προαιρετική για το λόγο ότι αυτές οι μεταβλητές από προεπιλογή έχουν αυτό το είδος διάρκειας.
     Παράδειγμα: **auto double x, y;**
   - Το προσδιοριστικό της κλάσης αποθήκευσης **register** χρησιμοποιήται για να δηλώσουμε αυτόματες μεταβλητές και ταυτόχρονα καθοδηγούμε το μεταγλωττιστή να αποθηκεύσει αυτές τις μεταβλητές μέσα σε ένα υψηλής ταχύτητας **καταχωριτή (register)** του Η/Υ. Ο μεταγλωττιστής κάποιες φορές μπορεί να αγνοήσει το register σε περιπτώσεις που δεν υπάρχουν αρκετά registers για το μεταγλωττιστή. Σήμερα μπορεί να μην είναι ανάγκη να δηλώνουμε μεταβλητές register για τον απλό λόγο της βελτίωσης που έχουμε στους μεταγλωττιστές.
     Παράδειγμα: **register int counter = 1;**

2. **extern** και **static** χρησιμοποιούνται για να δηλώσουμε αναγνωριστικά για μεταβλητές και συναρτήσεις με **στατική** (**static**) διάρκεια αποθήκευσης. Υφίστανται από το σημείο που ξεκινά η εκτέλεση του προγράμματος μέχρι το τέλος της εκτέλεσης. Υπάρχουν δύο είδη αναγνωριστικών με στατική διάρκεια αποθήκευσης:

   - Εξωτερικά αναγνωριστικά (όπως καθολικές (global) μεταβλητές και συναρτήσεις) που δηλώνονται με τη λέξη **extern** μπροστά. *Για το λόγο ότι από προεπιλογή είναι εξωτερικά δεν χρειάζεται να χρησιμοποιούμε το **extern** μπροστά και διατηρούν τη τιμή τους καθ' όλη την εκτέλεση του προγράμματος*. Μπορούν να χρησιμοποιηθούν από οποιανδήποτε συνάρτηση που ακολουθεί τον ορισμό τους. Αυτός είναι ένας λόγος για να χρησιμοποιούμε τα πρωτότυπα συναρτήσεων. Όταν για παράδειγμα συμπεριλαμβάνουμε το **stdio.h** που καλεί τη συνάρτηση **printf**, το πρωτότυπο αυτής της συνάρτησης τοποθετείται στην αρχή του προγράμματος μας έτσι ώστε το **printf** να είναι γνωστό σε όλο το πρόγραμμα.
   - Οι τοπικές μεταβλητές που δηλώνονται με τη λέξη **static** μπορούν να χρησιμοποιηθούν μόνο μέσα στη συνάρτηση, αλλά διαφορετικά από τις automatic μεταβλητές, διατηρούν τις τιμές τους μετά την έξοδο από τη συνάρτηση μέσα στην οποία δηλώθηκαν. Την επόμενη φορά που η συνάρτηση καλεστεί η **static** τοπική μεταβλητή έχει την τιμή που είχε πριν.
     Παράδειγμα: **static int count = 1;**

Όλες οι αριθμητικές μεταβλητές με στατική διάρκεια αποθήκευσης παίρνουν σαν αρχική τιμή το 0 από το σύστημα εκτός αν ο προγραμματιστής τους δίνει ρητά άλλη τιμή. Στις μεταβλητές pointer δίνεται η τιμή NULL.

Κανόνες πεδίου εφαρμογής

Παράδειγμα 33

```
1.  /* A scoping example */
2.
3.  #include <stdio.h>
4.
5.  void a ( void );                /* function prototype */
6.  void b ( void );                /* function prototype */
7.  void c ( void );                /* function prototype */
8.
9.  int x = 1;                      /* global variable */
10.
11. int main( )
12. {
13.     int x = 5;                  /* local variable to main */
14.
15.     printf( " local x in outer scope of main is %d\n", x );
16.
17.     {                           /* start new scope */
18.         int x = 7;
19.
20.         printf( " local x in inner scope of main is %d\n", x );
21.     }                           /* end new scope */
22.
23.     printf( " local x in outer scope of main is %d\n", x );
24.
25.     a( );                       /* a has automatic local x */
26.     b( );                       /* b has static local x */
27.     c( );                       /* uses global x */
28.     a( );                       /* a reinitializes automatic local x */
29.     b( );                       /* static local x retains its previous value */
30.     c( );                       /* global x also retains its value */
31.
32.     printf( " local x in main is %d\n", x );
33.     return 0;
34. }
35.
36. void a( void )
37. {
38.     int x = 25;                 /* initialized each time a is called */
39.
```

```
40.         printf( "\n local x in a is %d after entering a\n", x );
41.         ++x;
42.         printf( " local x in a is %d before exiting a\n", x );
43.   }
44.
45.   void b( void )
46.   {
47.         static int x = 50;          /* static initialization only */
48.                                     /* first time b is called */
49.         printf( " \nocal static x is %d on entering b\n", x );
50.         ++x;
51.         printf( " local static x is %d on exiting b\n", x );
52.   }
53.
54.   void c( void )
55.   {
56.         printf( " \nglobal x is %d on entering c\n", x );
57.         x *= 10;
58.         printf( " global x is %d on exiting c\n", x );
59.   }
```

ΕΞΟΔΟΣ:

```
local x in outer scope of main is 5
local x in inner scope of main is 7
local x in outer scope of main is 5

local x in a is 25 after entering a
local x in a is 26 before exiting a

local static x is 50 on entering b
local static x is 51 on exiting b

global x is 1 on entering c
global x is 10 on exiting c

local x in a is 25 after entering a
local x in a is 26 before exiting a

local static x is 51 on entering b
local static x is 52 on exiting b

global x is 10 on entering c
global x is 100 on exiting c
local x in main is 5
```

Αναδρομή (Recursion)

Για ορισμένους τύπους προβλημάτων είναι χρήσιμο οι συναρτήσεις να αυτό-καλούνται.

Μια αναδρομική συνάρτηση είναι μια συνάρτηση που καλεί τον εαυτό της είτε άμεσα είτε έμμεσα μέσω μιας άλλης συνάρτησης.

Αναδρομική κλήση είναι μια κλήση υποπρογράμματος στην οποία το υποπρόγραμμα που καλείται είναι το ίδιο με αυτό που πραγματοποιεί τη κλήση. Η αναδρομή έχει την ιδιότητα της επανάληψης.

Η αναδρομή είναι ένα πολύ ισχυρό χαρακτηριστικό της **C** και χρησιμοποιείται σε πολύ πιο προηγμένες και δύσκολες εφαρμογές.

Το παράδειγμα Παραγοντικό (Factorial)

Το παραγοντικό ενός θετικού ακέραιου (integer) αριθμού **n**, που γράφεται σαν **n!** και προφέρεται **n factorial**, είναι το γινόμενο

n * (n-1) * (n-2)* …… * 1

με το **1!** Να ισούται με **1**, **0!** να ισούται με **1**, και **5! = 5 * 4 * 3 * 2 * 1 = 120**.

Το παραγοντικό ενός ακέραιου αριθμού μεγαλύτερου ή ίσον με το **0** με τη **C** υπολογίζεται ως ακολούθως:

```
factorial = 1;
for ( counter = number;  counter >= 1; counter-- )
        factorial *= counter;
```

Ένας αναδρομικός ορισμός βγαίνει παρατηρώντας την ακόλουθη σχέση:

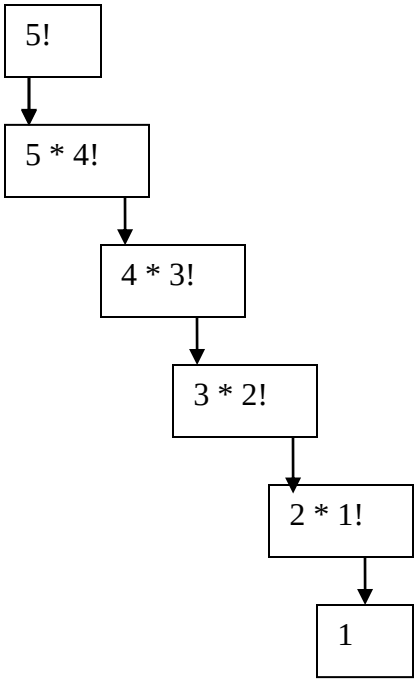
n! = n * (n – 1)!

Για παράδειγμα **5!** Είναι ίσον με

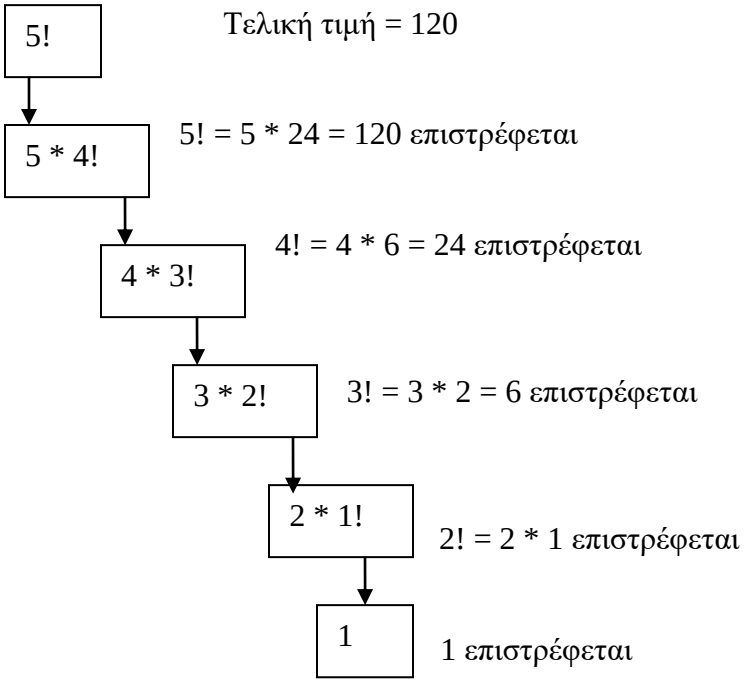
5! = 5 * 4 * 3 * 2 * 1 **ή**
5! = 5 * (4 * 3 * 2 * 1) **ή**
5! = 5 * (4!)

Η παρακάτω εικόνα δείχνει πως η διαδοχή των αναδρομικών κλήσεων προχωρά μέχρι το **1!** που αξιολογείται με το **1**, και η οποία τερματίζει την αναδρομή.

a. Η πομπή των αναδρομικών κλήσεων.



b. οι τιμές που επιστρέφονται από τη κάθε μια αναδρομική κλήση.



Παράδειγμα 34

```
1.  /* Recursive factorial function */
2.
3.  #include <stdio.h>
4.
5.  long factorial ( long );
6.
7.  int main ( )
8.  {
9.        int i;
10.
11.       for ( i = 1; i <= 10; i++ )
12.           printf( "%2d! = %ld\n", i, factorial( i ));
13.       return 0;
14.   }
15.
16.       long factorial( long number )
17.       {
18.             if ( number  <= 1 )
19.                 return 1;
20.             else
21.                 return ( number * factorial( number – 1 ) );
22.        }
```

ΕΞΟΔΟΣ:

```
1!  = 1
2!  = 2
3!  = 6
4!  = 24
5!  = 120
6!  = 720
7!  = 5040
8!  = 40320
9!  = 362880
10! = 3628800
```

Παράδειγμα 35

Η σειρά Fibonacci

0, 1, 1, 2, 3, 5, 8, 13, 21, ……..

Ξεκινά με το 0 και έχει την ιδιότητα ότι κάθε μεταγενέστερος Fibonacci αριθμός είναι το άθροισμα των προηγουμένων δύο Fibonacci αριθμών.

Η σειρά Fibonacci μπορεί να οριστεί αναδρομικά ως εξής:

fibonacci(0) = 0
fibonacci(1) = 1
fibonacci(n) = fibonacci(n – 1) + fibonacci(n – 2)

```
1.  /* Recursive Fibonacci function */
2.
3.  #include <stdio.h>
4.
5.  long fibonacci( long );
6.
7.  int main ( )
8.  {
9.        long result, number;
10.
11.       printf( "Enter an integer: " );
12.       scanf( "%ld", &number );
13.       result = fibonacci( number );
14.       printf( "Fibonacci ( %ld ) = %ld\n", number, result );
15.       return 0;
16. }
17.
18. /* Recursive definition of function Fibonacci */
19. long fibonacci( long n )
20. {
21.       if ( n == 0 || n == 1 )
22.           return n;
23.       else
24.           return fibonacci( n – 1 ) + fibonacci( n – 2 );
25. }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 0
Fibonacci ( 0 ) = 0
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 1
Fibonacci ( 1 ) = 1
```

3ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 2
Fibonacci ( 2 ) = 1
```

4ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 3
Fibonacci ( 3 ) = 2
```

5ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 4
Fibonacci ( 4 ) = 3
```

6ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 5
Fibonacci ( 5 ) = 5
```

7ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 6
Fibonacci ( 6 ) = 8
```

8ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 10
Fibonacci ( 10 ) = 55
```

9ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 20
Fibonacci ( 20 ) = 6765
```

10ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 30
Fibonacci ( 30 ) = 832040
```

11ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter an integer: 35
Fibonacci ( 35 ) = 9227465
```

Περισσότερα Παραδείγματα (Αναδρομή)

Παράδειγμα 36

```
1.  #include <stdio.h>
2.
3.  void show(int);
4.
5.  int main()
6.  {
7.      int i;
8.      printf("Enter Number: ");
9.      scanf("%d", &i);
10.     show(i);
11.     system("pause");
12.     return 0;
13. }
14. void show(int num)
15. {
16.     if (num > 1)
17.         show(num - 1);
18.     printf("Value = %d\n",num);
19. }
```

1. Στην πρώτη κλήση της **show()**, αφού **num = 3 > 1** θα γίνει νέα κλήση της **show()** με όρισμα **num – 1 = 3 – 1 = 2** και η **printf** δεν θα εκτελεστεί. Επίσης, η μνήμη που δεσμεύτηκε για την τοπική μεταβλητή **num** με τιμή **3** δεν αποδεσμεύεται, αφού δεν έχει τερματιστεί η εκτέλεση της συνάρτησης.
2. Στη δεύτερη κλήση της **show()**,αφού **num = 2 > 1** θα γίνει νέα κλήση της **show()** με όρισμα **num – 1 = 2 – 1 = 1**. Παρόμοια με πριν, η **printf** δεν θα εκτελεστεί. Επίσης, η μνήμη που δεσμεύτηκε για τη νέα τοπική μεταβλητή **num** με τιμή **2** δεν αποδεσμεύεται.
3. Στην τρίτη κλήση της **show()**, δεν θα γίνει νέα κλήση της **show()** αφού η τιμή του **num** δεν είναι μεγαλύτερα από το **1**. Άρα, θα εκτελεστεί η **printf** και θα εμφανιστεί **Val = 1**.
4. Στη συνέχεια όλες οι **printf** που δεν είχαν εκτελεστεί θα εκτελεστούν από το τέλος προς την αρχή. Επίσης, σε κάθε τερματισμό της **show()** θα αποδεσμεύεται και η μνήμη που είχε δεσμευτεί για την αντίστοιχη μεταβλητή **num**. Επομένως, το πρόγραμμα θα εμφανίσει:
   Val = 1
   Val = 2
   Val = 3

Παράδειγμα 37

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.
3.  int a = 4;
4.
5.  int main()
6.  {
7.      if (a == 0)
8.          return 0;
9.      else
10.     {
11.         printf("%d ",a—);
12.         main();
13.     }
14.     printf("\n");
15.     system("pause");
16.     return 0;
17. }
```

Παρατηρείστε ότι και η main μπορεί να κληθεί με αναδρομικό τρόπο, Με κάθε κλήση της, η τιμή του ***a*** μειώνεται κατά **1**. Οι κλήσεις της **main()** θα σταματήσουν, όταν η τιμή του ***a*** γίνει **0**. Επομένως, το πρόγραμμα εμφανίζει **4 3 2 1**.

Παράδειγμα 38

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.
3.  int unknown(int, int);
4.
5.  int main()
6.  {
7.      int num1, num2;
8.      printf("Enter First Number: ");
9.      scanf("%d", &num1);
10.     do
11.     {
12.         printf("Enter Second Number (greater than 0): ";
13.         scanf("%d", &num2);
14.     }while(num2 <= 0);
15.     printf(unknown(num1, num2));
16.     printf("\n");
17.     system("pause");
18.     return 0;
19. }
20. int unknown(int num1, int num2)
21. {
22.     if (num2 == 1)
23.         return num1;
24.     else
25.         return num1 + unknown(num1, num2 - 1);
26. }
```

Το πρόγραμμα εμφανίζει το γινόμενο των αριθμό **num1 * num2**.

Για παράδειγμα, αν ο χρήστης εισάγει τους αριθμούς **num1 = 10** και **num2 = 4** τότε η κλήση της **unknown(num1, num2)** επιστρέφει:

num1 + unknown(num1, num2 – 1 = 3) =
num1 + num1 + unknown(num1, num2 – 1 = 2) =
num1 + num1 + num1 + unknown(num1, num2 – 1 = 1)

Η τελευταία κλήση της **unknown(num1, 1)** επιστρέφει **num1**, αφού **num2 = 1**.
Επομένως η τελική επιστροφή είναι:

num1 + num1 + num1 + num1 = 4 * num1 = num2 * num1

Παράδειγμα 39

Ποια είναι η έξοδος του παρακάτω προγράμματος:

```
1.  #include <stdio.h>
2.
3.  int unknown(int [], int);
4.
5.  int main()
6.  {
7.      int arr[] = {10, 20, 30, 40};
8.
9.      printf(unknown(arr, 4));
10.     printf("\n");
11.     system("pause");
12.     return 0;
13. }
14. int unknown(int arr[], int num)
15. {
16.     if (num == 1)
17.         return arr[0];
18.     else
19.         return arr[num - 1] + unknown(arr, num - 1);
20. }
```

Όταν καλείται η unknown() επιστρέφει:

arr[4 – 1 = 3] + unknown(arr, 4 – 1 = 3) =
arr[3] + (arr[3 – 1] + unknown(arr, 3 – 1)) =
arr[3] + arr[2] + arr[2 – 1] + unknown(arr, 2 – 1)) =
arr[3] + arr[2] + arr[1] + unknown(arr, 1)

Η τελευταία κλήση της **unknown(arr, 1)** επιστρέφει **arr[0]** αφού **num = 1**. Επομένως, η τελική τιμή επιστροφής είναι:

arr[3] + arr[2] + arr[1] + arr[0], δηλαδή το πρόγραμμα εμφανίζει το άθροισμα των στοιχείων του πίνακα που είναι ίσο με **10 + 20 + 30 + 40 = 100**.

Παράδειγμα 40

Στα Μαθηματικά, αρκετές φορές, κάποια απλά ερωτήματα είναι πολύ δύσκολο να αποδειχθούν, όπως, για παράδειγμα, το εξής που τέθηκε το 1937 από τον Γερμανό μαθηματικό Lothar Collatz: «θα καταλήγουμε πάντα στη μονάδα (1), από όποιον θετικό κι αν ξεκινήσουμε;».

Σκεφτείτε τώρα έναν θετικό ακέραιο αριθμό **ν** και εκτελέστε τον εξής αλγόριθμο:

- Αν είναι άρτιος, υποδιπλασιάστε τον (ν\2)
- Αν είναι περιττός, τριπλασιάστε τον και προσθέστε του τη μονάδα (3ν + 1).

Επαναλάβατε την παραπάνω διαδικασία για τον αριθμό που προκύπτει και, τότε, θα παρατηρήσετε το εξής εκπληκτικό: όποιον θετικό ακέραιο και να είχατε αρχικά επιλέξει, πάντοτε θα καταλήγετε στο …1 !!!

Για παράδειγμα, αν είχατε επιλέξει τον αριθμό 53, οι αριθμοί που προκύπτουν είναι:
53 → 160 → 80 → 40 → 20 → 10 → 5 → 16 → 8 → 4 → 2 → 1

Το παραπάνω μαθηματικό πρόβλημα είναι γνωστό ως «Εικασία του Collatz» ή «Πρόβλημα των Συρακουσών» και παραμένει άλυτο χωρίς κανείς, έως σήμερα, να το έχει αποδείξει, παρόλο που με τη χρήση Η.Υ. έχει επαληθευτεί ότι, βάσει του παραπάνω αλγορίθμου, όλοι οι θετικοί ακέραιοι μέχρι και το 2^{60} (περίπου 1 πεντάκις εκατομμύριο) καταλήγουν τελικά στο 1.

Αυτό που σας ζητώ είναι, όχι βέβαια να αποδείξετε την «Εικασία του Collatz» για να διεκδικήσετε το βραβείο Φίλντς (Νόμπελ Μαθηματικών), αλλά κάτι σίγουρα πολύ ευκολότερο, δηλαδή απλώς να την επαληθεύσετε με ένα απλό πρόγραμμα.

Δημιουργήστε μία αναδρομική συνάρτηση που να δέχεται σαν παράμετρο έναν θετικό ακέραιο, να τον εμφανίζει στην οθόνη και να υπολογίζει τον επόμενο ακέραιο βάσει της «Εικασίας του Collatz». Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει έναν θετικό ακέραιο και να εμφανίζει τους αριθμούς που προκύπτουν βάσει της «Εικασίας του Collatz» με χρήση της συνάρτησης.

```
1.  #include <stdio.h>
2.
3.  int collatz( int );
4.
5.  int main()
6.  {
7.      int a;
8.
9.      do
10.     {
11.         printf("Enter a positive integer: ");
12.         scanf("%d", &a);
13.     } while(a <= 0);
14.
15.     printf("The result is %d indeed!!!",collatz(a));
16.     printf("\n");
17.     system("pause");
18.     return 0;
19. }
20. int collatz(int n)
21. {
22.     printf("%d ",n);
23.
24.     if(n == 1)
25.         return 1;
26.     else if(n & 1)              // Αν ο n είναι περιττός
27.         return collatz(3*n+1);
28.     else                        // Αν ο n είναι άρτιος
29.         return collatz(n/2);
30. }
```

ΠΙΝΑΚΕΣ (ARRAYS)

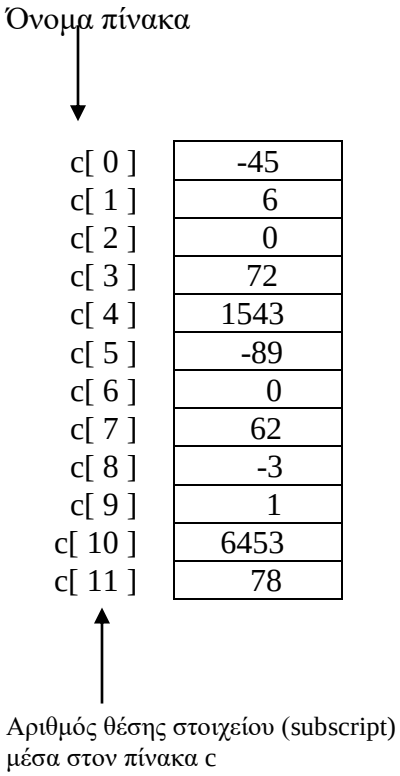
Οι **πίνακες** είναι δομές δεδομένων που αποτελούνται από αντικείμενα συναφή δεδομένων του ιδίου τύπου.

Μεταγενέστερα θα δούμε τη δομή **struct** – μια δομή δεδομένων που αποτελείτε από αντικείμενα συναφή δεδομένων διαφορετικού τύπου.

Οι **πίνακες** είναι στατικές οντότητες στο ότι παραμένουν με το ίδιο μέγεθος καθ' όλη την διάρκεια της εκτέλεσης του προγράμματος (μπορούν όμως βεβαίως να οριστούν με αυτόματη κλάση αποθήκευσης και ως εκ τούτου να δημιουργούνται και να καταστρέφονται κάθε φορά που τα μπλοκ μέσα στα οποία δηλώθηκαν μπαίνουν και βγαίνουν).

Ένας **πίνακας** είναι μια ομάδα θέσεων μνήμης που σχετίζονται με το γεγονός ότι όλες έχουν το ίδιο όνομα και τον ίδιο τύπο. Για να αναφερθούμε σε ένα συγκεκριμένο χώρο ή στοιχείο του πίνακα χρησιμοποιούμε έναν αριθμό θέσης που ονομάζεται **δείκτης (subscript)** του συγκεκριμένου στοιχείου του πίνακα.

Ένα **subscript** πρέπει να είναι ακέραιος αριθμός ή παράσταση που μας δίνει αποτέλεσμα ακεραίου αριθμού. Το πρώτο στοιχείο του πίνακα είναι το *zeroth στοιχείο*.



Παραδείγματα

1. *c[0]* είναι ίσον με το **-45**

2. *c[6]* είναι ίσον με το **0**

3. *c[a + b] += 2* εάν **a = 5** και **b = 6** τότε η εντολή προσθέτει **2** στο **11ο** στοιχείο

4. Για να τυπώσουμε το άθροισμα των τριών πρώτων στοιχείων που **c**, γράφουμε

```
printf( "%d", c[ 0 ] + c[ 1 ] + c[ 2 ] );
```

5. Για να διαιρέσουμε τη τιμή του έβδομου στοιχείου του πίνακα **c** δια **2** και να αντιστοιχίσουμε το αποτέλεσμα με τη μεταβλητή **x**, γράφουμε

```
x = c[ 6 ] / 2;
```

Ορισμός πινάκων

Οι πίνακες καταλαμβάνουν χώρο στη μνήμη.

Για να πούμε στον Η/Υ να δεσμεύσει **12** στοιχεία για τον **c** πίνακα ακεραίων, η δήλωση

```
int c[ 12 ];
```

χρησιμοποιείται.

Για να πούμε στον Η/Υ να δεσμεύσει **100** στοιχεία για τον **b** πίνακα ακεραίων και **27** στοιχεία για τον **x** πίνακα ακεραίων, η δήλωση

```
int b[ 100 ], x[ 27 ];
```

χρησιμοποιείται.

Οι πίνακες μπορούν να οριστούν για να περιέχουν και στοιχεία άλλων τύπων δεδομένων. Για παράδειγμα, ένας πίνακας τύπου **char** μπορεί να χρησιμοποιηθεί για να αποθηκεύσουμε μια γραμματοσειρά χαρακτήρων, ένα θέμα που θα συζητήσουμε μεταγενέστερα.

Η σχέση μεταξύ δεικτών (pointers) και πινάκων θα συζητηθεί επίσης σε μεταγενέστερο στάδιο.

Πριν να δούμε και ολοκληρωμένα παραδείγματα με πίνακες, ας δούμε ξανά ένα συμπληρωμένο πίνακα προτεραιότητας τελεστών.

Πίνακας προτεραιότητας τελεστών (Operator precedence and associativity)

Τελεστής	Προαιρετική ιδιότητα	Είδος
( ) [ ]	αριστερά στα δεξιά	παρενθέσεις
++ -- + - ! (type)	δεξιά στα αριστερά	μοναδιαίοι
* / %	αριστερά στα δεξιά	πολλαπλασιαστικά
+ -	αριστερά στα δεξιά	πρόσθετης ύλης
< <= > >=	αριστερά στα δεξιά	σχεσιακά
== !=	αριστερά στα δεξιά	ισότητας
&&	αριστερά στα δεξιά	λογικό AND
||	αριστερά στα δεξιά	λογικό OR
?:	δεξιά στα αριστερά	συνθήκη
= += -= *= /= %=	δεξιά στα αριστερά	ανάθεση
,	αριστερά στα δεξιά	κόμμα

Παραδείγματα χρησιμοποιώντας ΠΙΝΑΚΕΣ (ARRAYS)

Παράδειγμα 41

Αυτό το παράδειγμα δίνει σαν αρχική τιμή **0** σε όλα τα **10** στοιχεία του πίνακα **n** και εκτυπώνει τον πίνακα σε πινακοποιημένη μορφή (tabular format).

```
1.  /* Initializing an array */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.        int n[ 10 ], i;
8.
9.        for ( i = 0; i <= 9; i++ )                    /* initialize array */
10.             n[ i ] = 0;
11.
12.       printf( "%s%13s\n", "Element", "Value" );
13.       for ( i = 0; i <= 9; i++ )                    /* print array */
14.             printf("%7d%13d\n", i, n[ i ] );
15.
16.       return 0;
17. }
```

ΕΞΟΔΟΣ:

```
Element         Value
      0             0
      1             0
      2             0
      3             0
      4             0
      5             0
      6             0
      7             0
      8             0
      9             0
```

Παράδειγμα 42

Το ακόλουθο παράδειγμα δίνει **10** διαφορετικές αρχικές τιμές σε ένα πίνακα ακεραίων (ΓΡΑΜΜΗ 7) και εκτυπώνει τον πίνακα σε πινακοποιημένη μορφή (tabular format).

```
1.  /* Initializing an array with a declaration */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int n[ 10 ] = { 32, 27, 64, 18, 95, 14, 90, 70, 60, 37 };
8.      int i;
9.
10.     printf( "%s%13s\n", "Element", "Value" );
11.
12.     for ( i = 0; i <= 9; i++ )                  /* print array */
13.         printf("%7d%13d\n", i, n[ i ] );
14.
15.     return 0;
16. }
```

ΕΞΟΔΟΣ:

```
Element         Value
      0            32
      1            27
      2            64
      3            18
      4            95
      5            14
      6            90
      7            70
      8            60
      9            37
```

Σημείωση:

1. Εάν υπήρχαν λιγότερες τιμές στην ΓΡΑΜΜΗ 7 από τον αριθμό στοιχείων του πίνακα στο υπόλοιπο των στοιχείων δίνεται η τιμή 0. Έτσι,

   ```
   int n[ 10 ] = { 0 };
   ```

   το πρώτο στοιχείο παίρνει τη τιμή **0** αυτόματα και όλα τα στοιχεία παίρνουν επίσης τη τιμή **0**.

2. Εάν δεν συμπεριλάβετε το μέγεθος του πίνακα όταν τον ορίζετε με αρχικές τιμές ο αριθμός των στοιχείων του πίνακα θα είναι ίσος με τον αριθμό των τιμών που δίνονται αρχικά στον ορισμό του πίνακα.

   Για παράδειγμα:

   ```
   int n[ ] = { 1, 2, 3, 4, 5 };
   ```

3. Να θυμάστε ότι με τη **C** οι πίνακες δεν παίρνουν αρχικές τιμές αυτόματα εάν δεν το κάνουμε εμείς ρητά.

4. Το ακόλουθο είναι ΛΑΘΟΣ

   ```
   int n[ 5 ] = { 2, 27, 64, 18, 95, 14 };
   ```

   για το λόγο ότι υπάρχουν περισσότερες αρχικές τιμές από τον αριθμό των στοιχείων.

Παράδειγμα 43

Αυτό το πρόγραμμα δίνει τις ακόλουθες τιμές σε ένα 10-στοιχείων πίνακα που ονομάζεται **s** : **2,4,6,…….,20** και εκτυπώνει τον πίνακα σε πινακοποιημένη μορφή (tabular format). Ο καθοδηγητής του προ-επεξεργαστή **#define** χρησιμοποιείται για να δηλώσουμε κάποια σταθερή με τη **C**.

```
1.  /* Initialize the elements of array s to
2.     the even integers from2 to 20 */
3.
4.  #include <stdio.h>
5.  #define SIZE 10
6.
7.  int main()
8.  {
9.        int s[ SIZE ], j;
10.
11.       for ( j = 0 ; j <= SIZE-1; j++ )          /* set the values */
12.          s[ j ] = 2 + 2 * j;
13.
14.       printf( "%s%13s\n", "Element", "Value" );
15.
16.       for ( j  = 0; j < SIZE;  j++ )            /* print the values */
17.             printf("%7d%13d\n", j, s[ j ] );
18.
19.       return 0;
20. }
```

ΕΞΟΔΟΣ:

```
Element         Value
      0             2
      1             4
      2             6
      3             8
      4            10
      5            12
      6            14
      7            16
      8            18
      9            20
```

Παράδειγμα 44

Αυτό το πρόγραμμα αθροίζει τις τιμές που βρίσκονται στο 12-στοιχείων πίνακα ακεραίων **a**.

```
1.  /* Compute the sum of the elements of the array */
2.
3.  #include <stdio.h>
4.  #define SIZE 12
5.
6.  int main()
7.  {
8.        int a[ SIZE ] = { 1, 3, 5, 4, 7, 2, 99,
9.                          16,45, 67, 89, 45 } ;
10.       int i, total = 0;
11.
12.       for ( i = 0 ; i <= SIZE - 1; i++ )
13.           total += a[ i ];
14.
15.      printf("Total of array element nalues is %d\n", total);
16.      return 0;
17. }
```

ΕΞΟΔΟΣ:

```
Total of array element values is 383
```

Παράδειγμα 45

Αυτό το πρόγραμμα χρησιμοποιεί ένα 11-στοιχείων πίνακα, το **frequency**, για να μετρήσει τον αριθμό των εμφανίσεων κάθε απάντησης. Οι απαντήσεις βρίσκονται στον πίνακα **responses**.

```
1.  /* Student poll program */
2.
3.  #include <stdio.h>
4.  #define RESPONSE_SIZE 40
5.  #define FREQUENCY_SIZE 11
6.
7.  int main()
8.  {
9.       int answer, rating, frequency[ FREQUENCY_SIZE ] = { 0 };
10.      int responses[ RESPONSE_SIZE ] =
11.        { 1, 2, 6, 4, 8, 5, 9, 7, 8, 10,
12.          1, 6, 3, 8, 6, 10, 3, 8, 2, 7,
13.          6, 5, 7, 6, 8, 6, 7, 5, 6, 6,
14.          5, 6, 7, 5, 6, 4, 8, 6, 8, 10 };
15.
16.    for ( answer = 0 ; answer <= RESPONSE_SIZE – 1; answer++ )
17.         ++frequency[ responses [ answer ] ];
18.
19.    printf( "%s%17s\n", "Rating", "Frequency" );
20.
21.    for ( rating = 1; rating <= FREQUENCY_SIZE – 1; rating++ )
22.          printf( “%6d%17d\n”, rating, frequency[ rating ] );
23.
24.    return 0;
25. }
```

ΕΞΟΔΟΣ:

```
Rating        Frequency
     1                2
     2                2
     3                2
     4                2
     5                5
     6               11
     7                5
     8                7
     9                1
    10                3
```

ΓΡΑΜΜΜΗ 16: Ο βρόχος **for** παίρνει τα στοιχεία του πίνακα **responses**, ένα κάθε φορά, και αυξάνει κατά 1 έναν από τους δέκα μετρητές (**frequency [1] - frequency[10]** που βρίσκονται στον πίνακα **frequency**. Η εντολή-κλειδί του βρόχου είναι η
ΓΡΑΜΜΗ 17

++frequency[responses [answer]];

Αυτή η εντολή αυξάνει το κατάλληλο στοιχείο (μετρητή) του πίνακα **frequency** ανάλογα με τη τιμή του **responses[answer]**. Για παράδειγμα, όταν η τιμή της μεταβλητής ελέγχου του βρόχου **answer** είναι **0**, το **responses [answer]** είναι **1**, έτσι ώστε το **frequency[responses [answer]];** στην πραγματικότητα μεταφράζεται σαν

++frequency [1];

το οποίο αυξάνει το στοιχείο **1** κατά ένα.

Όταν η μεταβλητή **answer** γίνεται **1**, **responses [answer]** είναι **2**, έτσι ώστε το **frequency[responses [answer]];** μεταφράζεται σαν

++frequency [2];

το οποίο αυξάνει το στοιχείο **2** κατά ένα.

Όταν η μεταβλητή **answer** γίνεται **2**, **responses [answer]** είναι **6** έτσι ώστε το **frequency[responses [answer]];** μεταφράζεται σαν

++frequency [6];

το οποίο αυξάνει το στοιχείο **6** κατά ένα.

Σημείωση: εάν τα δεδομένα περιέχουν μη έγκυρη τιμή, όπως 13, το πρόγραμμα θα προσπαθούσε να προσθέσει **1** στο **frequency [13]** και αυτό θα ήταν έξω από τα όρια του πίνακα. Η **C** δεν έχει αυτόματο έλεγχο που ελέγχει αυτό και ως εκ τούτου θα πάρουμε ένα λάθος από τον Η/Υ όταν τρέξουμε το πρόγραμμα χωρίς καμιά προειδοποίηση. Ο προγραμματιστής θα πρέπει να εξασφαλίζει ότι όλες οι αναφορές του πίνακα παραμένουν εντός των ορίων του.

Παράδειγμα 46

Το επόμενο παράδειγμα μας διαβάζει αριθμούς από τον πίνακα και δημιουργεί μια γραφική παράσταση των δεδομένων της μπάρας γραφήματος ή ιστογράμματος.

```
1.  /* Histogram printing program */
2.
3.  #include <stdio.h>
4.  #define SIZE 10
5.
6.  int main()
7.  {
8.       int n[ SIZE ]= { 19, 3, 15, 7, 11, 9, 13, 5, 17,1 };
9.       int i, j;
10.
11.          printf( "%s%13s%17s\n", "Element", "Value","Histogram" );
12.
13.     for ( i = 0 ; i < SIZE – 1; i++ ) {
14.          printf("%7d%13d       ", i, n[ i ] );
15.
16.     for ( j = 0 ; j < n[ i ]; j++ )          /* print one bar */
17.          printf("%c",'*' );
18.
19.     printf("\n");
20.     }
21.
22.     return 0;
23. }
```

ΕΞΟΔΟΣ:

```
Element         Value         Histogram
      0            19         ********************
      1             3         ***
      2            15         ***************
      3             7         *******
      4            11         ***********
      5             9         *********
      6            13         *************
      7             5         *****
      8            17         *****************
      9             1         *
```

Παράδειγμα 47

Αυτό το πρόγραμμα υπολογίζει τη συχνότητα του ριξίματος ζαριών. Ρίχνουμε ένα ζάρι 6000 φορές τυχαία και χρησιμοποιώντας τον μηχανισμό τυχαίων αριθμό της C το πρόγραμμα θα μας δημιουργήσει ένα πίνακα που δείχνει τη συχνότητα του 1, 2, 3, 4, 5, και 6.

```
1.  /* Roll a six-sided die 6000 times */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.  #include <time.h>
6.  #define SIZE 7
7.
8.  int main ( )
9.  {
10.     int face, roll, frequency [ SIZE ] = { 0 };
11.
12.     srand( time( NULL ) );
13.
14.     for ( roll = 1; roll <= 6000; roll++ ) {
15.         face = rand( ) % 6 + 1;
16.         ++frequency [ face ];
17.     }
18.
19.    printf(" %s%17s\n", "Face", "Frequency" );
20.
21.    for (  face = 1; face <= SIZE – 1; face++ )
22.        printf(" %4d%17d\n", face, frequency [ face ] );
23.
24.    return 0;
25. }
```

ΕΞΟΔΟΣ:

```
Face          Frequency
   1               1037
   2                987
   3               1013
   4               1028
   5                952
   6                983
```

ΓΡΑΜΜΑΤΟΣΕΙΡΕΣ (STRINGS)

- Οι πίνακες μπορούν να περιέχουν δεδομένα οποιουδήποτε τύπου. Όπως έχουμε δει μια γραμματοσειρά όπως *"hello"* είναι στην πραγματικότητα, στη **C**, ένας **στατικός** (**static**) πίνακας με ατομικούς χαρακτήρες.

  Για παράδειγμα,

  ```
  char string1 [] =  "first";
  ```

  δίνει αρχικές τιμές στα στοιχεία του πίνακα **string1** τους ατομικούς χαρακτήρες της σταθεράς γραμματοσειράς *"first"* . Το μέγεθος του πίνακα εξαρτάται από τον μεταγλωττιστή.

- Είναι σημαντικό να γνωρίζουμε ότι η γραμματοσειρά *"first"* περιέχει πέντε χαρακτήρες συν ένας **ειδικός** (**special**) χαρακτήρας που ονομάζεται **null**. Ως εκ τούτου ο πίνακας **string1** στην πραγματικότητα περιέχει 6 χαρακτήρες. Ο τελευταίος χαρακτήρας (**null**) αντιπροσωπεύεται με το'**\0**'. Όλες οι γραμματοσειρές στη **C** τελειώνουν με αυτό το χαρακτήρα.

- Στους πίνακες χαρακτήρων επίσης μπορούν να δοθούν αρχικές τιμές ως ακολούθως:

  ```
  char string1 [] = { 'f', 'i', 'r', 's', 't', '\0' };
  ```

  που ισοδυναμεί με το προηγούμενο παράδειγμα.

- Για το λόγο ότι οι γραμματοσειρές είναι πίνακες μπορούμε να τις διαχειριστούμε χαρακτήρα με χαρακτήρα.

  Για παράδειγμα,

  ```
  string1 [ 0 ] είναι ο χαρακτήρας 'f' και
  string1 [ 3 ] είναι ο χαρακτήρας 's'
  ```

- Μπορούμε να διαβάσουμε γραμματοσειρές κατευθείαν από το πληκτρολόγιο χρησιμοποιώντας την εντολή **scanf** και τη μετατροπή **%s**.

  ```
  char string2 [ 20 ];
          :
          :
          :
  scanf( "%s", string2 );
  ```

- Το ακόλουθο παράδειγμα,
  διαβάζει μια γραμματοσειρά από το πληκτρολόγιο στο **string2**. Σημειώστε ότι το όνομα της γραμματοσειράς περνιέται στην συνάρτηση **scanf** δίχως να βάλουμε

μπροστά το **&** όπως κάναμε με άλλες μεταβλητές. Στην πραγματικότητα το **&** χρησιμοποιείται για να δώσει στην συνάρτηση **scanf** τη διεύθυνση της μεταβλητής στη μνήμη έτσι ώστε μια τιμή να αποθηκευτεί εκεί. Αργότερα θα δούμε ότι το όνομα ενός πίνακα είναι από μόνο του η διεύθυνση της πρώτης τιμής του πίνακα; ως εκ τούτου το **&** δεν είναι αναγκαίο.

- Μπορούμε να τυπώσουμε γραμματοσειρές χρησιμοποιώντας **printf** και το **%s**.

  Για παράδειγμα,

  printf(("%s\n", string2);

Παράδειγμα 48

```
1.  /* Treating character arrays as strings */
2.
3.  #include <stdio.h>
4.
5.  int main ( )
6.  {
7.      char string1 [ 20 ], string2 [] = "string literal";
8.      int i;
9.
10.     printf ( "Enter a string: " );
11.     scanf( "%s", string1 );
12.     printf( "string1 is: %s\nstring2: is %s\n "
13.             "string1 with spaces between characters is:\n",
14.              string1, string2 );
15.
16.     for (  i = 0; string1 [ i ] != '\0'; i++ )
17.         printf(" %c ", string1 [ i ] );
18.
19.      printf( "\n" );
20.     return 0;
21. }
```

ΕΞΟΔΟΣ:

```
Enter a string: Hello
string1 is: Hello
string2 is: string literal
string1 with spaces between characters is:
H e l l o
```

- Μια στατική τοπική μεταβλητή μιας συνάρτησης υπάρχει καθ' όλη τη διάρκεια του προγράμματος αλλά είναι ορατή μόνο στο σώμα της συνάρτησης.

- Μπορούμε να εφαρμόσουμε το **static** σε ορισμό ενός πίνακα έτσι ώστε ο πίνακας να μη δημιουργείται και να του δίνονται αρχικές τιμές κάθε φορά που η συνάρτηση καλείται και να μη καταστρέφεται κάθε φορά που η συνάρτηση τερματίζεται. Αυτό μειώνει το χρόνο εκτέλεσης ιδιαίτερα για προγράμματα με συχνές κλήσεις συναρτήσεων που περιέχουν μεγάλους πίνακες.

Παράδειγμα 49

```
1.  /* Static arrays are initialized to zero */
2.
3.  #include <stdio.h>
4.
5.  void staticArrayInit( void );
6.  void automaticArrayInit( void );
7.
8.  int main()
9.  {
10.     printf("First call to each function:\n");
11.     staticArrayInit();
12.     automaticArrayInit();
13.     printf( "\n\nSecond call to each function:\n" );
14.     staticArrayInit();
15.     automaticArrayInit();
16.     return 0;
17. }
18.
19. /* function to demonstrate static local array */
20. void staticArrayInit(void)
21. {
22.       static int a[ 3 ];
23.       int i;
24.
25.       printf( "\nValues on entering staticArrayInit:\n" );
26.
27.       for ( i = 0; i <= 2; i++ )
28.             printf( "Array1[%d] = %d ", i, a[ i ] );
29.
30.       printf( "\nValues on exiting staticArrayInit:\n" );
31.
32.       for ( i = 0; i <= 2; i++ )
33.             printf( "Array1[%d] = %d ", i, a[ i ] += 5 );
34. }
35.
36.     /* function to demonstrate automatic local array */
37. void automaticArrayInit(void)
38. {
```

```
39.     int a[ 3 ] = { 1, 2, 3 }, i;
40.
41.     printf( "\nValues on entering automaticArrayInit:\n" );
42.
43.     for ( i = 0; i <= 2; i++ )
44.         printf( "Array1[%d] = %d ", i, a[ i ] );
45.
46.     printf( "\nValues on exiting automaticArrayInit:\n" );
47.
48.     for ( i = 0; i <= 2; i++ )
49.         printf( "Array1[%d] = %d ", i, a[ i ] += 5 );
50. }
```

ΕΞΟΔΟΣ:

```
First call to each function

Values on entering staticArrayInit:
Array1[ 0 ] = 0    Array1[ 1 ] = 0    Array1[ 2 ] = 0
Values on exiting staticArrayInit:
Array1[ 0 ] = 5    Array1[ 1 ] = 5    Array1[ 2 ] = 5

Values on entering automaticArrayInit:
Array1[ 0 ] = 1    Array1[ 1 ] = 2    Array1[ 2 ] = 3
Values on exiting automaticArrayInit:
Array1[ 0 ] = 6    Array1[ 1 ] = 7    Array1[ 2 ] = 8

Second call to each function

Values on entering staticArrayInit:
Array1[ 0 ] = 5    Array1[ 1 ] = 5    Array1[ 2 ] = 5
Values on exiting staticArrayInit:
Array1[ 0 ] = 10    Array1[ 1 ] = 10    Array1[ 2 ] = 10

Values on entering automaticArrayInit:
Array1[ 0 ] = 1    Array1[ 1 ] = 2    Array1[ 2 ] = 3
Values on exiting automaticArrayInit:
Array1[ 0 ] = 6    Array1[ 1 ] = 7    Array1[ 2 ] = 8
```

ΠΕΡΑΣΜΑ ΠΙΝΑΚΩΝ ΣΕ ΣΥΝΑΡΤΗΣΕΙΣ

- Για να περάσουμε ένα πίνακα σαν όρισμα σε μια συνάρτηση καθορίζουμε το όνομα του πίνακα χωρίς οποιαδήποτε αγκύλη.

  Για παράδειγμα, εάν

  int hourlyTemperature[24];

  δηλώνεται τότε η εντολή κλήσης της συνάρτησης είναι

  modifyArray(hourlyTemperature, 24);

  που περνά τον πίνακα **hourlyTemperatures** και το **size** (μέγεθος) στη συνάρτηση.

- Οι πίνακες περνιούνται σε συναρτήσεις **διά αναφοράς** (by reference) (address), έτσι ώστε εάν ο πίνακας αλλάξει μέσα στη συνάρτηση αυτός τροποποιείται και στις αρχικές του θέσεις της μνήμης. Το όνομα ενός πίνακα είναι στην πραγματικότητα η διεύθυνση του πρώτου στοιχείου του πίνακα.

Παράδειγμα 50

```
1.  /* The name of an array is the same as &array[ 0 ] */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char array[ 5 ];
8.
9.      printf("    array = %p\n&array[ 0 ] = %p\n"
10.            "    &array = %p\n", array,
11.                &array[ 0 ] , &array );
12.     return 0;
13. }
```

ΕΞΟΔΟΣ:

```
array = 0065FDF0
array[ 0 ] = 0065FDF0
&array = 0065FDF0
```

Η προδιαγραφή της μετατροπής %p είναι μια ειδική μετατροπή για την εκτύπωση διευθύνσεων σαν δεκαεξαδικό αριθμό (με βάση 16)..

- Μπορούμε να περάσουμε ατομικά στοιχεία ενός πίνακα σαν ορίσματα (διά τιμή) όπως μεταβλητές. Αυτά τα απλά ενιαία κομμάτια δεδομένων ονομάζονται **scalars** ή **scalar quantities**.

  Για παράδειγμα, η κεφαλίδα της συνάρτησης *modifyArray* μπορεί να γραφτεί ως ακολούθως:

  void modifyArray(int b[ ], int size)
  ↓
  το μέγεθος δεν χρειάζεται

  Το πρωτότυπο θα είναι

  void modifyArray(int [ ], int);

  ή

  void modifyArray(int anyArrayName[ ], int anyvariableName);

  αλλά όπως έχουμε πει ο μεταγλωττιστής της C αγνοεί τα ονόματα μεταβλητών στα πρωτότυπα.

Παράδειγμα 51

```
1.  /* Passing arrays and individual elements to functions */
2.
3.  #include <stdio.h>
4.  #define SIZE 5
5.
6.  void modifyArray(int [ ], int);          /* appears strange */
7.  void modifyElement( int );
8.
9.  int main()
10. {
11.    int a[ SIZE ] = { 0, 1, 2, 3, 4 }, i;
12.
13.     printf( "Effects of passing entire array call "
14.             "by reference:\n\nThe values of the "
15.             "original array are:\n");
16.
17.     for ( i = 0; i <= SIZE – 1; i++ )
18.        printf( "%3d", a[ i ] );
19.
20.    printf(  "\n" );
21.    modifyArray( a, SIZE );     /* passed call by reference */
```

```
22.     printf( “The values of the modiofied array are:\n” );
23.
24.     for ( i = 0; i <= SIZE – 1; i++ )
25.         printf( “%3d”, a[ i ] );
26.
27.     printf( "\n\nEffects of passing array element call "
28.             “by value:\n\nThe value of a[ 3 ] is %d\n", a[ 3 ] );
29.     modifyElement( a[ 3 ] );
30.     printf( "The value of a[ 3 ] is %d\n ", a[ 3 ] );
31.     return 0;
32. }
33.
34. void modifyArray( int b[ ], int size )
35. {
36.      int j;
37.
38.     for (j = 0; j <= size – 1; j++ )
39.          b[ j ] *= 2;
40. }
41.
42. void modifyElement( int e );
43. {
44.     printf( “Value in modifyElement is %d\n”, e *= 2 );
45. }
```

ΕΞΟΔΟΣ:

```
Effects of passing entire array call by reference:

The values of the original array are:
        0 1 2 3 4
The values of the modified array are:
        0 2 4 6 8

Effects of passing array element call by value:

The value of a[3] is 6
Value in modifyElement is 12
The value of a[3] is 6
```

Δεδομένου ότι οι πίνακες πάντα περνιούνται διά αναφοράς και οι αλλαγές των στοιχείων του είναι δύσκολο να ελεγχτούν, η C μας παρέχει το προσδιοριστικό τύπου **const** που εμποδίζει αυτές τις αλλαγές.

Παράδειγμα 52

```
1.  /* Demostrating the const type qualifier with arrays */
2.
3.  #include <stdio.h>
4.
5.  void tryToModifyArray( const int [] );
6.
7.  int main()
8.  {
9.       int a[] = { 10, 20, 30 };
10.
11.     tryToModifyArray( a );
12.     printf( "%d %d %d\n", a[ 0 ], a[ 1 ], a[ 2 ] );
13.    return 0;
14. }
15.
16. void tryToModifyArray( const int b[] )
17. {
18.    b[ 0 ] = 2;          /* error */
19.    b[1 ] = 2;           /* error */
20.    b[ 2 ] = 2;          /* error */
21. }
```

ΕΞΟΔΟΣ:

```
Compiling…
Fig06_14.c(18) : error C2166: 1-value specifies const object
Fig06_14.c(19) : error C2166: 1-value specifies const object
Fig06_14.c(20) : error C2166: 1-value specifies const object
```

ΤΑΞΙΝΟΜΗΣΗ ΠΙΝΑΚΩΝ (SORTING)

Η ταξινόμηση δεδομένων (π.χ. τοποθετώντας τα δεδομένα σε μια συγκεκριμένη σειρά όπως αύξουσα (ascending) ή φθίνουσα (descending) είναι μια από τις πιο σημαντικές εφαρμογές στην πληροφορική.

Παράδειγμα 53 (Ταξινόμηση ΦΥΣΑΛΙΔΑΣ (BUBBLE Sort))

```
1.  /* This program sorts an array's values into
2.     ascending order */
3.
4.  #include <stdio.h>
5.  #define SIZE 10
6.
7.  int main()
8.  {
9.       int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
10.      int i, pass, hold;
11.
12.      printf( "Data items is original order\n" );
13.      for ( i = 0; i <= SIZE – 1; i++ )
14.         printf( "%4d", a[ i ] );
15.      for ( pass = 1; pass <= SIZE – 1; pass++ )      /* passes */
16.           for ( i = 0; i <= SIZE – 2; i++ )          /* one pass */
17.               if ( a[ i ] > a[ i+1 ] ) {             /* one comparison */
18.                  hold = a[ i ];                      /* one swap */
19.                  a[ i ] = a[ i + 1 ];
20.                  a[ i + 1 ] = hold;
21.               }
22.
23.      printf(" \nData items in ascending order\n" );
24.
25.       for ( i = 0; i <= SIZE – 1; i++ )
26.          printf( "%4d", a[ i ] );
27.
28.      printf( "\n");
29.
30.      return 0;
31. }
```

ΕΞΟΔΟΣ:

```
Data items in original order
   2   6   4   8   10  12   89   68   45   37
Data items in ascending order
   2   4   6   8   10  12   37   45   68   89
```

Παράδειγμα 54 (Υπολογίζουμε το mean, Median και mode χρησιμοποιώντας πίνακες)

```
1.  /* This program introduces the topic of survey data analysis.
2.     It computes the mean, median, and  mode of the data */
3.
4.  #include <stdio.h>
5.  #define SIZE 99
6.
7.  void mean( const int [] );
8.  void median( int [] );
9.  void mode( int [], const int [] ) ;
10. void bubbleSort( int [] );
11. void printArray( const int [] );
12.
13. int main()
14. {
15.      int frequency[ 10 ] = { 0 };
16.      int response[ SIZE ] =
17.                { 6, 7, 8, 9, 8, 7, 8, 9, 8, 9,
18.                  7, 8, 9, 5, 9, 8, 7, 8, 7, 8,
19.                  6, 7, 8, 9, 3, 9, 8, 7, 8, 7,
20.                  7, 8, 9, 8, 9, 8, 9, 7, 8, 9,
21.                  6, 7, 8, 7, 8, 7, 9, 8, 9, 2,
22.                  7, 8, 9, 8, 9, 8, 9, 7, 5, 3,
23.                  5, 6, 7, 2, 5, 3, 9, 4, 6, 4,
24.                  7, 8, 9, 6, 8, 7, 8, 9, 7, 8,
25.                  7, 4, 4, 2, 5, 3, 8, 7, 5, 6,
26.                  4, 5, 6, 1, 6, 5, 7, 8, 7 };
27.
28.      mean( response );
29.      median( response );
30.      mode( frequency, response );
31.       return 0;
32. }
33.
34. void mean( const int answer[] )
35. {
36.       int j, total = 0;
37.
38.       printf( "%s\n%s\n%s\n", "*********", "  Mean", "*********" );
39.
40.       for ( j = 0; j <= SIZE - 1; j++ )
41.             total += answer[  j  ];
42.
43.       printf( "The mean is the average value of the data\n"
44.                  "items. The mean is equal to the total of\n"
45.                  "all the data items divided by the number\n"
```

```
                "of data items ( %d ). The mean value for\n"
                "this run is: %d / %d = %.4f\n\n",
                SIZE, total, SIZE, (  double  ) total / SIZE );
}

void median( int answer[] )
{
      printf( "\n%s\n%s\n%s\n%s",
              "********", " Median", "********",
              "The unsorted array of responses is" );

      printArray( answer );
      bubbleSort( answer );
      printf( "\n\nThe sorted array is" );
      printArray( answer );
      printf( "\n\nThe median is element %d of\n"
              "the sorted %d element array.\n"
              "For this run the median is %d\n\n",
              SIZE / 2, SIZE, answer[ SIZE / 2 ] );
}

void mode( int freq[], const int answer[] )
{
      int rating, j, h, largest = 0, modeValue = 0;

      printf( "\n%s\n%s\n%s\n",
               "********", "  Mode", "********" );

      for ( rating = 1; rating <= 9; rating++ )
            freq[ rating ] = 0;

      for ( j = 0; j <= SIZE - 1; j++ )
           ++freq[ answer[ j ] ];

      printf( "%s%11s%19s\n\n%54s\n%54s\n\n",
              "Response", "Frequency", "Histogram",
              "1    1    2    2", "5    0    5    0    5" );

      for ( rating = 1; rating <= 9; rating++ ) {
          printf( "%8d%11d         ", rating, freq[ rating ] );

          if ( freq[ rating ] > largest ) {
              largest = freq[ rating ];
             modeValue = rating;
          }

```

```
92.             for ( h = 1; h <= freq[ rating ]; h++ )
93.                 printf( "*" );
94.
95.             printf( "\n" );
96.         }
97.
98.         printf( "The mode is the most frequent value.\n"
99.                 "For this run the mode is %d which occurred"
100.                " %d times.\n", modeValue, largest );
101. }
102.
103. void bubbleSort( int a[] )
104. {
105.        int pass, j, hold;
106.
107.        for ( pass = 1; pass <= SIZE - 1; pass++ )
108.
109.        for ( j = 0; j <= SIZE - 2; j++ )
110.
111.             if ( a[ j ] > a[ j + 1 ] ) {
112.                 hold = a[ j ];
113.                 a[ j ] = a[ j + 1 ];
114.                 a[ j + 1 ] = hold;
115.             }
116. }
117.
118. void printArray( const int a[] )
119. {
120.        int j;
121.
122.        for ( j = 0; j <= SIZE - 1; j++ ) {
123.
124.             if ( j % 20 == 0 )
125.                 printf( "\n" );
126.
127.             printf( "%2d", a[ j ] );
128.         }
```

ΕΞΟΔΟΣ:

```
********
 Mean
********
The mean is the average value of the data
items. The mean is equal to the total of
all the data items divided by the number
of data items (99). The mean value for
this run is: 681 / 99 = 6.8788

********
 Median
********
The unsorted array of responses is
6 7 8 9 8 7 8 9 8 9 7 8 9 5 9 8 7 8 7 8
6 7 8 9 3 9 8 7 8 7 7 8 9 8 9 8 9 7 8 9
6 7 8 7 8 7 9 8 9 2 7 8 9 8 9 8 9 7 5 3
5 6 7 2 5 3 9 4 6 4 7 8 9 6 8 7 8 9 7 8
7 4 4 2 5 3 8 7 5 6 4 5 6 1 6 5 7 8 7

The sorted array is
1 2 2 2 3 3 3 3 4 4 4 4 4 5 5 5 5 5 5 5
5 6 6 6 6 6 6 6 6 6 7 7 7 7 7 7 7 7 7 7
7 7 7 7 7 7 7 7 7 7 7 7 7 8 8 8 8 8 8 8
8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8 8
9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9 9

The median is element 49 of
the sorted 99 element array.
For this run the median is 7
```

```
********
 Mode
********
        Response    Frequency       Histogram
                                              1       1       2       2
                                        5     0       5       0       5
              1           1         *
              2           3         ***
              3           4         ****
              4           5         *****
              5           8         ********
              6           9         *********
              7          23         ***********************
              8          27         ***************************
              9          19         *******************
        The mode is the most frequent value.
        For this run the mode is 8 which occurred 27 times.
```

<u>Παράδειγμα 55</u> (Αναζήτηση σε πίνακες (Searching arrays) – Γραμμική αναζήτηση (Linear search)

- Ο πίνακας δεν χρειάζεται να είναι σε σειρά. Είναι η πιο απλή μέθοδος αναζήτησης αλλά είναι η πιο αργή σε μέσο όρο. Είναι καλή μέθοδος για μικρούς πίνακες που δεν είναι σε σειρά.

- Κατά μέσο όρο το πρόγραμμα θα πρέπει να συγκρίνει το κλειδί αναζήτησης με τα μισά στοιχεία του πίνακα.

```
1.  /* Linear search of an array */
2.
3.  #include <stdio.h>
4.  #define SIZE 100
5.
6.  int linearSearch( const int [], int, int );
7.
8.  int main()
9.  {
10.         int a[ SIZE ], x, searchKey, element;
11.
12.         for ( x = 0; x <= SIZE - 1; x++ )          /* create data */
13.                 a[ x ] = 2 * x;
14.
15.          printf( "Enter integer search key:\n" );
```

```
16.         scanf( "%d", &searchKey );
17.         element = linearSearch( a, searchKey, SIZE );
18.
19.         if ( element != -1 )
20.             printf( "Found value in element %d\n", element );
21.         else
22.             printf( "Value not found\n" );
23.
24.  return 0;
25.  }
26.
27.  int linearSearch( const int array[], int key, int size )
28.  {
29.         int n;
30.
31.         for ( n = 0; n <= size - 1; ++n )
32.         if ( array[ n ] == key )
33.               return n;
34.
35.  return -1;
36.  }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter integer search key:
36
Found value in element 18
```

2ο Τρέξιμο
ΕΞΟΔΟΣ:

```
Enter integer search key:
37
Value not found
```

Παράδειγμα 56 (Δυαδική αναζήτηση (Binary search))

- Ο αλγόριθμος της δυαδικής αναζήτησης εξαλείφει το μισά στοιχεία ενός **ταξινομημένου πίνακα** μετά από κάθε σύγκριση.

- Ο αλγόριθμος εντοπίζει το μεσαίο στοιχείο του πίνακα και το συγκρίνει με το κλειδί αναζήτησης.

- Εάν ισούνται σημαίνει ότι το κλειδί αναζήτησης βρέθηκε και ο δείκτης (η θέση) του εν λόγω στοιχείου του πίνακα επιστρέφεται.

- Εάν δεν ισούνται, το πρόβλημα περιορίζεται στην αναζήτηση μέσα στο μισό του πίνακα.

    - Εάν το κλειδί αναζήτησης είναι μικρότερο από το μεσαίο στοιχείο του πίνακα, το κλειδί αναζητείται στο πρώτο μισό του πίνακα; διαφορετικά το δεύτερο μισό του πίνακα ερευνάται.

    - Εάν το κλειδί αναζήτησης δεν βρέθηκε μέσα στο καθορισμένο υπό πίνακα ο αλγόριθμος επαναλαμβάνεται στο ένα τέταρτο του αρχικού πίνακα.

        (a) Η αναζήτηση συνεχίζεται μέχρι το κλειδί ισούται με το μεσαίο στοιχείο του υπό πίνακα, ή μέχρι ο υπό πίνακας δεν περιέχει κάποιο στοιχείο ίσον με το κλειδί αναζήτησης.

Στο χειρότερο σενάριο, ψάχνοντας ένα πίνακα 1024 στοιχείων θα λάβει μόνο 10 συγκρίσεις χρησιμοποιώντας τη δυαδική αναζήτηση: 1024 – 512 – 256 – 128 – 64 – 32 – 16 – 8 – 4 – 2 - 1 (2^{10}).

```
1.  /*    Binary search of an array  */
2.
3.  #include <stdio.h>
4.  #define SIZE 15
5.
6.  int binarySearch( const int [], int, int, int );
7.  void printHeader( void );
8.  void printRow( const int [], int, int, int );
9.
10. int main()
11. {
12.       int a[ SIZE ], i, key, result;
13.
14.       for ( i = 0; i <= SIZE - 1; i++ )
15.             a[ i ] = 2 * i;
16.
17.       printf( "Enter a number between 0 and 28: " );
```

```
18.       scanf( "%d", &key );
19.
20.       printHeader();
21.       result = binarySearch( a, key, 0, SIZE - 1 );
22.
23.       if ( result != -1 )
24.
25.            printf( "\n%d found in array element %d\n", key, result );
26.       else
27.            printf( "\n%d not found\n", key );
28.
29.  return 0;
30.  }
31.
32.  int binarySearch( const int b[], int searchKey, int low, int high )
33.
34.  {
35.       int middle;
36.
37.       while ( low <= high ) {
38.            middle = ( low + high ) / 2;
39.
40.       printRow( b, low, middle, high );
41.
42.        if ( searchKey == b[ middle ] )
43.             return middle;
44.       else if ( searchKey < b[ middle ] )
45.                high = middle - 1;
46.       else
47.            low = middle + 1;
48.  }
49.
50.       return -1;                    /* searchKey not found */
51.  }
52.
53.  /* Print a header for the output */
54.  void printHeader( void )
55.  {
56.       int i;
57.
58.       printf( "\nSubscripts:\n" );
59.
60.       for ( i = 0; i <= SIZE - 1; i++ )
61.            printf( "%3d ", i );
62.
63.       printf( "\n" );
```

```
64.
65.         for ( i = 1; i <= 4 * SIZE; i++ )
66.             printf( "-" );
67.
68.         printf( "\n" );
69.   }
70.
71.   /* Print one row of output showing the current
72.      part of the array being processed. */
73.   void printRow( const int b[], int low, int mid, int high )
74.   {
75.         int i;
76.
77.         for ( i = 0; i <= SIZE - 1; i++ )
78.               if ( i < low || i > high )
79.                     printf( "    " );
80.               else if ( i == mid )
81.                     printf( "%3d*", b[ i ] );  /* mark middle value */
82.               else
83.                     printf( "%3d ", b[ i ] );
84.
85.          printf( "\n" );
86.   }
```

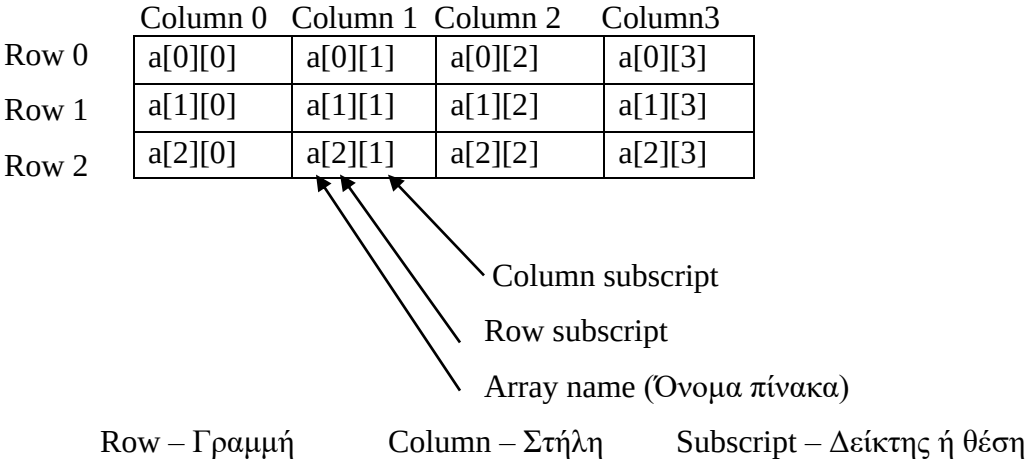
ΕΞΟΔΟΣ:

```
Enter a number between 0 and 28: 25
Subscripts:
0    1   2   3    4   5   6   7    8    9   10   11   12   13   14
---------------------------------------------------------------------------
0    2   4   6    8  10  12  14*  16   18   20   22   24   26   28
                                   16   18   20   22*  24   26   28
                                                     24   26*  28
                                                     24*
25 not found
```

Πολυδιάστατοι πίνακες (Multiple-Subscripted Arrays)

Οι πίνακες στη C μπορούν να έχουν πολλαπλές διαστάσεις (πολλαπλούς δείκτες). Μια κοινή χρήση πολλαπλών-διατάσεων πίνακες είναι η αναπαράσταση **tables** (δυσδιάστατοι πίνακες) με στοιχεία που αποτελούνται από πληροφορίες οργανωμένες σε γραμμές (**rows**) και στήλες (**columns**). Για να προσδιορίσουμε ένα συγκεκριμένο στοιχείο ενός πίνακα θα πρέπει να διευκρινίσουμε δύο διαστάσεις. Η πρώτη διάσταση (από προεπιλογή) προσδιορίζει τη γραμμή του στοιχείου και το δεύτερο (από προεπιλογή) προσδιορίζει τη στήλη του στοιχείου.

Ένα παράδειγμα ενός 3 x 4 πίνακα,



Row – Γραμμή Column – Στήλη Subscript – Δείκτης ή θέση

Παράδειγμα 57

Αυτό το παράδειγμα μας δείχνει πως μπορούμε να δώσουμε αρχικές τιμές σε ένα δυσδιάστατο πίνακα την στιγμή που τον ορίζουμε. Το πρόγραμμα ορίζει **τρεις πίνακες δύο γραμμών και τριών στηλών (έξι στοιχεία το καθένα)**.

```
1.  /* Initializing multidimensional arrays */
2.
3.  #include <stdio.h>
4.
5.  void printArray( const int [][ 3 ] );
6.
7.  int main()
8.  {
9.        int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } },
10.           array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 },
11.           array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };
12.
13.       printf( "Values in array1 by row are:\n" );
14.       printArray( array1 );
```

```
15.
16.         printf( "Values in array2 by row are:\n" );
17.         printArray( array2 );
18.
19.         printf( "Values in array3 by row are:\n" );
20.         printArray( array3 );
21.
22.         return 0;
23.   }
24.
25.   void printArray( const int a[][ 3 ] )
26.   {
27.         int i, j;
28.
29.         for ( i = 0; i <= 1; i++ ) {
30.
31.             for ( j = 0; j <= 2; j++ )
32.                 printf( "%d ", a[ i ][ j ] );
33.
34.             printf( "\n" );
35.         }
36.   }
```

ΕΞΟΔΟΣ:

```
Values in array1 by row are:
1 2 3
4 5 6
Values in array2 by row are:
1 2 3
4 5 0
Values in array3 by row are:
1 2 0
4 0 0
```

Παράδειγμα 58

Αυτό το παράδειγμα εκτελεί αρκετές άλλες κοινές χρήσεις σε ένα 3-επί-4 πίνακα **studentGrades** χρησιμοποιώντας τη δομή **for**. Η κάθε γραμμή του πίνακα αντιπροσωπεύει ένα φοιτητή και κάθε στήλη αντιπροσωπεύει ένα βαθμό ενός από τα τέσσερα διαγωνίσματα ενός μαθήματος που ο φοιτητής παρακάθισε κατά τη διάρκεια ενός εξαμήνου.

```
1.  /*   Double-subscripted array example */
2.  #include <stdio.h>
3.  #define STUDENTS 3
4.  #define EXAMS 4
5.
6.  int minimum( const int [][ EXAMS ], int, int );
7.  int maximum( const int [][ EXAMS ], int, int );
8.  double average( const int [], int );
9.  void printArray( const int [][ EXAMS ], int, int );
10. int main()
11. {
12.         int student;
13.         const int studentGrades[ STUDENTS ][ EXAMS ] =
14.         { { 77, 68, 86, 73 },
15.           { 96, 87, 89, 78 },
16.           { 70, 90, 86, 81 } };
17.         printf( "The array is:\n" );
18.         printArray( studentGrades, STUDENTS, EXAMS );
19.         printf( "\n\nLowest grade: %d\nHighest grade: %d\n",
20.                 minimum( studentGrades, STUDENTS, EXAMS ),
21.                 maximum( studentGrades, STUDENTS, EXAMS ) );
22.
23.         for ( student = 0; student <= STUDENTS - 1; student++ )
24.                 printf( "The average grade for student %d is %.2f\n",
25.                         student,
26.                         average( studentGrades[ student ], EXAMS ) );
27.
28.         return 0;
29. }
30.
31. /* Find the minimum grade */
32. int minimum( const int grades[][ EXAMS ],
33.                     int pupils, int tests )
34. {
35.         int i, j, lowGrade = 100;
36.
37.         for ( i = 0; i <= pupils - 1; i++ )
38.                 for ( j = 0; j <= tests - 1; j++ )
```

```
                if ( grades[ i ][ j ] < lowGrade )
                    lowGrade = grades[ i ][ j ];

    return lowGrade;
}

/* Find the maximum grade */
int maximum( const int grades[][ EXAMS ],
                int pupils, int tests )
{
    int i, j, highGrade = 0;

    for ( i = 0; i <= pupils - 1; i++ )
        for ( j = 0; j <= tests - 1; j++ )
            if ( grades[ i ][ j ] > highGrade )
                highGrade = grades[ i ][ j ];

    return highGrade;
}

/* Determine the average grade for a particular exam */
double average( const int setOfGrades[], int tests )
{
    int i, total = 0;

    for ( i = 0; i <= tests - 1; i++ )
        total += setOfGrades[ i ];

    return ( double ) total / tests;
}

/* Print the array */
void printArray( const int grades[][ EXAMS ],
                    int pupils, int tests )
{
    int i, j;

    printf( "            [0]  [1]  [2]  [3]" );

    for ( i = 0; i <= pupils - 1; i++ ) {
        printf( "\nstudentGrades[%d] ", i );

        for ( j = 0; j <= tests - 1; j++ )
            printf( "%-5d", grades[ i ][ j ] );
    }
}
```

ΕΞΟΔΟΣ:

```
The array is:
                        [0]   [1]     [2]     [3]
studentGrades[0] 77    68      86      73
studentGrades[1] 96    87      89      78
studentGrades[2] 70    90      86      81

Lowest grade:  68
Highest grade:   96
The average grade for student 0 is 76.00
The average grade for student 1 is 87.50
The average grade for student 2 is 81.75
```

ΔΕΙΚΤΕΣ (POINTERS)

- Οι δείκτες είναι ένα από τα πιο ισχυρά χαρακτηριστικά της γλώσσας προγραμματισμού **C**.
- Οι δείκτες είναι μεταξύ των πιο δύσκολων θεμάτων της C για κάποιο να τα διαχειριστεί.
- Οι δείκτες επιτρέπουν στα προγράμματα να προσομοιώνουν κλήσεις διά αναφοράς, να δημιουργούν και να χειρίζονται δυναμικές δομές δεδομένων (dynamic data structures), π.χ. δομές δεδομένων που μπορούν να μεγαλώνουν και να συρρικνωθούν όπως συνδεδεμένες λίστες (linked lists), ουρές (queues), στοίβες (stacks) και δέντρα (trees).

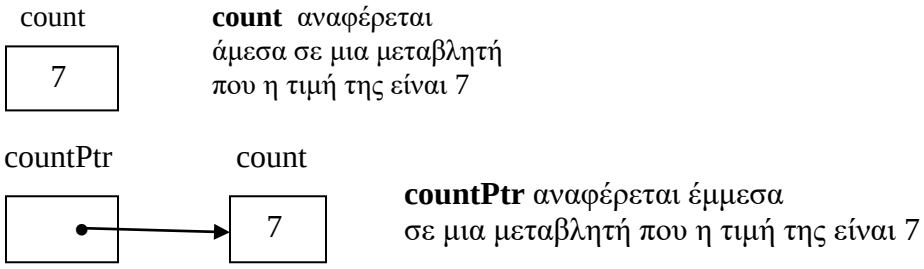
Ορισμός μεταβλητής δείκτη και αρχικοποίηση

- Οι δείκτες είναι μεταβλητές που περιέχουν σαν τιμές διευθύνσεις μνήμης.
- Μια μεταβλητή, ως συνήθως, περιέχει μια συγκεκριμένη τιμή (αναφέρεται άμεσα σε μια τιμή). Ένας δείκτης, από την άλλη, περιέχει μια διεύθυνση μιας μεταβλητής που περιέχει μια συγκεκριμένη τιμή (αναφέρεται έμμεσα σε μια τιμή).
- Ένας δείκτης, όπως οποιαδήποτε άλλη μεταβλητή, πρέπει πρώτα να οριστεί πριν να χρησιμοποιηθεί. Ο ορισμός,

  int *countPtr, count;

  ορίζει τη μεταβλητή **countPtr** να είναι τύπου **int ***. Η μεταβλητή **count** ορίζεται σαν μια ακέραια.

  Όταν ***** χρησιμοποιείται με αυτό τον τρόπο σε μια δήλωση, υποδεικνύει ότι η μεταβλητή που ορίζεται είναι ένας δείκτης. Ένας δείκτης μπορεί να οριστεί να δείχνει σε οποιοδήποτε τύπο δεδομένων.



Οι δείκτες πρέπει να αρχικοποιηθούν είτε όταν ορίζονται ή μέσω μιας εντολής αντιστοίχισης. Ένας δείκτης μπορεί να αρχικοποιηθεί με το **0**, **NULL** ή με μια **διεύθυνση**. NULL και 0 είναι το ίδιο. Όταν δίνεται το **0**, ο δείκτης μετατρέπεται πρώτα σε ένα δείκτη του κατάλληλου τύπου. Η τιμή **0 είναι η μοναδική ακεραία τιμή που μπορεί να δοθεί κατευθείαν σε μια μεταβλητή δείκτη.**

Τελεστές δεικτών

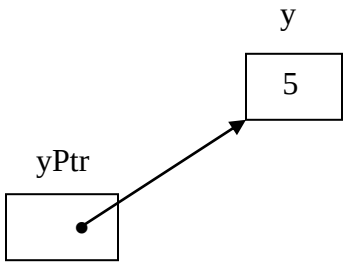
Το **&** ή ο τελεστής διεύθυνσης, είναι μοναδιαίος (unary) τελεστής που επιστρέφει στον τελεστέο (operand) τη διεύθυνση της μεταβλητής. Για παράδειγμα, υποθέτοντας τους ορισμούς

```
int y = 5;
int *yPtr;
```

η εντολή

```
yPtr = &y;
```

αντιστοιχίζει τη διεύθυνση της μεταβλητής **y** στη μεταβλητή δείκτη **yPtr**. **yPtr** σε αυτή την περίπτωση λέμε ότι δείχνει στο **y**.



Η εντολή,

printf("%d", *yPtr);

εκτυπώνει την τιμή της μεταβλητής **y**, η οποία είναι **5**. Η χρήση του ***** ονομάζεται **διακοπή αναφοράς ενός δείκτη (dereferencing a pointer),** Όταν θέλετε να έχετε πρόσβαση στα δεδομένα/τιμή στη μνήμη που ένας δείκτης-points to (δείχνει) αυτό ονομάζεται dereference a pointer..

Το παρακάτω σχήμα δείχνει την εκπροσώπηση του δείκτη στην μνήμη υποθέτοντας ότι η ακέραια μεταβλητή **y** αποθηκεύεται στη διεύθυνση **600000**, και η μεταβλητή δείκτη αποθηκεύεται στη διεύθυνση **500000**.



Παράδειγμα 59

Το ακόλουθο παράδειγμα δεικνύει τους τελεστές ενός δείκτη. Ο μετατροπέας **%p** της εντολής **printf** τυπώνει τη διεύθυνση μνήμης σαν μια δεκαεξαδική ακεραία.

Παρατηρήστε ότι η διεύθυνση της **a** και η τιμή της **aPtr** είναι οι ίδιες στην εκτύπωση. Οι τελεστές **&** και ***** συμπληρώνει το ένα το άλλο – όταν τα δύο εφαρμόζονται διαδοχικά στο **aPtr** με οποιαδήποτε σειρά, εκτυπώνεται το ίδιο αποτέλεσμα.

```
1.  /* Using the & and * operators */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.          int a;          /* a is an integer */
8.          int *aPtr;      /* aPtr is a pointer to an integer */
9.
10.         a = 7;
11.         aPtr = &a;      /* aPtr set to address of a */
12.
13.         printf( "The address of a is %p"
14.         "\nThe value of aPtr is %p", &a, aPtr );
15.
16.         printf( "\n\nThe value of a is %d"
17.         "\nThe value of *aPtr is %d", a, *aPtr );
18.
19.         printf( "\n\nShowing that * and & are inverses of "
20.         "each other.\n&*aPtr = %p"
21.         "\n*&aPtr = %p\n", &*aPtr, *&aPtr );
22.
23.         return 0;
24. }
```

ΕΞΟΔΟΣ:

```
The address of a is 0012FF88
The value of aPtr is 0012FF88

The value of a is 7
The value of *aPtr is 7
Showing that * and & are inverses of each other.
&*aPtr = 0012FF88
*&aPtr = 0012FF88
```

Η προτεραιότητα (precedence) και η προσεταιριστική (associativity) ιδιότητα τελεστών που συζητήθηκε μέχρι στιγμή

Τελεστές	Προσεταιριστική Ιδιότητα	Type
( ) [ ]	left to right	highest
+ - ++ -- ! * & (type)	right to left	unary
* / %	left to right	multiplicative
+ -	left to right	additive
< <= > >=	left to right	relational
== !=	left to right	equality
&&	left to right	logical AND
||	left to right	logical OR
?:	right to left	conditional
= += -= *= /= %=	right to left	assignment
,	left to right	comma

Κλήσεις συναρτήσεων διά αναφοράς

Υπάρχουν δύο τρόποι να περνούμε ορίσματα σε μια συνάρτηση – **διά τιμής (call by value)** και **διά αναφοράς (call by reference)**.

Κλήση διά τιμής

Χρησιμοποιούμε αυτή την μέθοδο όταν

- θέλουμε να χρησιμοποιήσουμε μια συνάρτηση μόνο για να υπολογίζει κάτι, χρησιμοποιώντας τα ορίσματα, και να μας επιστρέφει πίσω το αποτέλεσμα δίχως να αλλάζει τις τιμές των ορισμάτων.
- Επίσης απλά για να διαχωρίσουμε το πρόγραμμα μας σε ενότητες έτσι ώστε το πρόγραμμα μας είναι πιο αναγνώσιμο. Σε αυτή την περίπτωση το **return** χρησιμοποιείται για να επιστρέψει τον έλεγχο του προγράμματος πίσω στην εντολή που ακολουθεί την εντολή κλήσης της συνάρτησης.

Διά αναφοράς

Χρησιμοποιούμε αυτή την μέθοδο όταν

- θέλουμε να αλλάξουμε τις τιμές ορισμάτων μέσα στη συνάρτηση (περνούμε τις διευθύνσεις αυτών των ορισμάτων).
  Αυτό επιτυγχάνεται συνήθως εφαρμόζοντας τον τελεστή διεύθυνσης (**&**) στη μεταβλητή που θέλουμε να αλλάξουμε την τιμή.
  Οι πίνακες δεν περνιούνται χρησιμοποιώντας τον τελεστή **&** για το λόγο ότι η **C** αυτόματα περνά την αρχική θέση του πίνακα στη μνήμη (το όνομα του πίνακα είναι το ίδιο με το **&arrayName[0]**.

Όταν η διεύθυνση της μεταβλητής περνιέται σε μια συνάρτηση, ο έμμεσος τελεστής (*****) μπορεί να χρησιμοποιηθεί στη συνάρτηση για να τροποποιήσει την τιμή σε αυτή τη θέση μνήμης.

- Θέλουμε να περάσουμε μεγάλα αντικείμενα δεδομένων, περνούμε το δείκτη της αρχής αυτών των δεδομένων αντί μια αντιγραφή των, για να αποφύγουμε την επιβάρυνση σε χρόνο και μνήμη.

Παράδειγμα 60

Το ακόλουθο παράδειγμα παρουσιάζει την έκδοση της συνάρτησης που υπολογίζει το κύβο ενός ακεραίου αριθμού – **cubeByValue**.

- Περνά τη μεταβλητή **number** στη συνάρτηση **cubeByValue** χρησιμοποιώντας τη κλήση δια τιμής (ΓΡΑΜΜΗ 12).
- Η συνάρτηση **cubeByValue** υψώνει στην τρίτη δύναμη το όρισμα της και επιστρέφει πίσω τη νέα τιμή στην **main** χρησιμοποιώντας μια **return** εντολή.
- Η νέα τιμή δίνεται στη μεταβλητή **number** μέσα στην **main**.

```
1.  /* Cube a variable using call-by-value */
2.
3.  #include <stdio.h>
4.
5.  int cubeByValue( int );          /* prototype */
6.
7.  int main()
8.  {
9.        int number = 5;
10.
11.       printf( "The original value of number is %d", number );
12.       number = cubeByValue( number );
13.       printf( "\nThe new value of number is %d\n", number );
14.
15.       return 0;
16. }
17.
18. int cubeByValue( int n )
19. {
20.       return n * n * n;                  /* cube local variable n */
21. }
```

ΕΞΟΔΟΣ:

```
The original value of number is 5
The new value of number is 125
```

Παράδειγμα 61

Το ακόλουθο παράδειγμα παρουσιάζει την έκδοση συνάρτησης που υψώνει έναν ακέραιο στη τρίτη δύναμη – **cubeByReference**.

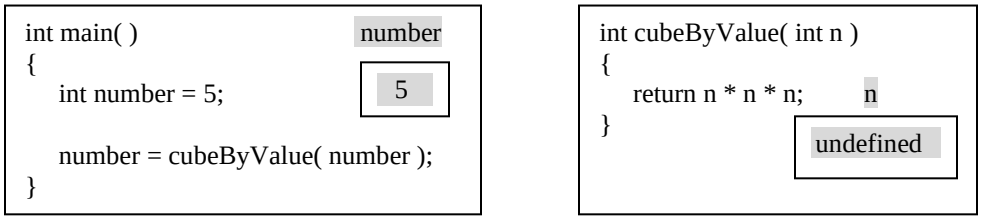
- Περνά τη διεύθυνση της μεταβλητής **number** στη συνάρτηση **cubeByReference** χρησιμοποιώντας κλήση διά αναφοράς (ΓΡΑΜΜΗ 13).
- Η συνάρτηση **cubeByReference** παίρνει ένα δείκτη ακεραίου (**int**) που ονομάζεται **nPtr** σαν ένα όρισμα.
- Η συνάρτηση καταργεί την αναφορά του δείκτη και υψώνει στο κύβο την τιμή που δείχνει ο δείκτης **nPtr**. Αυτό αλλάζει την τιμή της μεταβλητής **number** στην **main**.

```
1.  /* Cube a variable using call-by-reference
2.  with a pointer argument */
3.
4.  #include <stdio.h>
5.
6.  void cubeByReference( int * );         /* prototype */
7.
8.  int main()
9.  {
10.     int number = 5;
11.
12.     printf( "The original value of number is %d", number );
13.     cubeByReference( &number );
14.     printf( "\nThe new value of number is %d\n", number );
15.
16.     return 0;
17. }
18.
19. void cubeByReference( int *nPtr )
20. {
21.     *nPtr = *nPtr * *nPtr * *nPtr;  /* cube number in main */
22. }
```

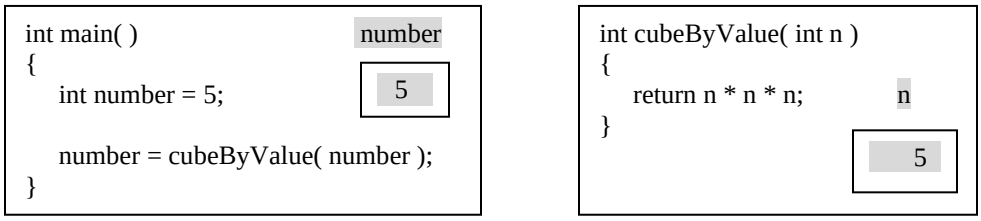
OUTPUT:

```
The original value of number is 5
The new value of number is 125
```

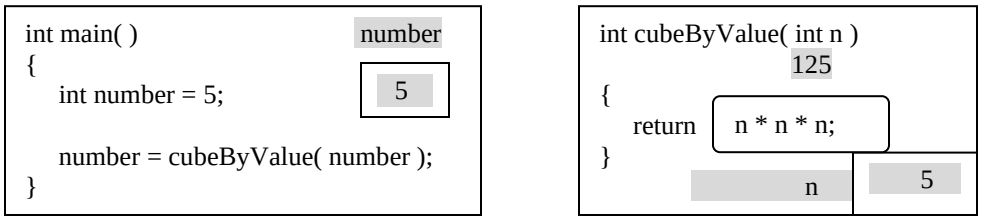
Πριν η **main** καλέσει **cubeByValue**:



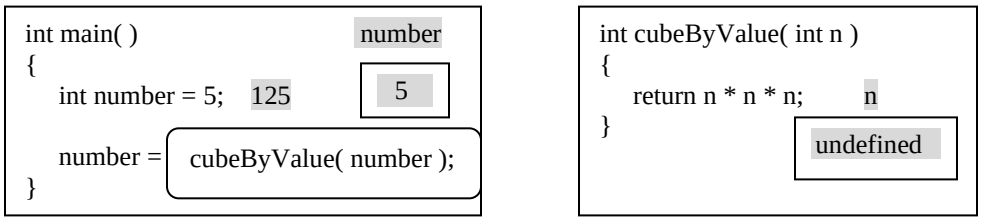
Μετά που η **cubeByValue** παίρνει τη κλήση:



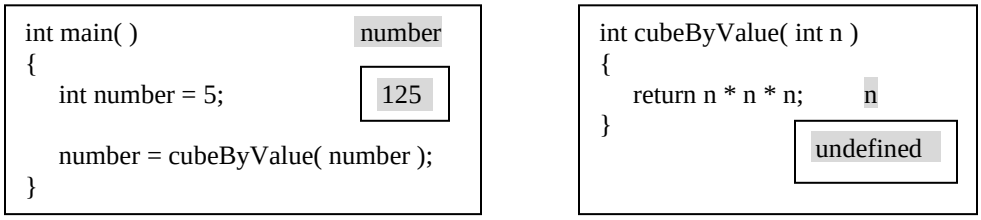
Μετά που η **cubeByValue** υψώνει στο κύβο την παράμετρο **n**:



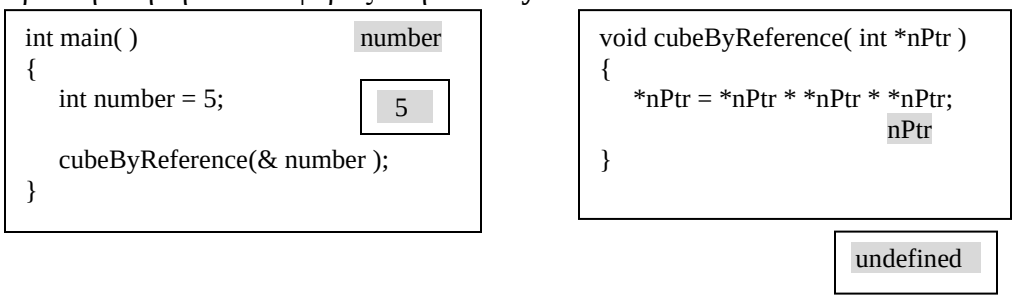
Μετά που η **cubeByValue** επιστρέφει στην main:



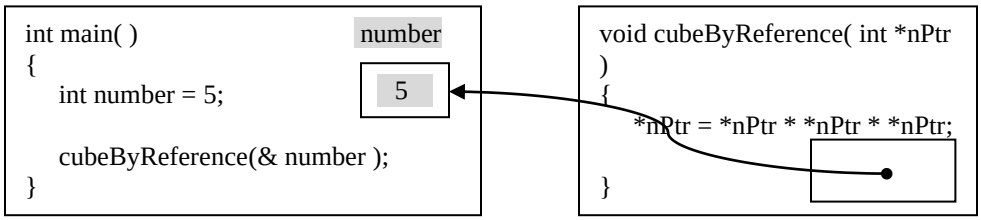
Μετά που η **main** συμπληρώνει την αντιστοίχιση στη μεταβλητή **number**:



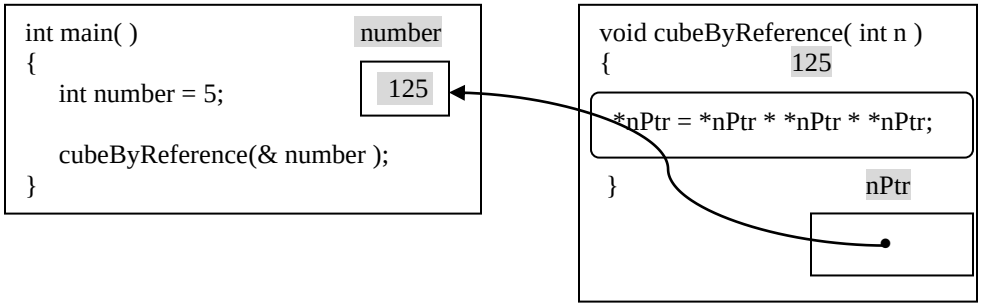
Πριν τη κλήση δια αναφοράς στη **cubeByReference**:



Μετά τη κλήση διά αναφοράς στη **cubeByReference** και πριν η ***nPtr** υψωθεί στο κύβο:



Μετά που η ***nPtr** υψώνεται στο κύβο



<u>Σημείωση:</u> Η συνάρτηση που παίρνει μια διεύθυνση σαν όρισμα πρέπει να ορίσει μια παράμετρο δείκτη για να πάρει τη διεύθυνση. Για παράδειγμα, η κεφαλίδα της συνάρτησης **cubeByReference** είναι

v**oid cubeByReference(int *nptr)**

Η κεφαλίδα καθορίζει ότι η **cubeByReference** παίρνει τη διεύθυνση της ακεραίας μεταβλητής σαν όρισμα, αποθηκεύει τη διεύθυνση τοπικά στη **nPtr**, και δεν επιστρέφει την τιμή.

<u>Η χρήση του προσδιοριστικού const με δείκτες</u>

Το προσδιοριστικό **const** επιτρέπει στον προγραμματιστή να ενημερώσει τον μεταγλωττιστή ότι η τιμή μιας συγκεκριμένης μεταβλητής δεν πρέπει να τροποποιηθεί. Το προσδιοριστικό **const** δεν υπήρχε σε παλιές εκδόσεις της γλώσσας **C**.

Εάν γίνει μια προσπάθεια για να τροποποιηθεί μια τιμή που ορίζεται με το **const**, ο μεταγλωττιστής το αντιλαμβάνεται και ανάλογα δίνει είτε μια προειδοποίηση ή ένα λάθος.
Υπάρχουν τέσσερεις τρόποι να περάσουμε δείκτες σε μια συνάρτηση:

1. Ένας non-constant δείκτης σε non-constant δεδομένα.
2. Ένας constant δείκτης σε non-constant δεδομένα.
3. Ένας non-constant δείκτης σε constant δεδομένα.
4. Ένας constant δείκτης σε constant δεδομένα.

Παράδειγμα 62

Αυτό το παράδειγμα χρησιμοποιεί **a non-constant pointer to non-constant data**.
Η συνάρτηση επεξεργάζεται τον πίνακα **string** (που δείχνεται από το **sPtr**) ένα χαρακτήρα τη κάθε φορά χρησιμοποιώντας αριθμητική δεικτών. Η συνάρτηση **islower** (ΓΡΑΜΜΗ 25) ελέγχει το χαρακτήρα, και η συνάρτηση **toupper** (ΓΡΑΜΜΗ 26) μετατρέπει το χαρακτήρα στο αντίστοιχο κεφαλαίο γράμμα.

```
1.  /*
2.  Converting lowercase letters to uppercase letters
3.  using a non-constant pointer to non-constant data  */
4.
5.  #include <stdio.h>
6.  #include <ctype.h>
7.
8.  void convertToUppercase( char * );
9.
10. int main()
11. {
12.       char string[] = "characters and $32.98";
13.
14.       printf( "The string before conversion is: %s", string );
15.       convertToUppercase( string );
16.       printf( "\nThe string after conversion is: %s\n", string );
17.
18.       return 0;
19. }
20.
21. void convertToUppercase( char *sPtr )
22.  {
23.       while ( *sPtr != '\0' ) {
24.
25.       if ( islower( *sPtr ) )
26.          *sPtr = toupper( *sPtr );              /* convert to uppercase */
27.
28.       ++sPtr;                                   /* move sPtr to the next character */
29.       }
30. }
```

ΕΞΟΔΟΣ:

```
The string before conversion is: characters and $32.98
The string after conversion is: CHARACTERS AND $32.98
```

Παράδειγμα 63

Αυτό το παράδειγμα χρησιμοποιεί **a non-constant pointer to constant data**. Η συνάρτηση **printCharacters** τυπώνει μια γραμματοσειρά ένα χαρακτήρα τη κάθε φορά.

```
1.  /* Printing a string one character at a time using
2.   a non-constant pointer to constant data  */
3.
4.  #include <stdio.h>
5.
6.  void printCharacters( const char * );
7.
8.  int main()
9.  {
10.     char string[] = "print characters of a string";
11.
12.     printf( "The string is:\n" );
13.     printCharacters( string );
14.     printf( "\n" );
15.
16.     return 0;
17. }
18.
19.      /*   In printCharacters, sPtr is a pointer to a character
20.           constant. Characters cannot be modified through sPtr
21.           (i.e., sPtr is a "read-only" pointer).    */
22.
23. void printCharacters( const char *sPtr )
24. {
25.       for ( ; *sPtr != '\0'; sPtr++ )              /* no initialization */
26.           printf( "%c", *sPtr );
27. }
```

ΕΞΟΔΟΣ:

```
The string is:
print characters of a string
```

Παράδειγμα 64

Το ακόλουθο πρόγραμμα παρουσιάζει το μήνυμα λάθους που παράγεται από το μεταγλωττιστή όταν προσπαθεί να μεταγλωττίσει μια συνάρτηση που παίρνει **a non-constant pointer** (**xPtr**) σε **constant data**. Αυτή η συνάρτηση προσπαθεί να τροποποιήσει τα δεδομένα που δείχνονται από το **xPtr** στη ΓΡΑΜΜΗ 21 – αυτό είναι που ο μεταγλώττισης δίνει λάθος.

```
1.  /* Attempting to modify data through a
2.  non-constant pointer to constant data. */
3.
4.  #include <stdio.h>
5.
6.  void f( const int * );
7.
8.  int main()
9.  {
10.       int y;
11.
12.       f( &y );                    /* f attempts illegal modification */
13.
14.       return 0;
15. }
16.
17. /* In f, xPtr is a pointer to an integer constant */
18.
19. void f( const int *xPtr )
20. {
21.       *xPtr = 100;                /* cannot modify a const object */
22. }
```

ΕΞΟΔΟΣ:

```
Example63.c
Error E2024 Example63.c 21: Cannot modify a const object in
Function f
Warning W8057 Example63.c 22: Parameter 'xPtr' is never used
in function f
*** 1 errors in Compile ***
```

Παράδειγμα 65

Αυτό το παράδειγμα προσπαθεί να τροποποιήσει **a constant pointer** σε **non-constant data**. Δημιουργεί ένα μήνυμα λάθους όπως εξηγείτε στο πλαίσιο εξόδου.

```
1.  /*
2.  Attempting to modify a constant pointer to
3.  non-constant data */
4.
5.  #include <stdio.h>
6.
7.  int main()
8.  {
9.      int x, y;
10.
11.     int * const ptr = &x;     /* ptr is a constant pointer to an
12.                                  integer. An integer can be modified
13.                                  through ptr, but ptr always points
14.                                  to the same memory location. */
15.     *ptr = 7;
16.     ptr = &y;
17.
18.     return 0;
19. }
```

ΕΞΟΔΟΣ:

```
Example64.c
Error E2024 Example64.c 16: Cannot modify a const object in
Function main
*** 1 errors in Compile ***
```

Παράδειγμα 66

Το λιγότερο προνόμιο πρόσβασης παρέχεται από το **a constant pointer** σε **a constant data**. Αυτού του είδους ο δείκτης δείχνει πάντα στην ίδια θέση μνήμης, και τα δεδομένα σε αυτή τη θέση δεν μπορούν να τροποποιηθούν. Το ακόλουθο παράδειγμα ορίζει τη μεταβλητή δείκτη **ptr** να είναι τύπου **cons tint * const** (ΓΡΑΜΜΗ 10).

```
1.  /* Attempting to modify a constant pointer to
2.  constant data. */
3.
4.  #include <stdio.h>
5.
6.  int main()
7.  {
8.          int x = 5, y;
9.
10.         const int *const ptr = &x;      /* ptr is a constant pointer to a
11.                                            constant integer. ptr always
12.                                            points to the same location
13.                                            and the integer at that
14.                                            location cannot be modified. */
15.         printf( "%d\n", *ptr );
16.         *ptr = 7;
17.         ptr = &y;
18.
19.         return 0;
20. }
```

ΕΞΟΔΟΣ:

```
Example65.c
Error E2024 Example65.c 16: Cannot modify a const object in
Function main
Error E2024 Example65.c 17: Cannot modify a const object in
Function main
*** 2 errors in Compile ***
```

Παράδειγμα 67

Ταξινόμηση φυσαλίδας (Bubble Sort) με τη χρήση κλήσης δια αναφοράς

Αυτό το παράδειγμα χρησιμοποιεί δύο συναρτήσεις – 1. **bubbleSort** και 2. **swap**.

1. **bubbleSort** εκτελεί την ταξινόμηση του πίνακα και μετά καλεί τη συνάρτηση **swap** για να ανταλλάξει τα στοιχεία του πίνακα **array[j]** και **array[j+1]**.
2. Για το λόγο ότι η **bubbleSort** θέλει τη **swap** να έχει πρόσβαση στα στοιχεία του πίνακα για την ανταλλαγή, η **bubbleSort** περνά κάθε ένα από αυτά τα στοιχεία με κλήση δια αναφοράς στη **swap**.
3. Αν και ολόκληροι οι πίνακες αυτόματα περνιούνται με κλήση δια αναφοράς, τα ατομικά στοιχεία του πίνακα περνιούνται με κλήση διά τιμής.

4. Δεν επιτρέπεται στη **swap** (για το λόγο ότι η **bubbleSort** χρησιμοποιεί τον τελεστή διεύθυνσης (**&**) για το κάθε ένα στοιχείο του πίνακα στη κλήση της swap (ΓΡΑΜΜΗ 39)) να χρησιμοποιήσει

   temp = array[j];
   array[j] = array[j + 1];
   array[j+1] = temp;

   διότι δεν γνωρίζει τα ονόματα **array[j]** και **array[j+1]**, αντί αυτού γνωρίζει τα ονόματα ***element1Ptr** και ***element2Ptr** σαν συνώνυμα, ως εκ τούτου η ανταλλαγή μπορεί να γίνει ως ακολούθως

   temp = *element1Ptr;
   ***element1Ptr = *element2Ptr;**
   ***element2Ptr = temp;**

```
1.  /* This program puts values into an array, sorts the values into
2.  ascending order, and prints the resulting array. */
3.
4.  #include <stdio.h>
5.  #define SIZE 10
6.  void bubbleSort( int *, const int );
7.
8.  int main()
9.  {
10.
11.     int a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
12.     int i;
13.
14.     printf( "Data items in original order\n" );
15.
16.     for ( i = 0; i < SIZE; i++ )
```

```
17.             printf( "%4d", a[ i ] );
18.
19.         bubbleSort( a, SIZE );                  /* sort the array */
20.         printf( "\nData items in ascending order\n" );
21.
22.         for ( i = 0; i < SIZE; i++ )
23.             printf( "%4d", a[ i ] );
24.
25.         printf( "\n" );
26.
27.         return 0;
28.     }
29.
30.     void bubbleSort( int *array, const int size )
31.     {
32.         void swap( int *, int * );
33.         int pass, j;
34.         for ( pass = 0; pass < size - 1; pass++ )
35.
36.             for ( j = 0; j < size - 1; j++ )
37.
38.                 if ( array[ j ] > array[ j + 1 ] )
39.                     swap( &array[ j ], &array[ j + 1 ] );
40.     }
41.
42.     void swap( int *element1Ptr, int *element2Ptr )
43.     {
44.         int hold = *element1Ptr;
45.         *element1Ptr = *element2Ptr;
46.         *element2Ptr = hold;
47.     }
```

ΕΞΟΔΟΣ:

```
Data items in original order
2  6  4  8  10  12  89  68  45  37
Data items in ascending order
2  4  6  8  10  12  37  45  68  89
```

Η κεφαλίδα της συνάρτησης ΓΡΑΜΜΗ 30 ορίζει τον πίνακα σαν **int *array** αντί **int array[]** για δείξει ότι η συνάρτηση **bubbleSort** λαμβάνει ένα μονοδιάστατο πίνακα σαν όρισμα.
Το πρωτότυπο της συνάρτησης **swap** ΓΡΑΜΜΗ 32 συμπεριλαμβάνεται στο σώμα της συνάρτησης **bubbleSort** διότι είναι η μόνη συνάρτηση που καλεί τη **swap**.

Παράδειγμα 68

Μέγεθος σε bytes ενός πίνακα

Η **C** μας παρέχει τον ειδικό μοναδιαίο τελεστή **sizeof** για να προσδιορίζουμε το μέγεθος σε bytes ενός πίνακα (ή οποιουδήποτε τύπου δεδομένων) κατά τη μεταγλώττιση του προγράμματος.

```
1.  /*   Sizeof operator when used on an array name
2.       returns the number of bytes in the array. */
3.
4.  #include <stdio.h>
5.
6.  size_t getSize( float * );
7.
8.  int main()
9.  {
10.      float array[ 20 ];
11.
12.      printf( "The number of bytes in the array is %d"
13.              "\nThe number of bytes returned by getSize is %d\n",
14.              sizeof( array ), getSize( array ) );
15.
16.      return 0;
17. }
18.
19. size_t getSize( float *ptr )
20. {
21.      return sizeof( ptr );
22. }
```

ΕΞΟΔΟΣ:

```
The number of bytes in the array is 80
The number of bytes returned by getsize is 4
```

Σημείωση: Η συνάρτηση **getSize** επιστρέφει τον τύπο **size_t**. Ο τύπος **size_t** είναι ένας τύπος που ορίζεται από τη πρότυπο **C** ως αναπόσπαστο τύπο (**unsigned** ή **unsigned long**)της τιμής από τον τελεστή **sizeof**.

Παράδειγμα 69

Αυτό το παράδειγμα εκτυπώνει τον αριθμό των bytes που χρησιμοποιούνται για αποθήκευση όλων των πρότυπων τύπων δεδομένων.

```
1.  /*   Demonstrating the sizeof operator */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.          char c;
8.          short s;
9.          int i;
10.         long l;
11.         float f;
12.         double d;
13.          long double ld;
14.          int array[ 20 ], *ptr = array;
15.
16.          printf( "     sizeof c = %d"
17.          "\tsizeof(char)  = %d"
18.          "\n     sizeof s = %d"
19.          "\tsizeof(short) = %d"
20.          "\n     sizeof i = %d"
21.          "\tsizeof(int) = %d"
22.          "\n     sizeof l = %d"
23.          "\tsizeof(long) = %d"
24.          "\n     sizeof f = %d"
25.          "\tsizeof(float) = %d"
26.          "\n     sizeof d = %d"
27.          "\tsizeof(double) = %d"
28.          "\n    sizeof ld = %d"
29.          "\tsizeof(long double) = %d"
30.          "\n sizeof array = %d"
31.          "\n   sizeof ptr = %d\n",
32.          sizeof c, sizeof( char ), sizeof s,
33.          sizeof( short ), sizeof i, sizeof( int ),
34.          sizeof l, sizeof( long ), sizeof f,
35.          sizeof( float ), sizeof d, sizeof( double ),
36.          sizeof ld, sizeof( long double ),
37.          sizeof array, sizeof ptr );
38.
39.          return 0;
40. }
```

ΕΞΟΔΟΣ:

```
    sizeof  c = 1                   sizeof(char)  = 1
    sizeof  s = 2                   sizeof(short)  = 2
    sizeof  i = 4                   sizeof(int)  = 4
    sizeof  l = 4                   sizeof(long)  = 4
    sizeof  f = 4                   sizeof(float)  = 4
    sizeof  d = 8                   sizeof(double)  = 8
    sizeof ld = 8                   sizeof(long double)  = 8
sizeof array = 80
   sizeof ptr = 4
```

Παραστάσεις δεικτών και αριθμητικά δεικτών

A. Οι δείκτες είναι έγκυροι τελεσταίοι (operands) ΟΧΙ ΓΙΑ ΟΛΕΣ τις αριθμητικές παραστάσεις, παραστάσεις αντιστοίχησης και παραστάσεις σύγκρισης, αλλά σε ΟΡΙΣΜΕΝΕΣ.

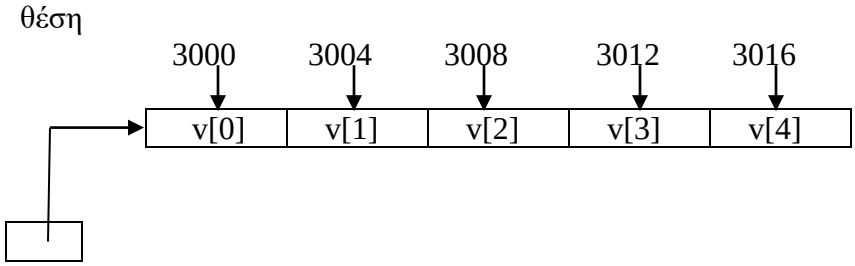
1. Ένας δείκτης μπορεί να αυξάνεται (++).
2. Ένας δείκτης μπορεί να μειώνεται (--).
3. Ένας ακέραιος μπορεί να προστεθεί σε ένα δείκτη (+ or +=).
4. Ένας ακέραιος μπορεί να αφαιρεθεί από δείκτη (- or -=).
5. Ένας δείκτης μπορεί να αφαιρεθεί από άλλο δείκτη.

Παράδειγμα:

Εάν το **int v[10]** είναι ένας πίνακας και η πρώτη του στη θέση στη μνήμη είναι **3000** και εάν ο δείκτης **vPtr** αρχικοποιήθηκε να δείχνει στο **v[0]** τότε ο **vPtr** δείχνει και αυτός στη διεύθυνση **3000**.

vPtr = v;
vPtr = &v[0];

αυτά είναι τα ίδια.



Η μεταβλητή δείκτη **vPtr**

Όταν ένας ακέραιος προσθέτεται ή αφαιρείται από ένα δείκτη απλά δεν προσθέτεται ή αφαιρείται ο ακέραιος, αλλά ο ακέραιος πολλαπλασιάζεται επί το μέγεθος του τύπου του αντικειμένου που δείχνεται από το δείκτη και μετά γίνεται η πρόσθεση ή η αφαίρεση.

Για παράδειγμα,

vPtr += 2;

θα μας δώσει το αποτέλεσμα **3008 (3000 + 4 * 2)** που στο προηγούμενο μας παράδειγμα θα δείχνει στο **v[2]**.

Εάν ένας δείκτης πρέπει να αυξηθεί ή μειωθεί κατά 1, η αύξηση (++) ή η μείωση (--) μπορούμε να χρησιμοποιήσουμε μία από τις δύο ακόλουθες εντολές.

++vPtr;

ή

vPtr++;

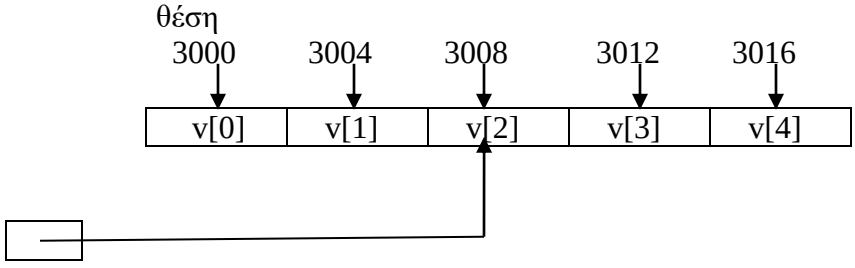
αυξάνει το δείκτη να δείχνει στο επόμενο στοιχείο του πίνακα.

--vPtr;

ή

vPtr--;

μειώνει το δείκτη να δείχνει στο προηγούμενο στοιχείο του πίνακα.



Μεταβλητή δείκτη **vPtr**

Οι μεταβλητές δεικτών μπορούν να αφαιρεθούν η μία από την άλλη.
Για παράδειγμα,

Εάν ο **vPtr** περιέχει τη θέση μνήμης **3000**, και ο **v2Ptr** περιέχει τη διεύθυνση **3008**, τοτε η εντολή

x = v2Ptr – vPtr

θα δώσει στο **x** τον αριθμό στοιχείων του πίνακα από τον **vPtr** μέχρι τον **v2Ptr**, σε αυτή την περίπτωση **2**.

ΣΗΜΕΙΩΣΗ: Η αριθμητική δεικτών είναι χωρίς νόημα, εκτός και να εκτελείται σε ένα πίνακα.

A. Ένας δείκτης μπορεί να αντιστοιχηθεί σε ένα άλλο εάν και οι δύο δείκτες είναι του ιδίου τύπου.

   Διαφορετικά πρέπει να χρησιμοποιήσουμε τον τελεστή CAST για να μετατρέψουμε το δείκτη που βρίσκεται δεξιά της αντιστοίχησης στο τύπο του δείκτη που βρίσκεται αριστερά της αντιστοίχισης.

   Η εξαίρεση αυτού του κανόνα είναι ο δείκτης **void** (π.χ. void *) ο οποίος είναι γενικός δείκτης που αντιπροσωπεύει οποιοδήποτε τύπο δείκτη. Όλοι οι τύποι δεικτών μπορούν να αντιστοιχηθούν σε δείκτη **void**, και ο δείκτης **void** μπορεί να αντιστοιχηθεί σε οποιοδήποτε τύπο δείκτη. Και στις δύο περιπτώσεις δεν απαιτείται ο τελεστής CAST.

   Σημείωση: Ένας δείκτης σε **void** δεν μπορεί να είναι dereferenced.

B. Οι δείκτες μπορούν να συγκριθούν χρησιμοποιώντας τους τελεστές ΙΣΟΤΗΤΑΣ (EQUALITY) και ΣΧΕΣΙΑΚΟΥΣ (RELATIONAL) τελεστές, αλλά οι συγκρίσεις δεν έχουν νόημα εκτός εάν οι δείκτες δείχνουν σε στοιχεία του ιδίου πίνακα. Οι συγκρίσεις δεικτών συγκρίνουν τις διευθύνσεις που είναι αποθηκευμένοι οι δείκτες.

   Μια κοινή χρήση σύγκρισης δεικτών είναι εάν η τιμή ενός δείκτης είναι **NULL**.

C. Οι πίνακες και οι δείκτες συνδέονται στενά στη **C** και μπορούν να χρησιμοποιηθούν σχεδόν εναλλακτικά. Ένα όνομα ενός πίνακα μπορεί να θεωρηθεί σαν ένας constant δείκτης. Οι δείκτες μπορούν να χρησιμοποιηθούν για να κάνουν κάθε πράξη που κάνει ο πίνακας με τα στοιχεία του με τη χρήση του subscript.

   Παράδειγμα,

   Θεωρείστε ότι ορίσαμε ένα πίνακα ακεραίων **b[5]** και μια μεταβλητή δείκτη σε ακέραιο **bPtr**. Για το λόγο ότι ένα όνομα πίνακα χωρίς subscript είναι ένας δείκτης που δείχνει στο πρώτο στοιχείο του πίνακα, μπορούμε να θέσουμε το **bPtr** ίσον με τη διεύθυνση του πρώτου στοιχείου του πίνακα **b** με την εντολή

bPtr = b;

ή με την εντολή

bPtr = &b[0];

Το στοιχείο του πίνακα **b[3]** εναλλακτικά μπορεί να αναφερθεί με την παράσταση δείκτη
***(bPtr + 3)** όπου **3** είναι η *μετατόπιση (offset)* από το δείκτη που είναι παρόμοιο με το subscript του πίνακα. Αυτή η σημειογραφία είναι γνωστή σαν *δείκτης/μετατόπιση* (*pointer/offset*).
Οι παρενθέσεις είναι αναγκαίες διότι η προτεραιότητα του ***** είναι ψηλότερη από την προτεραιότητα του **+**. Χωρίς τις παρενθέσεις η τιμή **3** θα προσθετόταν στην τιμή του ***bPtr**.

Η διεύθυνση

&b[3]

μπορεί να γραφτεί με την παράσταση

bPtr + 3

Οι δείκτες μπορούν να είναι subscripted ακριβώς όπως τους πίνακες.

Για παράδειγμα,

bPtr[1]

αναφέρεται στο στοιχείο του πίνακα

b[1] (δείκτης/subscription (pointer/subscription) σημειογραφία).

Σημείωση: Να θυμάστε ότι το όνομα ενός πίνακα είναι ουσιαστικά ένας σταθερός (constant) δείκτης – Πάντα δείχνει στην αρχή του πίνακα, ως εκ τούτου, η παράσταση

b += 3

δεν είναι έγκυρη διότι προσπαθεί να τροποποιήσει τη τιμή του ονόματος του πίνακα με αριθμητικό δείκτη.

Παράδειγμα 70

Το ακόλουθο παράδειγμα χρησιμοποιεί τις τέσσερις μεθόδους που έχουμε συζητήσει για την αναφορά των στοιχείων ενός πίνακα – **με το subscript του πίνακα**, **δείκτης/μετατόπιση με το όνομα του πίνακα σαν δείκτη**, **με το subscript του δείκτη**, και **δείκτης/μετατόπιση με ένα δείκτη** – για να εκτυπώσουμε τα στοιχεία ενός πίνακα ακεραίων.

```
1.  /* Example to use four methods for referring array elements.
2.  Using subscripting and pointer notations with arrays */
3.
4.  #include <stdio.h>
5.
6.  int main()
7.  {
8.      int b[] = { 10, 20, 30, 40 };
9.      int *bPtr = b;              /* set bPtr to point to array b */
10.     int i, offset;
11.     printf( "Array b printed with:\n"
12.             "Array subscript notation\n" );
13.
14.     for ( i = 0; i < 4; i++ )
15.         printf( "b[ %d ] = %d\n", i, b[ i ] );
16.
17.
18.     printf( "\nPointer/offset notation where\n"
19.             "the pointer is the array name\n" );
20.
21.     for ( offset = 0; offset < 4; offset++ )
22.         printf( "*( b + %d ) = %d\n", offset, *( b + offset ) );
23.
24.
25.     printf( "\nPointer subscript notation\n" );
26.
27.     for ( i = 0; i < 4; i++ )
28.         printf( " bPtr[ %d ] = %d\n", i, bPtr[ i ] );
29.
30.     printf( "\nPointer/offset notation\n" );
31.
32.     for ( offset = 0; offset < 4; offset++ )
33.         printf( "*( bPtr + %d ) = %d\n", offset, *( bPtr + offset ) );
34.
35.     return 0;
36. }
```

ΕΞΟΔΟΣ:

```
Array b printed with:
Array subscript notation
b[0] = 10
b[1] = 20
b[2] = 30
b[3] = 40

Pointer/offset notation where
the pointer is the array name
*(b + 0) = 10
*(b + 1) = 20
*(b + 2) = 30
*(b + 3) = 40

Pointer subscript notation
bPtr[0] = 10
bPtr [1] = 20
bPtr [2] = 30
bPtr [3] = 40

Pointer/offset notation
*(bPtr + 0) = 10
*(bPtr + 1) = 20
*(bPtr + 2) = 30
*(bPtr + 3)= 40
```

Παράδειγμα 71

Αυτό το παράδειγμα χρησιμοποιεί δύο συναρτήσεις που αντιγράφουν γραμματοσειρές – **copy1** και **copy2**.

```
1.  /* Copying a string using array notation
2.  and pointer notation. */
3.  #include <stdio.h>
4.  void copy1( char *, const char * );
5.  void copy2( char *, const char * );
6.
7.  int main()
8.  {
9.        char string1[ 10 ], *string2 = "Hello",
10.       string3[ 10 ], string4[] = "Good Bye";
11.
12.       copy1( string1, string2 );
13.       printf( "string1 = %s\n", string1 );
14.
15.       copy2( string3, string4 );
16.       printf( "string3 = %s\n", string3 );
17.
18.       return 0;
19. }
20.
21. /* copy s2 to s1 using array notation */
22. void copy1( char *s1, const char *s2 )
23. {
24.       int i;
25.       for ( i = 0; ( s1[ i ] = s2[ i ] ) != '\0'; i++ )
26.          ;            /* do nothing in body */
27. }
28.
29. /* copy s2 to s1 using pointer notation */
30. void copy2( char *s1, const char *s2 )
31. {
32.       for ( ; ( *s1 = *s2 ) != '\0'; s1++, s2++ )
33.          ;            /* do nothing in body */
34. }
```

ΕΞΟΔΟΣ:

```
String1 = Hello
String3 = Good Bye
```

Πίνακες δεικτών

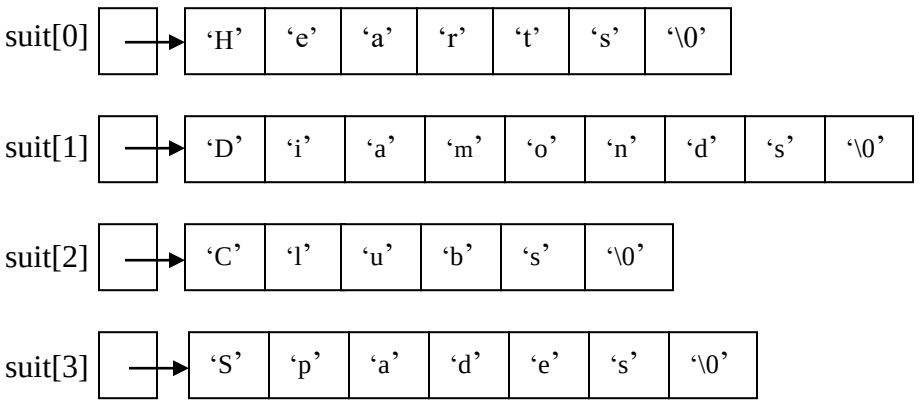
Οι πίνακες μπορούν να περιέχουν δείκτες σαν στοιχεία. Μια κοινή χρήση τέτοιας δομής δεδομένων είναι για να μορφοποιήσουμε ένα πίνακα γραμματοσειρών.

Κάθε εγγραφή του πίνακα είναι μια γραμματοσειρά, αλλά στη **C** μια γραμματοσειρά είναι ουσιαστικά ένας δείκτης στον πρώτο της χαρακτήρα.

Παράδειγμα,

```
const char *suit[4] = { "Hearts", "Diamonds",
                        "Clubs", "Spades" };
```

- Το **suit[4]** μέρος του ορισμού μας δεικνύει ένα πίνακα με **4** στοιχεία.
- Το **char *** μέρος του ορισμού μας δεικνύει ότι κάθε στοιχείο του πίνακα **suit** είναι ένας δείκτης τύπου **char**.
- Το προσδιοριστικό **const** μας δεικνύει ότι οι γραμματοσειρές που δείχνονται από το κάθε στοιχείο του πίνακα δεν μπορούν να τροποποιηθούν.
- Οι τέσσερις τιμές που δίνονται στον πίνακα είναι **"Hearts", "Diamonds", "Clubs", "Spades"**.
- Το κάθε ένα από αυτά αποθηκεύεται στη μνήμη σαν μια γραμματοσειρά με το null να τοποθετείται στο τέλος της και το μέγεθος της να είναι κατά ένα μεγαλύτερο από τον αριθμό των χαρακτήρων που είναι ανάμεσα στα εισαγωγικά.
- Φαίνεται ότι αυτές οι τέσσερις γραμματοσειρές τοποθετούνται στον πίνακα, αλλά ουσιαστικά μόνο τέσσερις δείκτες αποθηκεύονται σε αυτόν. (κοίτα σχήμα-1)
- Ο κάθε δείκτης δείχνει στον πρώτο χαρακτήρα της αντίστοιχης γραμματοσειράς και ακόμα κι αν ο πίνακας **suit** έχει σταθερό μέγεθος, μας παρέχει πρόσβαση σε γραμματοσειρές οποιουδήποτε μεγέθους.
- Οι φυλές της τράπουλας μπορούσαν να τοποθετηθούν σε δύο-διαστάσεων πίνακα στον οποίο η κάθε σειρά θα αντιπροσώπευε μια φυλή, και κάθε στήλη θα αντιπροσώπευε ένα γράμμα του ονόματος της φυλής. Αυτό το είδος δομής δεδομένων θα είχαν ένα σταθερό αριθμών στηλών ανά σειρά, που το μεγαλύτερο που θα μπορούσε να ήταν είναι το μέγεθος της πιο μεγάλης γραμματοσειράς.
- Αυτό θα μας οδηγούσε στη σπατάλη μνήμης.



σχήμα-1

Προσομοίωση ανακατέματος και μοιράσματος της τράπουλας

Αυτή η ενότητα χρησιμοποιεί δημιουργία τυχαίων αριθμών για να αναπτύξει ένα πρόγραμμα προσομοίωσης ανακατέματος και μοιράσματος της τράπουλας.

Χρησιμοποιώντας τη πάνω-κάτω (top-down), σταδιακή-βελτίωση (stepwise-refinement) προσέγγιση, έχουμε αναπτύξει ένα πρόγραμμα που ανακατεύει μια τράπουλα με 52 χαρτιά παιχνιδιού και μετά μοιράζει. Η top-down μέθοδος είναι ιδιαίτερα χρήσιμη για την επίλυση μεγαλυτέρων και πολύπλοκων προβλημάτων.

Χρησιμοποιούμε ένα 4-επί-13 δύο – διαστάσεων πίνακα για να αντιπροσωπεύσουμε την τράπουλα των χαρτιών παιχνιδιού.

		Ace	Deuce	Three	Four	Five	Six	Seven	Eight	Nine	Ten	Jack	Queen	King
		0	1	2	3	4	5	6	7	8	9	10	11	12
Hearts	0													
Diamonds	1													
Clubs	2													
Spades	3													

Αυτή η προσομοιωμένη τράπουλα μπορεί να ανακατευτεί ως ακολούθως

- Πρώτα η τράπουλα **deck** αρχικοποιείται με μηδενικά.
- Μετά, η σειρά (0-3) και η στήλη (0-12) επιλέγονται τυχαία.
- Ο αριθμός 1 δίνεται στο στοιχείο του πίνακα **deck** deck[row] [column] για να δείξουμε ότι αυτό το χαρτί είναι το πρώτο που θα μοιραστεί από την ανακατεμένη τράπουλα.
- Αυτή η διαδικασία συνεχίζετε με τους αριθμούς 2, 3, …, 52 να μπαίνουν στον πίνακα **deck** τυχαία για να δείξουμε πιο χαρτί θα τοποθετηθεί δεύτερο, τρίτο, …, 52^{ο} στην ανακατεμένη τράπουλα.
- Όταν ο πίνακας **deck** ξεκινά να γεμίζει με αριθμούς χαρτιών, είναι δυνατό ένα χαρτί να επιλεγεί δύο φορές (π.χ., deck [row] [column] θα έχει μια τιμή διαφορετική από **μηδέν** όταν θα επιλεγεί). Αυτή η επιλογή απλά αγνοείται, και άλλες σειρές και στήλες διαλέγονται τυχαία μέχρι να βρεθεί ένα χαρτί που δεν επιλέγηκε.
- Τελικά, οι αριθμοί από το 1 μέχρι 52 θα καταλαμβάνουν τις 52 θέσεις του πίνακα **deck**.
- Αυτή τη στιγμή η τράπουλα είναι ολοκληρωτικά ανακατεμένη.

- Για να δώσουμε το πρώτο χαρτί, ψάχνουμε τον πίνακα για το deck[row] [column] που είναι ίσον με 1. Αυτό επιτυγχάνεται με τον ένθετο βρόχο *for* ο οποίος δίνει στις σειρές τις τιμές από το 0 μέχρι το 3 και στις στήλες από το 0 μέχρι το 12.

- Τυπώνουμε τη γραμματοσειρά suit[row] για να πάρουμε τη φυλή.
- Το ίδιο, τυπώνουμε τη γραμματοσειρά face[column] για να πάρουμε το είδος του χαρτιού.

Παράδειγμα 72

```
1.  /* Example
2.     Card shuffling dealing program */
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.  #include <time.h>
6.
7.  void shuffle( int [][ 13 ] );
8.  void deal( const int [][ 13 ], const char *[], const char *[] );
9.
10. int main()
11. {
12.       const char *suit[ 4 ] =
13.           { "Hearts", "Diamonds", "Clubs", "Spades" };
14.       const char *face[ 13 ] =
15.           { "Ace", "Deuce", "Three", "Four",
16.             "Five", "Six", "Seven", "Eight",
17.             "Nine", "Ten", "Jack", "Queen", "King" };
18.       int deck[ 4 ][ 13 ] = { 0 };
19.
20.       srand( time( 0 ) );
21.
22.       shuffle( deck );
23.       deal( deck, face, suit );
24.
25.       return 0;
26. }
27.
28. void shuffle( int wDeck[][ 13 ] )
29. {
30.       int row, column, card;
31.
32.       for ( card = 1; card <= 52; card++ ) {
33.           do {
34.                   row = rand() % 4;
35.                   column = rand() % 13;
36.               } while( wDeck[ row ][ column ] != 0 );
37.
38.               wDeck[ row ][ column ] = card;
39.       }
40. }
41.
42. void deal( const int wDeck[][ 13 ], const char *wFace[],
43.             const char *wSuit[] )
44. {
```

```
45.         int card, row, column;
46.
47.         for ( card = 1; card <= 52; card++ )
48.
49.             for ( row = 0; row <= 3; row++ )
50.
51.                 for ( column = 0; column <= 12; column++ )
52.
53.                     if ( wDeck[ row ][ column ] == card )
54.                         printf( "%5s of %-8s%c",
55.                         wFace[ column ], wSuit[ row ],
56.                         card % 2 == 0 ? '\n' : '\t' );
57.  }
```

ΕΞΟΔΟΣ

```
   Six  of  Clubs          Seven  of  Diamonds
   Ace  of  Spades           Ace  of  Diamonds
   Ace  of  Hearts         Queen of  Diamonds
 Queen  of  Clubs          Seven of  Hearts
   Ten of  Hearts          Deuce of  Clubs
   Ten of  Spades          Three of  Spades
   Ten of   Diamonds         Four of  Spades
  Four  of  Diamonds          Ten of  Clubs
   Six  of  Diamonds          Six of  Spades
 Eight  of  Hearts          Three of  Diamonds
  Nine  of   Hearts         Three of  Hearts
 Deuce  of  Spades            Six of  Hearts
  Five  of  Clubs           Eight of  Clubs
 Deuce  of  Diamonds        Eight of  Spades
  Five  of  Clubs            King of  Clubs
  King of  Diamonds           Jack of  Spades
 Deuce  of  Hearts         Queen of   Hearts
   Ace  of  Clubs           King  of  Spades
 Three   of Clubs           King  of  Hearts
  Nine   of  Clubs           Nine  of  Spades
  Four   of  Hearts         Queen  of  Spades
 Eight   of  Diamonds         Nine  of  Diamonds
  Jack   of  Diamonds        Seven  of  Clubs
  Five   of  Hearts           Five   of  Diamonds
  Four    of  Clubs           Jack   of  Hearts
  Jack   of  Clubs          Seven   of  Spades
```

Δείκτες σε συναρτήσεις

Ένας δείκτης σε συνάρτηση περιέχει τη διεύθυνση της συνάρτησης στη μνήμη. Όπως έχουμε δει, το όνομα ενός πίνακα είναι στην πραγματικότητα η διεύθυνση στη μνήμη του πρώτου στοιχείου του πίνακα. Το ίδιο, το όνομα μιας συνάρτησης είναι στην πραγματικότητα η αρχική διεύθυνση του κώδικα της συνάρτησης στη μνήμη που εκτελεί το έργο της συνάρτησης. Δείκτες σε συναρτήσεις μπορούν να είναι

a. περασμένοι στις συναρτήσεις,
b. επιστρεφόμενοι από τις συναρτήσεις,
c. αποθηκευμένοι σε ένα πίνακα και
d. αντιστοιχισμένοι σε άλλους δείκτες συναρτήσεων.

Η μέθοδος για **ταξινόμηση Selection (με επιλογή) με τη χρήση δεικτών σε συναρτήσεις**

Το ακόλουθο παράδειγμα δείχνει τη χρήση των δεικτών που δείχνουν σε συναρτήσεις.

Παράδειγμα 73

```
1.  /* Example
2.  Multipurpose sorting program using function pointers */
3.  #include <stdio.h>
4.  #define SIZE 10
5.  void bubble( int [], const int, int (*)( int, int ) );
6.  int ascending( int, int );
7.  int descending( int, int );
8.
9.  int main()
10. {
11.
12.     int order,
13.         counter,
14.             a[ SIZE ] = { 2, 6, 4, 8, 10, 12, 89, 68, 45, 37 };
15.
16.     printf( "Enter 1 to sort in ascending order,\n"
17.     "Enter 2 to sort in descending order: " );
18.     scanf( "%d",  &order );
19.     printf( "\nData items in original order\n" );
20.
21.     for ( counter = 0; counter < SIZE; counter++ )
22.         printf( "%5d", a[ counter ] );
23.
24.     if ( order == 1 ) {
25.         bubble( a, SIZE, ascending );
26.         printf( "\nData items in ascending order\n" );
27.     }
```

```
28.     else {
29.             bubble( a, SIZE, descending );
30.             printf( "\nData items in descending order\n" );
31.         }
32.
33.     for ( counter = 0; counter < SIZE; counter++ )
34.         printf( "%5d", a[ counter ] );
35.
36.     printf( "\n" );
37.
38.     return 0;
39. }
40.
41. void bubble( int work[], const int size,
42.                             int (*compare)( int, int ) )
43. {
44.     int pass, count;
45.
46.     void swap( int *, int * );
47.
48.     for ( pass = 1; pass < size; pass++ )
49.
50.         for ( count = 0; count < size - 1; count++ )
51.
52.             if ( (*compare)( work[ count ], work[ count + 1 ] ) )
53.                 swap( &work[ count ], &work[ count + 1 ] );
54. }
55.
56. void swap( int *element1Ptr, int *element2Ptr )
57. {
58.     int temp;
59.
60.     temp = *element1Ptr;
61.     *element1Ptr = *element2Ptr;
62.     *element2Ptr = temp;
63. }
64.
65. int ascending( int a, int b )
66. {
67.     return b < a;       /* swap if b is less than a */
68. }
69.
70. int descending( int a, int b )
71. {
72.     return b > a;       /* swap if b is greater than a */
73. }
```

1ο Τρέξιμο
ΕΞΟΔΟΣ

```
Enter 1 to sort in ascending order,
Enter 2 to sort in descending order: 1

Data items in original order
    2   6    4   8   10  12  89  68  45  37
Data items in ascending order
    2   4    6   8   10  12  37  45  68  89
```

2ο Τρέξιμο
ΕΞΟΔΟΣ

```
Enter 1 to sort in ascending order,
Enter 2 to sort in descending order: 2

Data items in original order
    2   6    4   8   10  12  89  68  45  37
Data items in ascending order
    89  68  45  37  12  10  8    6    4    2
```

Περισσότερα Παραδείγματα (Δείκτες)

Παράδειγμα 74

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει δύο ακέραιους, να δηλώνει δύο δείκτες προς αυτούς και να εμφανίζει τις διευθύνσεις των ακεραίων μεταβλητών, τα περιεχόμενα των δεικτών και τις διευθύνσεις μνήμης των δεικτών.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int *ptr1, *ptr2;
5.      int i, j;
6.
7.      printf("Enter numbers: ");
8.      scanf("%d%d", &i, &j);
9.
10.     ptr1 = &i;
11.     ptr2 = &j;
12.
13.     printf("Num1 address = %p\n", ptr1);
14.     printf("Num2 address = %p\n", ptr2);
15.
16.     printf("Ptr1 content = %d\n", *ptr1);
17.     printf("Ptr2 content = %d\n", *ptr2);
18.
19.     printf("Ptr1 address = %p\n", &ptr1);
20.     printf("Ptr2 address = %p\n", &ptr2);
21.     return 0;
22. }
```

Παράδειγμα 75

Να γραφεί ένα πρόγραμμα το οποίο, με χρήση δείκτη, να διαβάζει μία πραγματική τιμή και να εμφανίζει την απόλυτη τιμή της.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      double *p;
5.      double val;
6.
7.      p = &val;
8.      printf("Enter number: ");
9.      scanf("%lf", p);
10.
11.     if(*p >= 0)
12.         printf("%f\n", *p);
13.     else
14.         printf("%f\n", -*p);
15.     return 0;
16. }
```

Παράδειγμα 76

Να γραφεί ένα πρόγραμμα το οποίο με χρήση δεικτών να διαβάζει τις τιμές δύο προϊόντων και να εμφανίζει τη μεγαλύτερη τιμή.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      float *ptr1, *ptr2;
5.      float i, j;
6.
7.      ptr1 = &i;
8.      ptr2 = &j;
9.  /* Πριν χρησιμοποιηθούν οι δείκτες πρέπει να δείχνουν στις διευθύνσεις
10.                     των αντιστοίχων μεταβλητών */
11.     printf("Enter prices: ");
12.     scanf("%f%f", ptr1, ptr2); /* Αποθήκευση των τιμών στις διευθύνσεις
13.                                   μνήμης που δείχνουν οι δείκτες */
14.     if(*ptr1 > *ptr2)
15.         printf("%f\n", *ptr1);
16.     else
17.         printf("%f\n", *ptr2);
18.     return 0;
19. }
```

Παράδειγμα 77

Να γραφεί ένα πρόγραμμα το οποίο με χρήση δεικτών να διαβάζει τις τιμές δύο ακεραίων και να τις αντιμεταθέτει.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int *ptr1, *ptr2;
5.      int i, j, tmp;
6.      ptr1 = &i;
7.      ptr2 = &j;
8.      printf("Enter numbers: ");
9.      scanf("%d%d", ptr1, ptr2);
10.     tmp = *ptr2;
11.     *ptr2 = *ptr1;
12.     *ptr1 = tmp;
13.     printf("Values: %d %d\n", i, j);
14.     return 0;
15. }
```

Παράδειγμα 78

Να συμπληρώσετε το παρακάτω πρόγραμμα χρησιμοποιώντας τον δείκτη **p** και έναν **while** βρόχο, ώστε να εμφανίζει τους ακέραιους από το **1** έως και το **10**.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.  int i, *p;
5.  ……..
6.  }
```

ΛΥΣΗ

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      int i, *p;
5.      p = &i;
6.     *p = 1;
7.      while(*p <= 10)
8.      {
9.          printf("%d\n", *p);
10.         (*p)++;
11.     }
12.     return 0;
13. }
```

Παράδειγμα 79

Να συμπληρώσετε το παρακάτω πρόγραμμα χρησιμοποιώντας τον δείκτη **p2** για να διαβάζετε συνεχώς βαθμούς φοιτητών, τον δείκτη **p1** για να εμφανίσετε πόσοι φοιτητές πήραν βαθμό στο **[5, 10]** και τον δείκτη **p3** για να εμφανίσετε τον μεγαλύτερο βαθμό. Αν ο χρήστης εισάγει την τιμή **-1**, η εισαγωγή των βαθμών να τερματίζεται.

```
1. #include <stdio.h>
2. int main()
3. {
4. int *p1, sum;
5. float *p2, *p3, grade, max;
6. …….
7. }
```

ΛΥΣΗ

```
1. #include <stdio.h>
2. int main()
3. {
4.     int *p1, sum;
5.     float *p2, *p3, grade, max;
6.     p1 = &sum;
7.     *p1 = 0;
8.     p3 = &max;
9.     *p3 = 0;
10.    p2 = &grade;
11.    while(1)
12.    {
13.        printf("Enter grade: ");
14.        scanf("%f", p2);
15.        if(*p2 == -1)
16.            break;
17.        if(*p2 >=5 && *p2 <= 10)
18.        {
19.            (*p1)++;
20.            if(*p2 > *p3)
21.                *p3 = *p2;
22.        }
23.    }
24.    printf("%d students passed (max = %.2f)\n", *p1, *p3);
25.    return 0;
26. }
```

Παράδειγμα 80

Να γραφεί ένα πρόγραμμα το οποίο με χρήση δεικτών να διαβάζει τους βαθμούς ενός φοιτητή σε τρεις εργασίες. Αν και οι τρεις βαθμοί είναι μεγαλύτεροι ή ίσοι του **5**, το πρόγραμμα να τους εμφανίζει με αύξουσα σειρά. Αλλιώς, να εμφανίζει τον μέσο όρο τους.

```
1.  #include <stdio.h>
2.  int main()
3.  {
4.      float *ptr1, *ptr2, *ptr3;
5.      float i, j, k;
6.      ptr1 = &i;
7.      ptr2 = &j;
8.      ptr3 = &k;
9.      printf("Enter grades: ");
10.     scanf("%f%f%f", ptr1, ptr2, ptr3);
11.     if((*ptr1 >= 5) && (*ptr2 >= 5) && (*ptr3 >= 5))
12.     {
13.         if(*ptr1 <= *ptr2 && *ptr1 <= *ptr3)
14.         {
15.             printf("%f ", *ptr1);
16.             if(*ptr2 < *ptr3)
17.                 printf("%f %f\n", *ptr2, *ptr3);
18.             else
19.                 printf("%f %f\n", *ptr3, *ptr2);
20.         }
21.         else
22.             if(*ptr2 <= *ptr1 && *ptr2 <= *ptr3)
23.             {
24.                 printf("%f ", *ptr2);
25.                 if(*ptr1 < *ptr3)
26.                     printf("%f %f\n", *ptr1, *ptr3);
27.                 else
28.                     printf("%f %f\n", *ptr3, *ptr1);
29.             }
30.       else
31.       {
32.           printf("%f ", *ptr3);
33.           if(*ptr2 < *ptr1)
34.               printf("%f %f\n", *ptr2, *ptr1);
35.           else
36.               printf("%f %f\n", *ptr1, *ptr2);
37.       }
38.     }
39.     else
40.         printf("Avg = %.2f\n", (*ptr1 + *ptr2 + *ptr3)/3);
41. return 0;
```

```
42. }
```

Παράδειγμα 81

Να συμπληρώσετε το παρακάτω πρόγραμμα χρησιμοποιώντας τους δείκτες **p1** και **p2**, για να εμφανίσετε το γινόμενο των άρτιων αριθμών από το **10** μέχρι και το **20**.

```
1. #include <stdio.h>
2. int main()
3. {
4. int i, mul, *p1, *p2;
5. …….
6. }
```

ΛΥΣΗ

```
1. #include <stdio.h>
2. int main()
3. {
4.       int i, mul, *p1, *p2;
5.
6.       p1 = &i;
7.       p2 = &mul;
8.       for(*p1 = 10, *p2 = 1; *p1 <= 20; (*p1)+=2)
9.                *p2 = *p2 * *p1;
10.
11.       printf("Mul = %d\n", *p2);
12.       return 0;
13. }
```

Παράδειγμα 82

Να συμπληρώσετε το παρακάτω πρόγραμμα ώστε να διαβάζει δύο ακέραιους και να εμφανίζει το άθροισμα των ακεραίων που μεσολαβούν μεταξύ τους, χρησιμοποιώντας τους δείκτες **p1**, **p2** και **p3**. Για παράδειγμα, αν ο χρήστης εισάγει τους ακεραίους **6** και **10**, το πρόγραμμα να εμφανίζει **24 (7 + 8 +9)**. Το πρόγραμμα να υποχρεώνει τον χρήστη να εισάγει τιμές μικρότερες από **100** και ο πρώτος ακέραιος να είναι μικρότερος από τον δεύτερο.

```
1. #include <stdio.h>
2. int main()
3. {
4. int *p1, *p2, *p3;
5. int i, j, sum;
6. ……..
7. }
```

ΛΥΣΗ

```
1. #include <stdio.h>
2. int main()
3. {
4.     int *p1, *p2, *p3;
5.     int i, j, sum;
6.     p1 = &i;
7.     p2 = &j;
8.     p3 = &sum;
9.     *p3 = 0;
10.    Do
11.    {
12.        printf("Enter two numbers (a < b < 100): ");
13.        scanf("%d%d", p1, p2);
14.     } while(*p1 >= *p2 || *p2 > 100);
15.    (*p1)++;
16.    while(*p1 < *p2)
17.    {
18.        *p3 += *p1;
19.        (*p1)++;
20.    }
21.    printf("Sum = %d\n", *p3);
22.    return 0;
23. }
```

Παράδειγμα 83

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει τους βαθμούς **10** φοιτητών, να τους αποθηκεύει σε έναν πίνακα και να εμφανίζει τη μεγαλύτερη και τη μικρότερη τιμή, καθώς και τις θέσεις τους στον πίνακα. Το πρόγραμμα να ελέγχει αν ο εισαγόμενος βαθμός ανήκει στο **[0, 10]**. Η διαχείριση του πίνακα να γίνει με σημειογραφία δείκτη.

```
1.  #include <stdio.h>
2.
3.  #define SIZE 10
4.
5.  int main()
6.  {
7.      int i, max_pos, min_pos;
8.      float max, min, arr[SIZE];
9.
10.     max = 0;
11.     min = 10;
12.     max_pos = min_pos = 0;
13.     for(i = 0; i < SIZE; i++)
14.     {
15.         Do
16.         {
17.             printf("Enter grade: ");
18.             scanf("%f", arr+i);
19.         } while(*(arr+i) > 10 || *(arr+i) < 0); /* έλεγχος αν ο βαθμός
20.                                                    ανήκει στο [0, 10] */
21.         if(*(arr+i) > max)
22.         {
23.             max = *(arr+i);
24.             max_pos = i;
25.         }
26.         if(*(arr+i) < min)
27.         {
28.            min = *(arr+i);
29.            min_pos = i;
30.         }
31.     }
32.     printf("Max grade is %.2f in pos #%d\n", max, max_pos);
33.     printf("Min grade is %.2f in pos #%d\n", min, min_pos);
34.     return 0;
35. }
```

Παράδειγμα 84

Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει τις ημερήσιες θερμοκρασίες του Ιανουαρίου σε μια περιοχή και να τις αποθηκεύει σε έναν πίνακα. Στη συνέχεια, να διαβάζει μία θερμοκρασία και να εμφανίζει την πρώτη μέρα που η θερμοκρασία ήταν μικρότερη από αυτήν. Η διαχείριση του πίνακα να γίνει με σημειογραφία δείκτη.

```
1.  #include <stdio.h>
2.
3.  #define SIZE 31
4.
5.  int main()
6.  {
7.      int i;
8.      double temp, arr[SIZE];
9.
10.     for(i = 0; i < SIZE; i++)
11.     {
12.         printf("Enter temperatures: ");
13.         scanf("%lf", arr+i); /* Το arr+i είναι ισοδύναμο με &arr[i] */
14.     }
15.     printf("Enter temperature to check: ");
16.     scanf("%lf", &temp);
17.     for(i = 0; i < SIZE; i++)
18.     { /* Τα άγκιστρα δεν χρειάζονται. Τα προσθέτουμε για λόγους
19.                 ευαναγνωσιμότητας */
20.          if(*(arr+i) < temp)
21.              break;
22.     }
23.     if(i == SIZE)
24.         printf("No temperature less than %.1f\n", temp);
25.     else
26.         printf("The first temperature less than %.1f was %.1f in day %d\n", temp,
27.                     *(arr+i), i+1);
28.      return 0;
29. }
```

Παράδειγμα 85

Να συμπληρώσετε το παρακάτω πρόγραμμα χρησιμοποιώντας τον δείκτη **ptr** για να διαβάσετε και να αποθηκεύσετε τους βαθμούς **50** φοιτητών στον πίνακα **arr** και να εμφανίσετε τις τιμές του πίνακα με αντίστροφη σειρά.

```
1.  #include <stdio.h>
2.  #define SIZE 50
3.
4.  int main()
5.  {
6.  float arr[SIZE], *ptr;
7.
8.  ptr = arr;
9.  ……..
10. }
```

ΛΥΣΗ

```
1.  #include <stdio.h>
2.  #define SIZE 50
3.
4.  int main()
5.  {
6.      float arr[SIZE], *ptr;
7.
8.      ptr = arr;
9.      while((ptr – arr) != SIZE)
10.     {
11.         printf("Enter grade: ");
12.         scanf("%f", ptr);
13.         ptr++;
14.     }
15.     ptr--;
16.     while(ptr >= arr)
17.     {
18.         printf("%f\n", *ptr);
19.         ptr--;
20.     }
21.     return 0;
22. }
```

Παράδειγμα 86

Να γραφεί ένα πρόγραμμα το οποίο να δηλώνει δύο πίνακες **10** ακεραίων και να τους αναθέτει τυχαίες τιμές. Επίσης, να δηλώσετε δύο δείκτες και με χρήση αυτών το πρόγραμμα να αντιγράφει τις τιμές του δεύτερου πίνακα στον πρώτο πίνακα και να εμφανίζει τα στοιχεία και των δύο πινάκων.

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <time.h>
4.
5.  #define SIZE 10
6.
7.  int main()
8.  {
9.      int *ptr1, *ptr2;
10.     int i, arr1[SIZE], arr2[SIZE];
11.
12.     srand((unsigned)time(NULL));
13.     for(i = 0; i < SIZE; i++)
14.     {
15.         arr1[i] = rand();
16.         arr2[i] = rand();
17.     }
18.     ptr1 = arr1;
19.     ptr2 = arr2;
20.     for(i = 0; i < SIZE; i++)
21.     {
22.          *ptr1 = *ptr2; /* Ισοδύναμο με arr1[i] = arr2[i]; */
23.          ptr1++;
24.          ptr2++;
25.     }
26.     ptr1 = arr1; /* Επαναφορά του δείκτη */
27.     ptr2 = arr2; /* Επαναφορά του δείκτη */
28.     for(i = 0; i < SIZE; i++)
29.          printf("arr1[%d] = %d, arr2[%d] = %d\n", i, *(ptr1+ i), i, *(ptr2+i));
30.     return 0;
31. }
```

Παράδειγμα 87

Να γραφεί ένα πρόγραμμα χρησιμοποιώντας τους δείκτες **ptr1** και **ptr2** για να διαβάσετε και να αποθηκεύσετε στον πίνακα **arr** τους ακέραιους κωδικούς **100** προϊόντων, με τον περιορισμό ένας κωδικός να αποθηκεύεται στον πίνακα μόνο αν δεν έχει ήδη αποθηκευτεί. Αν ο χρήστης εισάγει την τιμή **-1** η εισαγωγή των κωδικών να τερματίζει. Το πρόγραμμα να εμφανίζει τους κωδικούς πριν τερματίσει.

```
1.  #include <stdio.h>
2.  #define SIZE 100
3.  int main()
4.  {
5.      int *ptr1, *ptr2, arr[SIZE];
6.      ptr1 = arr;
7.    while(ptr1 < arr + SIZE)
8.    {
9.          printf("Enter code_%d ", ptr1 - arr + 1);
10.         scanf("%d", ptr1);
11.         if (*ptr1 == -1 )
12.             break;
13.         for (ptr2 = arr; ptr2 < ptr1; ptr2++) /* Ξεκινώντας από την αρχή του πίνακα
14.                                 Ελεγχουμε αν ο κωδικός έχει ήδη αποθηκευτεί */
15.        {
16.            if  (*ptr1 == *ptr2)
17.           {
18.               printf("Error: Code %d exists\n", *ptr1);
19.               break;
20.            }
21.        }
22. /* Αν ο κωδικός δεν υπάρχει στον πίνακα αυξάνουμε τον δείκτη. */
23.         if   (ptr2 == ptr1)
24.             ptr1++;
25. }
26. /* Εμφάνιση κωδικών. */
27. for (ptr2 = arr; ptr2 < ptr1; ptr2++)
28.          printf("C: %d\n", *ptr2);
29. return 0;
30. }
```

Παράδειγμα 88

Να συμπληρώσετε το παρακάτω πρόγραμμα χρησιμοποιώντας τον δείκτη **ptr** και έναν επαναληπτικό βρόχο για να μειώσετε τις τιμές των στοιχείων του πίνακα **arr** κατά ένα. Το πρόγραμμα να εμφανίζει το άθροισμα τους πριν τερματίσει.

```
1. #include <stdio.h>
2. int main()
3. {
4. int *ptr;
5. int sum, arr[5] = {11, 21, 31, 41, 51};
6. ……
7. }
```

ΛΥΣΗ

```
1. #include <stdio.h>
2. int main()
3. {
4.     int *ptr;
5.     int sum, arr[5] = {11, 21, 31, 41, 51};
6.
7.     sum = 0;
8.     for(ptr = arr; ptr <= arr+4; ptr++)
9.     {
10.        --*ptr;
11.        sum += *ptr;
12.     }
13.     printf("Sum = %d\n", sum);
14.     return 0;
15. }
```

Παράδειγμα 89

Δημιουργήστε μια συνάρτηση που να δέχεται σαν παραμέτρους τους βαθμούς δύο φοιτητών και να επιστρέφει τον μεγαλύτερο από αυτούς. Να γραφεί ένα πρόγραμμα το οποίο να διαβάζει δύο βαθμούς και να εμφανίζει τον μεγαλύτερο από αυτούς χρησιμοποιώντας δείκτη στη συνάρτηση.

```
1.  #include <stdio.h>
2.
3.  float test(float a, float b);
4.
5.  int main()
6.  {
7.      float (*ptr)(float a, float b); /* Η μεταβλητή ptr δηλώνεται σαν δείκτης σε μια
8.                                         συνάρτηση, η οποία δέχεται δύο πραγματικούς
9.                                         αριθμούς και επιστρέφει μία πραγματική τιμή */
10.     float i, j, max;
11.
12.     printf("Enter grades: ");
13.     scanf("%f%f", &i, &j);
14.
15.     ptr = test;
16.     max = (*ptr)(i, j); /* Κλήση της συνάρτησης στην οποία δείχνει ο δείκτης ptr */
17.     printf("Max = %f\n", max);
18.     return 0;
19. }
20.
21. float test(float a, float b)
22. {
23.    if(a > b)
24.        return a;
25.    else
26.        return b;
27. }
```

ΧΑΡΑΚΤΗΡΕΣ ΚΑΙ ΓΡΑΜΜΑΤΟΣΕΙΡΕΣ (ΣΥΜΒΟΛΟΣΕΙΡΕΣ)

- Μια σταθερά ενός χαρακτήρα είναι μια ακεραία (**int**) τιμή που αντιπροσωπεύεται σαν ένας χαρακτήρας ανάμεσα στα απλά εισαγωγικά.

- Η τιμή μιας σταθεράς ενός χαρακτήρα είναι η ακέραια τιμή του χαρακτήρα στο σύνολο χαρακτήρων της μηχανής (machine's character set).

  Για παράδειγμα,

  '**z**' αντιπροσωπεύει την ακέραια τιμή του **z**.
  '**\n**' αντιπροσωπεύει την ακέραια τιμή της νέας γραμμής.

- Μια γραμματοσειρά είναι μια σειρά χαρακτήρων που αντιμετωπίζεται ως ενιαία μονάδα. Μια γραμματοσειρά μπορεί να περιλαμβάνει γράμματα, ψηφία και διάφορους ειδικού χαρακτήρες όπως **+**, **-**, *****, **/** και **$**.

- Οι σταθερές γραμματοσειράς στη **C** γράφονται ανάμεσα σε διπλά εισαγωγικά όπως:

  "John Q. Doe" ---------- **ένα όνομα**
  "99999 Main Street" ---------- **μια διεύθυνση**
  "Waltham, Massachusetts" --------- **μια πόλη**
  "(201) 555-1212" ---------- **ένας αριθμός τηλεφώνου**

- Μια γραμματοσειρά είναι ένας πίνακας χαρακτήρων που τελειώνει με το χαρακτήρα (null) ('**\0**').

- Μια γραμματοσειρά είναι προσβάσιμη μέσω ενός δείκτη στον πρώτο της χαρακτήρα.

- Η τιμή μιας γραμματοσειράς είναι η διεύθυνση του πρώτου της χαρακτήρα στη μνήμη. Ως εκ τούτου, στη **C** είναι σκόπιμο να πούμε ότι μια γραμματοσειρά είναι ένας δείκτης – στην πραγματικότητα, ένας δείκτης στον πρώτο χαρακτήρα της γραμματοσειράς. (Με αυτή την έννοια, οι γραμματοσειρές θεωρούνται πίνακες, επειδή ένας πίνακας είναι επίσης ένας δείκτης στο πρώτο του στοιχείο.)

- Μια γραμματοσειρά μπορεί να αντιστοιχηθεί στον ορισμό σε είτε ένα πίνακα χαρακτήρων ή σε μια μεταβλητή τύπου **char ***.

  Για παράδειγμα,

  (a) char color[] = "blue";
  (b) const char *colorPtr = "blue";

Το παράδειγμα **(a)** δημιουργεί τον 5-στοιχείων πίνακα **color** που περιέχει 'b', 'l', 'u', 'e' και '\0'.
Το παράδειγμα **(b)** δημιουργεί τη μεταβλητή δείκτη **colorPtr** ο οποίος δείχνει στη γραμματοσειρά "blue" κάπου στη μνήμη.

Το παράδειγμα (a) μπορεί επίσης να γραφτεί σαν

```
char color[] = {'b', 'l', 'u', 'e', '\0'};
```

- Μια γραμματοσειρά μπορεί να αντιστοιχίσει σε ένα πίνακα με τη χρήση της εντολής **scanf**.

Για παράδειγμα, η ακόλουθη εντολή αντιστοιχίζει στον πίνακα χαρακτήρων **word[20]** τιμές:

```
scanf( "%s", word );
```

Η εντολή **scanf** θα διαβάζει χαρακτήρες μέχρι να συναντησεί ένα κενό, νέα γραμμή ή end-of-file δείκτης συναντηθεί. Σημειώστε ότι η γραμματοσειρά σε αυτό το παράδειγμα δεν μπορεί να είναι μεγαλύτερη από **19** χαρακτήρες για το λόγο ότι υπάρχει χώρος για το χαρακτήρα null. Ένας πίνακας χαρακτήρων για να τυπωθεί σαν γραμματοσειρά, ο πίνακας πρέπει να περιέχει τον τερματικό χαρακτήρα null.

Βιβλιοθήκη χειρισμού χαρακτήρων

Συμπεριλαμβάνει διάφορες συναρτήσεις που εκτελούν χρήσιμες δοκιμές και χειρισμούς των χαρακτήρων. Η κάθε συνάρτηση παίρνει ένα χαρακτήρα που αντιπροσωπεύεται σαν **int** ή **EOF** σαν όρισμα.

Πρωτότυπο	Περιγραφή συνάρτησης
int isdigit (int c);	Επιστρέφει true εάν c είναι ψηφίο διαφορετικά επιστρέφει 0 (false).
int isalpha (int c);	Επιστρέφει true εάν c είναι γράμμα, διαφορετικά επιστρέφει 0 (false).
int isalnum (int c);	Επιστρέφει true εάν c είναι γράμμα ή ψηφίο, διαφορετικά επιστρέφει 0 (false).
int isxdigit (int c);	Επιστρέφει true εάν c είναι δεκαεξαδικό ψηφίο, διαφορετικά επιστρέφει 0 (false).
int islower (int c);	Επιστρέφει true εάν c είναι μικρό γράμμα, διαφορετικά επιστρέφει 0 (false).
int isupper (int c);	Επιστρέφει true εάν c είναι κεφαλαίο γράμμα, διαφορετικά επιστρέφει 0 (false).
int tolower (int c);	Εάν c είναι κεφαλαίο, η **tolower** επιστρέφει c σαν μικρό γράμμα διαφορετικά η **tolower** επιστρέφει το όρισμα χωρίς να το αλλάξει.
int toupper (int c);	Εάν c είναι μικρό γράμμα, η **tolupper** επιστρέφει c σαν κεφαλαίο γράμμα διαφορετικά η **tolupper** επιστρέφει το όρισμα χωρίς να το αλλάξει.
int isspace (int c);	Επιστρέφει true εάν c είναι κενό (' '), νέα γραμμή ('\n'), αλλαγή σελίδας ('\f') (Form feed), επαναφορά γραμμής ('\r') (Carriage return), κάθετος στηλοθέτης ('\v') (vertical tab), οριζόντιος στηλοθέτης ('\b') (horizontal tab), διαφορετικά επιστρέφει 0 (false).
int iscntr1 (int c);	Επιστρέφει true εάν c είναι ένας χαρακτήρας ελέγχου (a control character), διαφορετικά επιστρέφει 0 (false).
int ispunct (int c);	Επιστρέφει true εάν c είναι χαρακτήρας εκτύπωσης άλλος από κενό, ψηφίο ή γράμμα, διαφορετικά επιστρέφει 0 (false).
int isprint (int c);	Επιστρέφει true εάν c είναι χαρακτήρας εκτύπωσης συμπεριλαμβανομένου του κενού (' '), διαφορετικά επιστρέφει 0 (false).
int isgraph (int c);	Επιστρέφει true εάν c είναι δεκαεξαδικό ψηφίο, διαφορετικά επιστρέφει 0 (false).

Παράδειγμα 90

Το παράδειγμα αυτό παρουσιάζει τις συναρτήσεις **isdigit, isalpha, isalnum** και **isxdigit**.

```
1.  /* Using functions isdigit, isalpha, isalnum, and isxdigit */
2.
3.  #include <stdio.h>
4.  #include <ctype.h>
5.
6.  int main()
7.  {
8.          printf( "%s\n%s%s\n%s%s\n\n", "According to isdigit: ",
9.              isdigit( '8' ) ? "8 is a " : "8 is not a ", "digit",
10.             isdigit( '#' ) ? "# is a " :
11.             "# is not a ", "digit" );
12.         printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n\n",
13.             "According to isalpha:",
14.              isalpha( 'A' ) ? "A is a " : "A is not a ", "letter",
15.              isalpha( 'b' ) ? "b is a " : "b is not a ", "letter",
16.              isalpha( '&' ) ? "& is a " : "& is not a ", "letter",
17.              isalpha( '4' ) ? "4 is a " :
18.              "4 is not a ", "letter" );
19.         printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
20.            "According to isalnum:",
21.             isalnum( 'A' ) ? "A is a " : "A is not a ",
22.             "digit or a letter",
23.             isalnum( '8' ) ? "8 is a " : "8 is not a ",
24.             "digit or a letter",
25.             isalnum( '#' ) ? "# is a " : "# is not a ",
26.             "digit or a letter" );
27.        printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n%s%s\n",
28.             "According to isxdigit:",
29.             isxdigit( 'F' ) ? "F is a " : "F is not a ",
30.             "hexadecimal digit",
31.             isxdigit( 'J' ) ? "J is a " : "J is not a ",
32.             "hexadecimal digit",
33.             isxdigit( '7' ) ? "7 is a " : "7 is not a ",
34.             "hexadecimal digit",
35.             isxdigit( '$' ) ? "$ is a " : "$ is not a ",
36.             "hexadecimal digit",
37.             isxdigit( 'f' ) ? "f is a " : "f is not a ",
38.             "hexadecimal digit" );
39. return 0;
40. }
```

ΕΞΟΔΟΣ:

```
According to is digit:
8 is a digit
# is not a digit

According to isalpha:
A is a letter
b is a letter
& is not a letter
4 is not a letter

According to isalnum:
A is a digit or a letter
8 is a digit or a letter
# is not a digit or a letter

According to isxdigit:
F is a hexadecimal digit
J is not a hexadecimal digit
7 is a hexadecimal digit
$ is not hexadecimal digit
f is hexadecimal digit
```

Παράδειγμα 91

Το παράδειγμα αυτό παρουσιάζει τις συναρτήσεις **islower, isupper, tolower** και **toupper**.

```
1.  /* Using functions islower, isupper, tolower, toupper */
2.
3.  #include <stdio.h>
4.  #include <ctype.h>
5.
6.  int main()
7.  {
8.          printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n\n",
9.               "According to islower:",
10.              islower( 'p' ) ? "p is a " : "p is not a ",
11.              "lowercase letter",
12.              islower( 'P' ) ? "P is a " : "P is not a ",
13.              "lowercase letter",
14.               islower( '5' ) ? "5 is a " : "5 is not a ",
15.               "lowercase letter",
16.                islower( '!' ) ? "! is a " : "! is not a ",
17.                "lowercase letter" );
18.         printf( "%s\n%s%s\n%s%s\n%s%s\n%s%s\n\n",
19.              "According to isupper:",
20.              isupper( 'D' ) ? "D is an " : "D is not an ",
21.              "uppercase letter",
22.              isupper( 'd' ) ? "d is an " : "d is not an ",
23.              "uppercase letter",
24.               isupper( '8' ) ? "8 is an " : "8 is not an ",
25.               "uppercase letter",
26.                isupper( '$' ) ? "$ is an " : "$ is not an ",
27.                "uppercase letter" );
28.         printf( "%s%c\n%s%c\n%s%c\n%s%c\n",
29.                 "u converted to uppercase is ", toupper( 'u' ),
30.                 "7 converted to uppercase is ", toupper( '7' ),
31.                 "$ converted to uppercase is ", toupper( '$' ),
32.                 "L converted to lowercase is ", tolower( 'L' ) );
33.          return 0;
34.  }
```

ΕΞΟΔΟΣ:

```
According to islower:
p is a lowercase letter
P is not a lowercase letter
5 is not a lowercase letter
! is not a lowercase letter

According to isupper:
D is an uppercase letter
d is not an uppercase letter
8 is not an uppercase letter
$ is not an uppercase letter

u converted to uppercase is U
7 converted to uppercase is 7
$ converted to uppercase is $
L converted to lowercase is l
```

Παράδειγμα 92

Το παράδειγμα αυτό παρουσιάζει τις συναρτήσεις **isspace, iscntr1, ispunct, isprint** και **isgraph**.

```
1.  /*  Using functions isspace, iscntrl, ispunct, isprint, isgraph */
2.
3.  #include <stdio.h>
4.  #include <ctype.h>
5.
6.  int main()
7.  {
8.          printf( "%s\n%s%s%s\n%s%s%s\n%s%s\n\n",
9.               "According to isspace:",
10.              "Newline", isspace( '\n' ) ? " is a " : " is not a ",
11.              "whitespace character", "Horizontal tab",
12.               isspace( '\t' ) ? " is a " : " is not a ",
13.               "whitespace character",
14.               isspace( '%' ) ? "% is a " : "% is not a ",
15.               "whitespace character" );
16.         printf( "%s\n%s%s%s\n%s%s\n\n", "According to iscntrl:",
17.               "Newline", iscntrl( '\n' ) ? " is a " : " is not a ",
18.               "control character", iscntrl( '$' ) ? "$ is a " :
19.               "$ is not a ",
20.               "control character" );
21.         printf( "%s\n%s%s\n%s%s\n%s%s\n\n",
22.              "According to ispunct:",
23.               ispunct( ';' ) ? "; is a " : "; is not a ",
24.               "punctuation character",
25.                ispunct( 'Y' ) ? "Y is a " : "Y is not a ",
26.                "punctuation character",
27.                ispunct( '#' ) ? "# is a " : "# is not a ",
28.                "punctuation character" );
29.         printf( "%s\n%s%s\n%s%s%s\n\n", "According to isprint:",
30.               isprint( '$' ) ? "$ is a " : "$ is not a ",
31.               "printing character",
32.               "Alert", isprint( '\a' ) ? " is a " : " is not a ",
33.               "printing character" );
34.         printf( "%s\n%s%s\n%s%s%s\n",  "According to isgraph:",
35.               isgraph( 'Q' ) ? "Q is a " : "Q is not a ",
36.               "printing character other than a space",
37.               "Space", isgraph( ' ' ) ? " is a " : " is not a ",
38.               "printing character other than a space" );
39.         return 0;
40. }
```

ΕΞΟΔΟΣ:

```
According to isspace:
Newline is a whitespace character
Horizontal tab is a whitespace character
% is not a whitespace character

According to iscntr1:
Newline is a control character
$ is not a control character

According to ispunct:
; is a punctuation character
Y is not a punctuation character
# is a punctuation character

According to isprint:
$ is a printing character
Alert is not a printing character

According to isgraph:
Q is a printing character other than a space
Space is not a printing character other than a space
```

Συναρτήσεις μετατροπής γραμματοσειράς

Αυτές οι συναρτήσεις (βιβλιοθήκη γενικών υπηρεσιών (**stdlib**)) μετατρέπουν γραμματοσειρές από ψηφία σε ακέραιες τιμές κινητής υποδιαστολής.

Πρωτότυπο συνάρτησης	**Επεξήγηση συνάρτησης**
double atof(const char *nPtr);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **double**.
int atoi(const char *nPtr);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **int**.
long atol(const char *nPtr);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **long int**.
double strtod (const char *nPtr, char **endPtr);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **double**.
long strtol (const char *nPtr, char **endPtr, int base);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **long**.
unsigned long strtoul (const char *nPtr, char **endPtr, int base);	Μετατρέπει τη γραμματοσειρά **nPtr** σε **unsigned long**.

**** Σημείωση:** Η χρήση του **const** στον ορισμό της μεταβλητής **nPtr** στη κεφαλίδα της συνάρτησης δηλώνει ότι η τιμή του ορίσματος δεν μπορεί να αλλάξει.

Παράδειγμα 93 (χρήση της atof)

```
1.  /*  Using atof */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.          double d;
9.
10.         d = atof( "99.0" );
11.         printf( "%s%.3f\n%s%.3f\n",
12.                 "The string \"99.0\" converted to double is ", d,
13.                 "The converted value divided by 2 is ",
14.                 d / 2.0 );
15.         return 0;
16. }
```

ΕΞΟΔΟΣ:

```
The string "99.0" converted to double is 99.000
The converted value divided by 2 is 49.500
```

Παράδειγμα 94 (χρήση της atoi)

```
1.  /* Using atoi */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.      int i;
9.
10.     i = atoi( "2593" );
11.     printf( "%s%d\n%s%d\n",
12.             "The string \"2593\" converted to int is ", i,
13.             "The converted value minus 593 is ", i - 593 );
14.     return 0;
15. }
```

ΕΞΟΔΟΣ:

```
The string “2593” converted to int is 2593
The converted value minus 593 is 2000
```

Παράδειγμα 95 (χρήση της atol)

```
1.  /* Using atol */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.      long l;
9.
10.     l = atol( "1000000" );
11.     printf( "%s%ld\n%s%ld\n",
12.             "The string \"1000000\" converted to long int is ", l,
13.             "The converted value divided by 2 is ", l / 2 );
14.     return 0;
15. }
```

ΕΞΟΔΟΣ:

```
The string "1000000" converted to long is 1000000
The converted value divided by 2 is 500000
```

**** Γενική Σημείωση:** Εάν η τιμή μετατροπής δεν μπορεί να αντιπροσωπευθεί η συμπεριφορά όλων των πιο πάνω συναρτήσεων **(παρ. 47, παρ. 48, παρ.49)** δεν μπορεί να προβλεφθεί.

Παράδειγμα 96 (χρήση της strtod)

- Αυτή η συνάρτηση παίρνει δύο ορίσματα – μια γραμματοσειρά και ένα δείκτη στη γραμματοσειρά.
- Η γραμματοσειρά περιέχει τη σειρά των χαρακτήρων που θα μετατραπούν σε double. Στο **δείκτη (pointer)** δίδεται η θέση του πρώτου χαρακτήρα μετά την μετατροπή μέρους της γραμματοσειράς.

  Η εντολή,

  d = strtod(string, &stringPtr);

  δείχνει ότι στο **d** δίνεται η **double** τιμή που μετατρέπεται από τη γραμματοσειρά **string**, και στο **stringPtr** δίνεται η θέση του πρώτου χαρακτήρα της μετατρεπόμενης τιμής (**51.2**).

```
1.  /* Using strtod */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.        double d;
9.        const char *string = "51.2% are admitted";
10.       char *stringPtr;
11.
12.       d = strtod( string, &stringPtr );
13.       printf( "The string \"%s\" is converted to the\n",
14.             string );
15.       printf( "double value %.2f and the string \"%s\"\n",
16.             d, stringPtr );
17.
18.       return 0;
19. }
```

ΕΞΟΔΟΣ:

```
The string “51.2% are admitted” is converted to the
double value 51.20 and the string “% are admitted”
```

Παράδειγμα 97 (χρήση της strtol)

- Η συνάρτηση δέχεται τρία ορίσματα – μια γραμματοσειρά (char *), ένα δείκτη σε γραμματοσειρά, και έναν ακέραιο.
- Η γραμματοσειρά περιέχει τη σειρά των χαρακτήρων που θα μετατραπεί.
- Στο **δείκτη** (**pointer)** δίδεται η θέση του πρώτου χαρακτήρα μετά την μετατροπή μέρους της γραμματοσειράς.
- Ο ακέραιος καθορίζει τη **βάση (base)** της μετατρεπόμενης τιμής.

Η εντολή,

```
x = strtol( string, &remainderPtr, 0);
```

δείχνει ότι το **x** παίρνει τη μετατρεπόμενη τιμή σε **long** από το **string**, και το **remainderPtr** δίνεται το υπόλοιπο του **string** μετά τη μετατροπή.

**** Σημείωση:** Η χρήση του **NULL** για το δεύτερο όρισμα είναι η αιτία να αγνοηθεί το υπόλοιπο.

0 δείχνει ότι η τιμή που θα μετατραπεί μπορεί να είναι σε οκταδική (octal), δεκαδική (decimal) ή δεκαεξαδική (hexadecimal) μορφή.

Η βάση μπορεί να είναι **0** ή οποιαδήποτε τιμή μεταξύ **2-36**.

Παραδείγματα:

Ένας **βάση-11** ακέραιος μπορεί να αποτελείται από τα ψηφία **0-9** και το χαρακτήρα **A**.

Ένας **βάση-24** ακέραιος μπορεί να αποτελείται από τα ψηφία **0-9** και το χαρακτήρα **A-N**.

Ένας **βάση-36** ακέραιος μπορεί να αποτελείται από τα ψηφία **0-9** και το χαρακτήρα **A-Z**.

```
1.  /* Using strtol */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.        long x;
9.        const char *string = "-1234567abc";
10.       char *remainderPtr;
11.
12.       x = strtol( string, &remainderPtr, 0 );
13.       printf( "%s\"%s\"\n%s%ld\n%s\"%s\"\n%s%ld\n",
14.                 "The original string is ", string,
15.                 "The converted value is ", x,
16.                 "The remainder of the original string is ",
17.                  remainderPtr,
18.                 "The converted value plus 567 is ", x + 567 );
19.       return 0;
20. }
```

ΕΞΟΔΟΣ:

```
The original string is "-1234567abc"
The converted value is -1234567
The remainder of the original string is "abc"
The converted value plus 567 is -1234000
```

Παράδειγμα 98 (χρήση της strtoul)

Αυτή η συνάρτηση εκτελείται το ίδιο όπως η συνάρτηση **strtol**.

```
1.  /* Using strtoul */
2.
3.  #include <stdio.h>
4.  #include <stdlib.h>
5.
6.  int main()
7.  {
8.          unsigned long x;
9.          const char *string = "1234567abc";
10.         char *remainderPtr;
11.
12.         x = strtoul( string, &remainderPtr, 0 );
13.         printf( "%s\"%s\"\n%s%lu\n%s\"%s\"\n%s%lu\n",
14.                 "The original string is ", string,
15.                 "The converted value is ", x,
16.                 "The remainder of the original string is ",
17.                  remainderPtr,
18.                  "The converted value minus 567 is ", x - 567 );
19.          return 0;
20. }
```

ΕΞΟΔΟΣ:

```
The original string is “1234567abc”
The converted value is 1234567
The remainder of the original string is “abc”
The converted value minus 567 is 1234000
```

Συναρτήσεις της βασικής βιβλιοθήκης Input\Output (Standard Input\Output Library functions)

Αυτή η ενότητα παρουσιάζει διάφορες συναρτήσεις από την βασική βιβλιοθήκη (**stdio**) ειδικά για χειρισμό δεδομένων χαρακτήρων και γραμματοσειρών.

Πρωτότυπο συνάρτησης	**Περιγραφή συνάρτησης**
int getchar(void);	Εισάγει τον πρώτο χαρακτήρα από το πληκτρολόγιο και το επιστρέφει σαν ακέραιο.
char *gets(char *s);	Εισάγει χαρακτήρες από το πληκτρολόγιο στον πίνακα **s** μέχρι να βρει το χαρακτήρα new line ή end-of-file. Ένας τερματικός null χαρακτήρας character προσθέτεται στο τέλος του πίνακα.
int putchar(int c);	Εκτυπώνει το χαρακτήρα που είναι αποθηκευμένο στο **c**.
int puts(const char *s);	Εκτυπώνει τη γραμματοσειρά **s**.
int sprintf(char *s, const char *format, ……..);	Είναι το ίδιο με το printf εκτός στο ότι το αποτέλεσμα αποθηκεύεται στον πίνακα **s** από το να εκτυπωθεί στην οθόνη.
int sscanf(char *s, const char *format, ……..);	Είναι το ίδιο όπως το scanf εκτός στο ότι το input δίνεται από τον πίνακα **s** αντί από το πληκτρολόγιο.

Παράδειγμα 99 (η χρήση της gets και putchar)

Αυτό το παράδειγμα χρησιμοποιεί τις συναρτήσεις **gets** και **putchar** για να διαβάσει μια γραμμή κειμένου από το πληκτρολόγιο και αναδρομικά εκτυπώνει τους χαρακτήρες της γραμμής αντίστροφα.

```
1.  /* Using gets and putchar */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.        char sentence[ 80 ];
8.        void reverse( const char * const );
9.
10.       printf( "Enter a line of text:\n" );
11.       gets( sentence );
12.
13.       printf( "\nThe line printed backwards is:\n" );
14.       reverse( sentence );
15.
16.       return 0;
17.  }
18.
19.  void reverse( const char * const sPtr )
20.  {
21.       if ( sPtr[ 0 ] == '\0' )
22.            return;
23.       else {
24.           reverse( &sPtr[ 1 ] );
25.           putchar( sPtr[ 0 ] );
26.       }
27.  }
```

1ο Τρέξιμο

ΕΞΟΔΟΣ:

```
Enter a line of text:
Characters and strings

The line printed backwards is:
sgnirtS dna sretcarahC
```

2ο Τρέξιμο

ΕΞΟΔΟΣ:

```
Enter a line of text:
Characters and strings

The line printed backwards is:
sgnirtS dna sretcarahC
```

- Η συνάρτηση **gets** διαβάζει χαρακτήρες από το πληκτρολόγιο – ένα πίνακα χαρακτήρων – μέχρι τον χαρακτήρα newline ή μέχρι να βρει end-of-file.
- Η συνάρτηση **putchar** εκτυπώνει το όρισμα χαρακτήρα.
- Το πρόγραμμα καλεί τη συνάρτηση αναδρομής **reverse** για να εκτυπώσει τη γραμμή του κειμένου αντίστροφα.

Παράδειγμα 100 (η χρήση της gets και της puttcharl)

Αυτό το παράδειγμα χρησιμοποιεί τις συναρτήσεις **getchar** και **puts** για να διαβάσει χαρακτήρες από το πληκτρολόγιο στον πίνακα **sentence** και εκτυπώνει τον πίνακα σαν μια γραμματοσειρά.

- Η συνάρτηση **getchar** διαβάζει ένα χαρακτήρα από το πληκτρολόγιο και το επιστρέφει σαν ένα ακέραιο.
- Η συνάρτηση **puts** δέχεται μια γραμματοσειρά (char *) σαν όρισμα και την εκτυπώνει ακολουθούμενη από το newline χαρακτήρα.

```
1.  /* Using getchar and puts */
2.
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char c, sentence[ 80 ];
8.      int i = 0;
9.
10.     puts( "Enter a line of text:" );
11.     while ( ( c = getchar() ) != '\n')
12.         sentence[ i++ ] = c;
13.
14.     sentence[ i ] = '\0';                   /* insert null at end of string */
15.     puts( "\nThe line entered was:" );
16.     puts( sentence );
17.     return 0;
18. }
```

ΕΞΟΔΟΣ:

```
Enter a line of text:
This is a test.

The line entered was:
This is a test.
```

Το πρόγραμμα σταματά να εισάγει χαρακτήρες όταν η συνάρτηση **getchar** διαβάσει το newline χαρακτήρα που εισάγεται από το χρήστη για να τερματίσει τη γραμμή του κειμένου. Ο null χαρακτήρας δίνεται στο τέλος του πίνακα **sentence** έτσι ώστε ο πίνακας να θεωρηθεί σαν μια γραμματοσειρά.

- Η συνάρτηση **puts** εκτυπώνει τη γραμματοσειρά που περιέχεται στον πίνακα **sentence**.

Παράδειγμα 101 (η χρήση της sprintf)

Αυτό το παράδειγμα χρησιμοποιεί τη συνάρτηση **sprint** για να εκτυπώσει μορφοποιημένα δεδομένα στον πίνακα **s** – ένας πίνακας χαρακτήρων.

```
1. /* Using sprintf */
2.
3. #include <stdio.h>
4.
5. int main()
6. {
7.     char s[ 80 ];
8.     int x;
9.     double y;
10.
11.     printf( "Enter an integer and a double:\n" );
12.     scanf( "%d%lf", &x, &y );
13.     sprintf( s, "integer:%6d\ndouble:%8.2f", x, y );
14.     printf( "%s\n%s\n",
15.     "The formatted output stored in array s is:", s );
16.     return 0;
17. }
```

ΕΞΟΔΟΣ:

```
Enter an integer and a double:
298 87.375
The formatted output stored in array s is:
integer:     298
double:    87.38
```

Παράδειγμα 102 (η χρήση της sscanf)

Αυτό το παράδειγμα χρησιμοποιεί τη συνάρτηση **sscanf** για να διαβάσει μορφοποιημένα δεδομένα από τον πίνακα χαρακτήρων **s**.

```
1. /* Using sscanf */
2.
3. #include <stdio.h>
4.
5. int main()
6. {
7.      char s[] = "31298 87.375";
8.      int x;
9.     double y;
10.
11.       sscanf( s, "%d%lf", &x, &y );
12.       printf( "%s\n%s%6d\n%s%8.3f\n",
13.    "The values stored in character array s are:",
14.    "integer:", x, "double:", y );
15.    return 0;
16.  }
```

ΕΞΟΔΟΣ:

```
The values stored in character array s are:
integer: 31298
double:  87.375
```

Συναρτήσεις χειρισμού της Βιβλιοθήκης χειρισμού γραμματοσειρών (String Handling Library)

Αυτή η βιβλιοθήκη μας παρέχει πολλές χρήσιμες συναρτήσεις για χειρισμό δεδομένων γραμματοσειρών, όπως, σύγκριση, αναζήτηση άλλων γραμματοσειρών ή χαρακτήρων, tokenizing γραμματοσειρές (χωρισμός γραμματοσειρών σε λογικά κομμάτια) και προσδιορισμό μεγέθους γραμματοσειρών.

Οι συναρτήσεις δίνονται περιληπτικά στο ακόλουθο σχεδιάγραμμα. Η κάθε συνάρτηση εκτός της **strncpy** στο τέλος προσαρτά το χαρακτήρα **null**.

Πρωτότυπο συνάρτησης	**Περιγραφή συνάρτησης**
char *strcpy(char *s1, const char *s2)	Αντιγράφει τη γραμματοσειρά **s2** στον πίνακα **s1**. Επιστρέφεται η τιμή του **s1**.
char *strncpy(char *s1, const char *s2, size_t n)	Αντιγράφει τουλάχιστο n χαρακτήρες της γραμματοσειράς **s2** στον πίνακα **s1**. Επιστρέφεται η τιμή του **s1**.
char *strcat(char *s1, const char *s2)	Προσαρτά (append) τη γραμματοσειρά **s2** στον πίνακα **s1**. Ο πρώτος χαρακτήρας του **s2** αντικαθιστά τον τερματικό χαρακτήρα null του **s1.** Επιστρέφεται η τιμή του **s1**.
char *strncat(char *s1, const char *s2, size_t n)	Προσαρτά τουλάχιστο n χαρακτήρες της γραμματοσειράς **s2** στον πίνακα **s1**. Ο πρώτος χαρακτήρας του **s2** αντικαθιστά τον τερματικό χαρακτήρα null του **s1.** Επιστρέφεται η τιμή του **s1**.

Σημείωση: Η παράμετρος τύπου **size_t** είναι ένας τύπος που ορίζεται από το πρότυπο ως αναπόσπαστον τύπο της τιμής που επιστρέφεται στον τελεστή **sizeof**.

Παράδειγμα 103 (χρήση της strcpy και της strncpy)

Αυτό το παράδειγμα χρησιμοποιεί τη συνάρτηση **strcpy** για να αντιγράψει ολόκληρη τη γραμματοσειρά του πίνακα **x** στον πίνακα **y** και χρησιμοποιεί τη συνάρτηση **strncpy** για να αντιγράψει τους πρώτους 14 χαρακτήρες του πίνακα **x** στον πίνακα **z**.

```
1.  /* Using strcpy and strncpy */
2.
3.  #include <stdio.h>
4.  #include <string.h>
5.
6.  int main()
7.  {
8.      char x[] = "Happy Birthday to You";
9.      char y[ 25 ], z[ 15 ];
10.
11.     printf( "%s%s\n%s%s\n",
12.     "The string in array x is: ", x,
13.     "The string in array y is: ", strcpy( y, x ) );
14.
15.     strncpy( z, x, 14 );
16.     z[ 14 ] = '\0';
17.     printf( "The string in array z is: %s\n", z );
18.     return 0;
19. }
```

ΕΞΟΔΟΣ:

```
The string in array x is: Happy Birthday to You
The string in array y is: Happy Birthday to You
The string in array z is: Happy Birthday
```

Σημείωση: Ο null χαρακτήρας (’\0’) δίνεται στο τέλος του πίνακα **z**, για το λόγο ότι η κλήση της **strncpy** δεν γράφει τον τερματικό null χαρακτήρα (το τρίτο όρισμα είναι μικρότερο από το μέγεθος του δευτέρου ορίσματος).

Μορφοποιημένοι Είσοδοι / Έξοδοι (Input / Output) στη C

Ένα σημαντικό μέρος της λύσης οποιουδήποτε προβλήματος είναι η παρουσίαση των αποτελεσμάτων.

Όλοι οι είσοδοι κα έξοδοι εκτελούνται με **streams**, σειρές χαρακτήρων οργανωμένες σε γραμμές. Η κάθε γραμμή αποτελείται από 0 ή περισσότερους χαρακτήρες και τελειώνει με το χαρακτήρα νέας γραμμής (new line character). Όλες οι εφαρμογές της C πρέπει να υποστηρίζουν γραμμές τουλάχιστον 254 χαρακτήρων συμπεριλαμβανομένου του τερματικού χαρακτήρα.

Όταν ξεκινά η εκτέλεση ενός προγράμματος τρία streams ενώνονται με το πρόγραμμα αυτόματα. Κανονικά το stream για εισόδους ενώνεται με το πληκτρολόγιο και το stream για εξόδους με την οθόνη του Η/Υ. Τα λειτουργικά συστήματα συχνά μας επιτρέπουν να αλλάξουμε την κατεύθυνση αυτών των streams για άλλες συσκευές. Το τρίτο stream, standard error, είναι ενωμένο με την οθόνη όπου τυπώνονται τα μηνύματα λάθους.

Μορφοποίηση εξόδων με την εντολή printf

Η ακριβή μορφοποίηση εξόδων επιτυγχάνεται με τη χρήση της printf. Κάθε κλήση της printf συμπεριλαμβάνει μια μορφοποιημένη γραμματοσειρά ελέγχου που περιγράφει τη μορφοποίηση των εξόδων. Αυτό αποτελείται από προσδιοριστικά μετατροπής (conversion specifiers), σημαίες (flags), μεγέθη πεδίου (field widths), χαρακτήρες ακρίβειας (precise characters) και χαρακτήρες λεκτικών σταθερών (literals). Μαζί με το σύμβολο % αυτά μας μορφοποιούν τις προδιαγραφές μετατροπής.

Η συνάρτηση **printf** μπορεί να εκτελέσει τις ακόλουθες μορφοποιήσεις.

- Στρογγυλοποίηση τιμών κινητής υποδιαστολής (floating-point) σε συγκεκριμένους αριθμούς δεκαδικών ψηφίων.
- Ευθυγράμμιση μιας στήλης από αριθμούς με δεκαδικά σημεία για να παρουσιάζονται το ένα πάνω από το άλλο.
- Δεξιά και αριστερή ευθυγράμμιση αποτελεσμάτων.
- Τοποθέτηση λεκτικών σταθερών σε ακριβή τοποθεσία σε μια γραμμή αποτελεσμάτων.
- Αντιπροσώπευση αριθμών με κινητή υποδιαστολή σε εκθετική μορφή (exponential format).
- Αντιπροσώπευση αριθμών χωρίς πρόσημο σε οκταδική ή δεκαεξαδική μορφοποίηση.
- Παρουσίαση όλων των τύπων δεδομένων με σταθερό μέγεθος πεδίο και ακρίβειας.

printf(format-control-string, other-arguments);

Το **format-control-string** περιγράφει τη μορφοποίηση εξόδων και το **άλλο-όρισμα** (το οποίο είναι προαιρετικό) αντιστοιχεί στη κάθε προδιαγραφή μετατροπής στο **format-control-string**. Η κάθε προδιαγραφή μετατροπής ξεκινά με το % και τελειώνει με το προσδιοριστικό της μετατροπής. Μπορούν να υπάρχουν πολλά προσδιοριστικά μετατροπής σε **format-control-string**.

Εκτύπωση ακεραίων αριθμών (integer)

Ένας ακέραιος αριθμός είναι ένας ολόκληρος αριθμός χωρίς δεκαδικό σημείο, όπως 776 και -52. Οι ακέραιες τιμές παρουσιάζονται με μια από τις ακόλουθες μορφοποιήσεις.

Προσδιοριστικό μετατροπής	**Περιγραφή**
d	παρουσιάζει ένα δεκαδικό ακέραιο με πρόσημο.
i	παρουσιάζει ένα δεκαδικό ακέραιο με πρόσημο. (Σημείωση: Το **d** και το **i** διαφέρουν όταν χρησιμοποιούνται με το **scanf**)
o	παρουσιάζει ένα οκταδικό ακέραιο χωρίς πρόσημο.
u	παρουσιάζει ένα δεκαδικό ακέραιο χωρίς πρόσημο.
x ή X	παρουσιάζει ένα δεκαεξαδικό ακέραιο χωρίς πρόσημο. Το **X** παρουσιάζει τα ψηφία **0-9** και τα γράμματα **A-Z** και το x παρουσιάζει τα ψηφία **0-9** και τα γράμματα **a-z.**
h ή l (γράμμα l)	τοποθετείται πριν από τον προσδιοριστή για να δείξει ότι ένας **short** ή **long** ακέραιος θα παρουσιαστεί αντίστοιχα.

Παράδειγμα 104

```
1.  /* Integer format */
2.  /* Using the integer conversion specifiers */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      printf( "%d\n", 455 );
8.      printf( "%i\n", 455 );  /* i same as d in printf */
9.      printf( "%d\n", +455 );
10.     printf( "%d\n", -455 );
11.     printf( "%hd\n", 32000 );
12.     printf( "%ld\n", 2000000000 );
13.     printf( "%o\n", 455 );
14.     printf( "%u\n", 455 );
15.     printf( "%u\n", -455 );
16.     printf( "%x\n", 455 );
17.     printf( "%X\n", 455 );
18.
19.     return 0;
20. }
```

ΕΞΟΔΟΣ:

```
455
455
455
-455
32000
2000000000
707
455
65081
1c7
1C7
```

Σημείωση: Προσέξτε ότι μόνο το αρνητικό πρόσημο εκτυπώνεται, το θετικό πρόσημο καταστέλλεται. Μετά θα δούμε πως τυπώνουμε το θετικό πρόσημο. Επίσης προσέξτε ότι η τιμή **-455** διαβάζεται από **%u** και μετατρέπεται στην τιμή χωρίς πρόσημο **65081**.

Εκτύπωση αριθμών κινητής υποδιαστολής (floating-point)
Μια τιμή κινητής διαστολής περιέχει δεκαδικό σημείο όπως **33.5** ή **657.983**.Αυτές οι τιμές μπορούν να τυπωθούν σε μια από διάφορες μορφοποιήσεις που μας παρέχει η **C**.

Οι προσδιοριστές μετατροπής **e** και **E** παρουσιάζουν τιμές κινητής υποδιαστολής σε εκθετική μορφή. Αυτή η μορφή είναι η ίδια με την επιστημονική μορφή που χρησιμοποιείται στα μαθηματικά. Για παράδειγμα ο αριθμός 150.4582 αντιπροσωπεύεται σε επιστημονική μορφή

$1.504582 \text{ X } 10^2$

και σε εκθετική μορφή

1.504582E+02

Οι αριθμοί που τυπώνονται με τους προσδιοριστές μετατροπής **e**, **E** και **f** έχουν από προεπιλογή ακρίβεια **6** ψηφίων στα δεξιά του δεκαδικού σημείου: άλλη ακρίβεια μπορεί να καθοριστεί ρητά. Ο προσδιοριστής μετατροπής **f** πάντα τυπώνει **τουλάχιστο ένα ψηφίο** στα αριστερά του δεκαδικού σημείου. Οι προσδιοριστές μετατροπής **e** και **E** τυπώνουν μικρό **e** και το κεφαλαίο **E** πριν τη δύναμη αντίστοιχα και πάντα τυπώνουν **ακριβώς ένα ψηφίο** στα αριστερά του δεκαδικού σημείου.

Οι προσδιοριστές μετατροπής **g** και **G** τυπώνουν είτε σε μορφή **e** (**E**) ή **f** χωρίς μηδενικά στο τέλος (**1.234000** θα τυπωθεί σαν **1.234**). Οι τιμές εκτυπώνονται με **e** (**E**) εάν μετά τη μετατροπή της τιμής σε εκθετική μορφή ο εκθέτης είναι μικρότερος από **-4** ή μεγαλύτερος ή ίσος με τη καθορισμένη ακρίβεια (**6** σημαντικά ψηφία). Διαφορετικά, χρησιμοποιείται ο προσδιοριστής μετατροπής **f** για εκτύπωση της τιμής.

Μερικά παραδείγματα με το **%g**

0.0000875	**8.7e-05 τυπώνεται με e γιατί ο εκθέτης είναι μικρότερος από -4**
8750000.0	**8.75e+06 τυπώνεται με e γιατί ο εκθέτης είναι ίσος με 6**
8.75	**8.75**
87.50	**87.5**
875	**875**

Αν χρησιμοποιούσαμε το **%G** η μόνη διαφορά θα ήταν στο ότι θα τυπωνόταν το **E** αντί **e**.

Το **%g** και το **%G** τυπώνουν ακριβώς 6 ψηφία συμπεριλαμβανομένου του ψηφίο που βρίσκεται αριστερά από το δεκαδικό σημείο, δηλαδή

Το **1234567.0** θα εκτυπωθεί σαν **1.23457e+06** ή **1.23457E+06**

Προσδιοριστικό μετατροπής	Περιγραφή
e ή E	παρουσιάζει μια τιμή κινητής υποδιαστολής σε εκθετική γραφή (επιστημονική μορφή).
f	μια τιμή κινητής υποδιαστολής.
g ή G	παρουσιάζει μια τιμή κινητής υποδιαστολής σε είτε μορφή **f** η σε εκθετική γραφή **e** ή **E**.
L	τοποθετείται πριν από τον προσδιοριστή για να δείξει ότι ένας **long double** κινητής υποδιαστολής θα παρουσιαστεί.

Παράδειγμα 105

```
1.  /* Example  */
2.  /* Printing floating-point numbers with
3.  floating-point conversion specifiers */
4.
5.  #include <stdio.h>
6.
7.  int main()
8.  {
9.      printf( "%e\n", 1234567.89 );
10.     printf( "%e\n", +1234567.89 );
11.     printf( "%e\n", -1234567.89 );
12.     printf( "%E\n", 1234567.89 );
13.     printf( "%f\n", 1234567.89 );
14.     printf( "%g\n", 1234567.89 );
15.     printf( "%G\n", 1234567.89 );
16.
17.     return 0;
18. }
```

ΕΞΟΔΟΣ:

```
1.234568e+006
1.234568e+006
-1.234568e+006
1.234568E+006
1234567.890000
1.23457e+006
1.23457E+006
```

Εκτύπωση γραμματοσειρών και χαρακτήρων

Οι προσδιοριστές μετατροπής **c** και **s** χρησιμοποιούνται για την εκτύπωση ατομικών χαρακτήρων και γραμματοσειρών, αντίστοιχα. Ο προσδιοριστής **c** απαιτεί ένα **char** σαν όρισμα. Ο προσδιοριστής **s** απαιτεί ένα **δείκτη σε char** σαν όρισμα. Ο προσδιοριστής **s** εκτυπώνει χαρακτήρες μέχρι να βρει το **null** (**'\0'**).

Παράδειγμα 106

```
1.  /* Example  */
2.  /* Printing strings and characters */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char character = 'A';
8.      char string[] = "This is a string";
9.      const char *stringPtr = "This is also a string";
10.
11.     printf( "%c\n", character );
12.     printf( "%s\n", "This is a string" );
13.     printf( "%s\n", string );
14.     printf( "%s\n", stringPtr );
15.
16.     return 0;
17. }
```

ΕΞΟΔΟΣ:

```
A
This is a string
This is a string
This is also a string
```

Άλλοι προσδιοριστές μετατροπής

Οι τρεις υπόλοιποι προσδιοριστές μετατροπής είναι **p**, **n** και **%**.

Προσδιοριστικό μετατροπής	Περιγραφή
p	παρουσιάζει την τιμή ενός δείκτη.
n	αποθηκεύει τον αριθμό των χαρακτήρων που έχουν εκτυπωθεί από το **printf**. Ένας δείκτης σε ένα ακέραιο προμηθεύεται σαν αντίστοιχο όρισμα. Τίποτα δεν παρουσιάζεται.
%	παρουσιάζει το χαρακτήρα %.

Παράδειγμα 107

```
1.  /* Example */
2.  /* Using the p, n, and % conversion specifiers */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int *ptr;
8.      int x = 12345, y;
9.
10.     ptr = &x;
11.     printf( "The value of ptr is %p\n", ptr );
12.     printf( "The address of x is %p\n\n", &x );
13.
14.     printf( "Total characters printed on this line is:%n", &y );
15.     printf( " %d\n\n", y );
16.
17.     y = printf( "This line has 28 characters\n" );
18.     printf( "%d characters were printed\n\n", y );
19.
20.     printf( "Printing a %% in a format control string\n" );
21.
22.     return 0;
23. }
```

ΕΞΟΔΟΣ:

```
The value of ptr is 0065FDF0
The address of x is 0065FDF0

Total characters printed on this line is: 41

This line has 28 characters
28 characters were printed

Printing a % in a format control string
```

Εκτύπωση πεδίων με μεγέθη και ακρίβεια

Παράδειγμα 108 (εκτύπωση ακεραίων)

```
1.  /* Example */
2.  /* Printing integers right-justified */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      printf( "%4d\n", 1 );
8.      printf( "%4d\n", 12 );
9.      printf( "%4d\n", 123 );
10.     printf( "%4d\n", 1234 );
11.     printf( "%4d\n\n", 12345 );
12.
13.     printf( "%4d\n", -1 );
14.     printf( "%4d\n", -12 );
15.     printf( "%4d\n", -123 );
16.     printf( "%4d\n", -1234 );
17.     printf( "%4d\n", -12345 );
18.
19.     return 0;
20. }
```

ΕΞΟΔΟΣ:

```
   1
  12
 123
1234
12345

  -1
 -12
-123
-1234
-12345
```

Σημείωση: Εάν το μέγεθος του πεδίου είναι μεγαλύτερο από τα δεδομένα που θα τυπωθούν, τα δεδομένα είναι right-justified.

Παράδειγμα 109 (εκτύπωση ακεραίων, αριθμών κινητής υποδιαστολής και γραμματοσειρών με ΑΚΡΙΒΕΙΑ)

```
1.  /* Example */
2.  /* Using precision while printing integers,
3.  floating-point numbers, and strings */
4.  #include <stdio.h>
5.
6.  int main()
7.  {
8.      int i = 873;
9.      double f = 123.94536;
10.     char s[] = "Happy Birthday";
11.
12.     printf( "Using precision for integers\n" );
13.     printf( "\t%.4d\n\t%.9d\n\n", i, i );
14.     printf( "Using precision for floating-point numbers\n" );
15.     printf( "\t%.3f\n\t%.3e\n\t%.3g\n\n", f, f, f );
16.     printf( "Using precision for strings\n" );
17.     printf( "\t%.11s\n", s );
18.
19.     return 0;
20. }
```

ΕΞΟΔΟΣ:

```
Using precision for integers
        0873
        000000873

Using precision for floating-point numbers

        123.945
        1.239e+02
        124

Using precision for strings

        Happy Birth
```

Σημείωση: Η ακρίβεια έχει διαφορετική έννοια για διαφορετικούς τύπους δεδομένων.

1. Όταν χρησιμοποιείται με ακέραιους (**d**) η ακρίβεια δεικνύει τον μικρότερο αριθμό ψηφίων που θα τυπωθούν. Εάν η τιμή που θα εκτυπωθεί περιέχει πιο λίγα ψηφία από τη καθορισμένη ακρίβεια, προσθέτονται μηδενικά στην αρχή του αριθμού για να συμπληρωθεί ο αριθμός της ακρίβειας. Η προεπιλεγμένη ακρίβεια για τους ακεραίους είναι το 1.
2. Όταν χρησιμοποιείται με αριθμούς κινητής υποδιαστολής (**e**, **E** και **f**) η ακρίβεια σημαίνει τον αριθμό των ψηφίων που εμφανίζονται μετά το δεκαδικό σημείο.
3. Όταν χρησιμοποιείται με τους προσδιοριστές μετατροπής **g** και **G** η ακρίβεια σημαίνει τον μεγαλύτερο αριθμό των σημαντικών ψηφίων που θα εκτυπωθούν.
4. Όταν χρησιμοποιείται με γραμματοσειρές (**s**) η ακρίβεια σημαίνει τον μεγαλύτερο αριθμό των χαρακτήρων που θα εκτυπωθούν.

Το μέγεθος του πεδίου μπορεί να χρησιμοποιηθεί μαζί με την ακρίβεια, όπως

printf(“%9.3f”, 123.456789);

θα τυπώσει ~~bb~~**123.457**

Είναι δυνατόν να χρησιμοποιήσετε παραστάσεις ακεραίων για το μέγεθος και την ακρίβεια, όπως

printf(“%*.*f, 7, 2, 98.736);

θα τυπώσει ~~bb~~**98.74**

αν θέλουμε left-justified χρησιμοποιούμε αρνητικούς αριθμούς στην παράσταση μας.

Παράδειγμα 110 (εκτύπωση με right-justification και left-justification)

```
1.  /* Example */
2.  /* Right justifying and left justifying values */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      printf( "%10s%10d%10c%10f\n\n", "hello", 7, 'a', 1.23 );
8.      printf( "%-10s%-10d%-10c%-10f\n", "hello", 7, 'a', 1.23 );
9.      return 0;
10. }
```

ΕΞΟΔΟΣ:

```
   hello         7         a   1.230000

hello      7         a         1.230000
```

Η χρήση σημαιών (flags) στο printf

Flag	Περιγραφή
- (πλην)	left-justified
+ (συν)	παρουσιάζει το + μπροστά από τους θετικούς αριθμούς και το – μπροστά από τους αρνητικούς.
κενό	τυπώνει κενό μπροστά από ένα θετικό αριθμό χωρίς το +
#	προσθέτει **0** μπροστά από ένα οκταδικό αριθμό (**o**) και τυπώνει **0x** ή **0X** μπροστά από δεκαεξαδικό αριθμό (**x** ή **X**)
0	προσθέτει το **0** μπροστά από ένα αριθμό

Παράδειγμα 111 (με flags)

```
1.  /* Example */
2.  /* Printing numbers with and without the + flag */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      printf( "%d\n%d\n", 786, -786 );
8.      printf( "%+d\n%+d\n", 786, -786 );
9.      return 0;
10. }
```

ΕΞΟΔΟΣ:

```
786
-786
+786
-786
```

Παράδειγμα 112 (με κενό μπροστά από θετικούς αριθμούς)

```
1.  /* Example */
2.  /* Printing a space before signed values
3.  not preceded by + or - */
4.  #include <stdio.h>
5.  int main()
6.  {
7.      printf( "% d\n% d\n", 547, -547 );
8.      return 0;
9.  }
```

ΕΞΟΔΟΣ:

```
 547
-547
```

Παράδειγμα 113 (με χρήση # και x ή X)

```
1.  /* Example */
2.  /* Using the # flag with conversion specifiers
3.  o, x, X and any floating-point specifier */
4.  #include <stdio.h>
5.  int main()
6.  {
7.      int c = 1427;
8.      double p = 1427.0;
9.
10.     printf( "%#o\n", c );
11.     printf( "%#x\n", c );
12.     printf( "%#X\n", c );
13.     printf( "\n%g\n", p );
14.     printf( "%#g\n", p );
15.
16.     return 0;
17. }
```

ΕΞΟΔΟΣ:

```
02623
0x593
0X593

1427
1427.00
```

Παράδειγμα 114 (συνδυασμός του + και του 0 για εκτύπωση του αριθμού 452 μέσα σε ένα πεδίου με μέγεθος 9

```
1.  /* Example */
2.  /* Printing with the 0( zero ) flag fills in leading zeros */
3.  #include <stdio.h>
4.  int main()
5.  {
6.      printf( "%+09d\n", 452 );
7.      printf( "%09d\n", 452 );
8.     return 0;
9.  }
```

ΕΞΟΔΟΣ:

```
+00000452
000000452
```

Εκτύπωση λεκτικών σταθερών (literals) και ακολουθίες διαφυγής (escape sequences)

Escape sequence	Περιγραφή
\’	τυπώνει το χαρακτήρα ’
\”	τυπώνει το χαρακτήρα ”
\?	τυπώνει το χαρακτήρα ?
\\	τυπώνει το χαρακτήρα \
\a	ακουστική ή οπτική ειδοποίηση
\b	μετακινεί το κέρσορα μία θέση πίσω στην τρέχουσα γραμμή
\f	μετακινεί το κέρσορα στην αρχή νέας σελίδας
\n	μετακινεί το κέρσορα στην αρχή νέας γραμμής
\r	μετακινεί το κέρσορα στην αρχή της τρέχουσας γραμμής
\t	μετακινεί το κέρσορα στην επόμενη θέση του οριζοντίου στηλοθέτη
\v	μετακινεί το κέρσορα στην επόμενη θέση του κάθετου στηλοθέτη

Μορφοποίηση εισόδων με την εντολή scanf

Η ακριβής μορφοποίηση εισόδων επιτυγχάνεται με την εντολή scanf. Κάθε εντολή scanf περιλαμβάνει μια γραμματοσειρά ελέγχου μορφοποίησης που περιγράφει την μορφοποίηση των δεδομένων που θα εισαχθούν. Η γραμματοσειρά ελέγχου μορφοποίησης περιέχει προσδιοριστικά μετατροπής και λεκτικές σταθερές.

Η συνάρτηση scanf έχει τις ακόλουθες δυνατότητες μορφοποίησης εισόδων:

1. Εισαγωγή όλων των τύπων δεδομένων.
2. Εισαγωγή συγκεκριμένων χαρακτήρων σε ένα ρεύμα εισόδου.
3. Παράκαμψη συγκεκριμένων χαρακτήρων σε ένα ρεύμα εισόδου.

Προσδιοριστικό μετατροπής	Περιγραφή
Ακέραιοι (integers)	
d	διαβάζει ένα δεκαδικό ακέραιο με προαιρετικό πρόσημο. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ακέραιο.
i	διαβάζει ένα δεκαδικό, οκταδικό ή δεκαεξαδικό ακέραιο με προαιρετικό πρόσημο. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ένα ακέραιο.
o	διαβάζει έναν οκταδικό ακέραιο. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ακέραιο χωρίς πρόσημο.
u	διαβάζει ένα δεκαδικό ακέραιο χωρίς πρόσημο. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ακέραιο χωρίς πρόσημο.
x ή X	διαβάζει ένα δεκαεξαδικό ακέραιο. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ακέραιο χωρίς πρόσημο.
h ή l	μπαίνει μπροστά από οποιοδήποτε προσδιοριστικό μετατροπής ακεραίου για να δείξει ότι ένας **short** ή **long** ακέραιος θα διαβαστεί.
Αριθμοί κινητής υποδιαστολής (floating-point)	
e, E, f, g ή G	διαβάζει ένα αριθμό κινητής υποδιαστολής. Το αντίστοιχο όρισμα είναι ένας δείκτης σε μεταβλητή κινητής υποδιαστολής.
l ή L	μπαίνει μπροστά από οποιοδήποτε προσδιοριστικό μετατροπής αριθμού κινητής υποδιαστολής για να δείξει ότι ένας **double** ή **long double** τιμή θα διαβαστεί.
Χαρακτήρες και γραμματοσειρές	
c	διαβάζει ένα χαρακτήρα. Το αντίστοιχο όρισμα είναι ένας δείκτης σε χαρακτήρα, δεν προσθέτεται το null (‘\0’).
s	διαβάζει μια γραμματοσειρά. Το αντίστοιχο όρισμα είναι ένας δείκτης σε πίνακα χαρακτήρων αρκετά μεγάλο για να δεχτεί τη γραμματοσειρά και το null (‘\0’) το οποίο προσθέτεται αυτόματα.

Σύνολο σάρωσης (scan set)

[scan characters	ανιχνεύει μια γραμματοσειρά για ένα σύνολο χαρακτήρων που βρίσκονται στον πίνακα.

Διάφορα

p	διαβάζει μια διεύθυνση της ιδίας μορφοποίησης που παράγεται όταν μια διεύθυνση εκτυπώνεται με το **%p** από μια εντολή **printf**.
n	αποθηκεύει τον αριθμό των χαρακτήρων που διαβαστήκαν μέχρι στιγμής με την εντολή **scanf**. Το αντίστοιχο όρισμα είναι ένας δείκτης σε ακέραιο.
%	προσπερνάει το σύμβολο ποσοστού.

Παράδειγμα 115

```
1.  /* Example */
2.  /* Reading integers */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int a, b, c, d, e, f, g;
8.
9.      printf( "Enter seven integers: " );
10.     scanf( "%d%i%i%i%o%u%x", &a, &b, &c, &d, &e, &f, &g );
11.     printf( "The input displayed as decimal integers is:\n" );
12.     printf( "%d %d %d %d %d %d %d\n", a, b, c, d, e, f, g );
13.
14.     return 0;
15. }
```

ΕΞΟΔΟΣ:

```
Enter 7 integers: -70 -70 070 0x70 70 70 70
The input displayed as decimal integers is:
-70 -70 56 112 56 70 112
```

Παράδειγμα 116

```
1.  /* Example */
2.  /* Reading floating-point numbers */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      double a, b, c;
8.
9.      printf( "Enter three floating-point numbers: \n" );
10.     scanf( "%le%lf%lg", &a, &b, &c );
11.     printf( "Here are the numbers entered in plain\n" );
12.     printf( "floating-point notation:\n" );
13.     printf( "%f\n%f\n%f\n", a, b, c );
14.
15.     return 0;
16. }
```

ΕΞΟΔΟΣ:

```
Enter three floating-point numbers:
1.27987 1.27987e+03 3.38476e-06
Here are the numbers entered in plain
floating-point notation:
1.279870
1279.870000
0.000003
```

Παράδειγμα 117

```
1.  /* Example */
2.  /* Reading characters and strings */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char x, y[ 9 ];
8.
9.      printf( "Enter a string: " );
10.     scanf( "%c%s", &x, y );
11.
12.     printf( "The input was:\n" );
13.     printf( "the character \"%c\" ", x );
14.     printf( "and the string \"%s\"\n", y );
15.
16.     return 0;
17. }
```

ΕΞΟΔΟΣ:

```
Enter a string: Sunday
The input was:
the character “S” and the string “unday”
```

Παράδειγμα 118

```
1.  /* Example */
2.  /* Using a scan set */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char z[ 9 ];
8.
9.      printf( "Enter string: " );
10.     scanf( "%[aeiou]", z );
11.     printf( "The input was \"%s\"\n", z );
12.
13.     return 0;
14. }
```

ΕΞΟΔΟΣ:

Enter String: ooeeooahah
The input was “ooeeooa”

Παράδειγμα 119

```
1.  /* Example */
2.  /* Using an inverted scan set */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      char z[ 9 ] = { '\0' };
8.
9.      printf( "Enter a string: " );
10.     scanf( "%[^aeiou]", z );
11.     printf( "The input was \"%s\"\n", z );
12.
13.     return 0;
14. }
```

ΕΞΟΔΟΣ:

Enter a string: String
The input was “Str”

Παράδειγμα 120

```
1.  /* Example */
2.  /* inputting data with a field width */
3.  #include <stdio.h>
4.  int main()
5.  {
6.      int x, y;
7.      printf( "Enter a six digit integer: " );
8.      scanf( "%2d%d", &x, &y );
9.      printf( "The integers input were %d and %d\n", x, y );
10.     return 0;
11. }
```

ΕΞΟΔΟΣ:

```
Enter a six digit integer: 123456
The integers input were 12 and 3456
```

Παράδειγμα 121

```
1.  /* Example */
2.  /* Reading and discarding characters from the input stream */
3.  #include <stdio.h>
4.  int main()
5.  {
6.      int month1, day1, year1, month2, day2, year2;
7.      printf( "Enter a date in the form mm-dd-yyyy: " );
8.      scanf( "%d%*c%d%*c%d", &month1, &day1, &year1 );
9.      printf( "month = %d  day = %d  year = %d\n\n",
10.         month1, day1, year1 );
11.     printf( "Enter a date in the form mm/dd/yyyy: " );
12.     scanf( "%d%*c%d%*c%d", &month2, &day2, &year2 );
13.     printf( "month = %d  day = %d  year = %d\n",
14.         month2, day2, year2 );
15.     return 0;
16. }
```

ΕΞΟΔΟΣ:

```
Enter a date in the form mm-dd-yyyy: 11-18-2000
month = 11 day = 18 year = 2000

Enter a date in the form mm/dd/yyyy: 11/18/2000
month = 11 day = 18 year = 2000
```

Επεξεργασία Αρχείων

Η αποθήκευση δεδομένων σε μεταβλητές και πίνακες είναι προσωρινή – αυτά τα δεδομένα χάνονται όταν το πρόγραμμα τελειώνει.

Τα αρχεία χρησιμοποιούνται για μόνιμη διατήρηση μεγάλων ποσοτήτων δεδομένων. Οι Η/Υ αποθηκεύουν αρχεία σε βοηθητικές συσκευές αποθήκευσης ειδικά σε δίσκους.

Σε αυτή την ενότητα θα εξηγήσουμε πως μπορούμε να δημιουργήσουμε, να ενημερώνουμε και να επεξεργαζόμαστε αρχεία δεδομένων με τη **C**. Θα δούμε αρχεία διαδοχικής πρόσβασης (sequential-access) και τυχαίας πρόσβασης (random-access).

Ιεραρχία δεδομένων

Το μικρότερο είδος δεδομένων που μπορεί ο ηλεκτρονικός υπολογιστής να διαχειριστεί είναι το **bit** (**bi**nary digi**t**), ένα ψηφίο που μπορεί να πάρει μια εκ των δύο τιμών, **1** ή **0**. Ένα κύκλωμα Η/Υ εκτελεί διάφορους απλούς χειρισμούς των bit όπως να εξετάσει την τιμή του, να δώσει τιμή σε ένα bit ή να την αλλάξει από 0 σε 1 ή και αντίθετα. Επειδή ο προγραμματισμός με δεδομένα χαμηλού επιπέδου μορφής είναι πολύ δύσκολος είναι προτιμότερο να προγραμματίζομαι με δεδομένα με μορφή όπως δεκαδικά ψηφία (0..9) γράμματα (A..Z, a..z) και ειδικούς χαρακτήρες ($, @, %, &, *, κ.τ.λ.). Αυτά όλα είναι γνωστά σαν χαρακτήρες. Το σύνολο αυτών των χαρακτήρων που τα χρησιμοποιούμε να γράφουμε προγράμματα και να αντιπροσωπεύουμε δεδομένα είναι γνωστά σαν character set (ASCII) του Η/Υ. Επειδή ο Η/Υ όπως προαναφέραμε μπορεί να επεξεργαστεί μόνο μοτίβο από 1'ς και 0'ς όλοι οι χαρακτήρες είναι μια σειρά από 8-bits που μας δίνει 1 byte. Ένα record (εγγραφή) αποτελείται από πολλά σχετιζόμενα fields και πολλά records μαζί μας δημιουργούν ένα file(αρχείο). Πολλά σχετιζόμενα files μας δίνουν ένα Data base.

Παράδειγμα record

Record υπαλλήλου

1. Αριθμός υπαλλήλου (long int)
2. Όνομα υπαλλήλου (string)
3. Διεύθυνση (string)
4. Αμοιβή ανά ώρα (float)
5. Αριθμός φόρο-απαλλαγών (int)
6. Έσοδα μέσα στο χρόνο (Double)
7. Φόρος που παρακρατήθηκε (Double)

Το αρχείο μισθοδοσίας μιας εταιρείας συνήθως περιέχει μιαν εγγραφή (record) για κάθε υπάλληλο. Για τη διευκόλυνση ανάκτησης συγκεκριμένων εγγραφών από ένα αρχείο ένα πεδίο (field) πρέπει να επιλεγεί σαν κλειδί εγγραφής (record key). Αυτό το κλειδί προσδιορίζει μια εγγραφή, στο παράδειγμα μας, για κάποιο συγκεκριμένο υπάλληλο. Το

πιο κατάλληλο πεδίο στο πιο πάνω παράδειγμα που μπορεί να είναι κλειδί εγγραφής είναι ο αριθμός του υπαλλήλου που είναι μοναδικός για κάθε υπάλληλο.

Υπάρχουν πολλοί διαφορετικοί τρόποι για την οργάνωση εγγραφών μέσα σε ένα αρχείο. Ένας πολύ γνωστός τρόπος είναι το sequential file, όπου οι εγγραφές φυλάγονται σε σειρά με κάποιο πεδίο (field) σαν κλειδί εγγραφής (record key).

Αρχεία και Ρέματα (Files and Streams)

Η C βλέπει κάθε αρχείο σαν μια ροή από bytes. Κάθε αρχείο τελειώνει είτε με ένα end-of-file marker ή με κάποιο μοναδικό byte που προσδιορίζεται από το σύστημα. Όταν ένα αρχείο ανοίξει ένα αντικείμενο δημιουργείται μαζί με κάποιο ρέμα (stream) που συσχετίζεται με αυτό το αντικείμενο. Τρία αρχεία και οι συναφείς ροές αυτόματα ανοίγουν όταν ξεκινά η εκτέλεση του προγράμματος – standard input, standard output και το standard error – οι ροές μας παρέχουν κανάλια επικοινωνίας μεταξύ ενός προγράμματος και κάποιου συγκεκριμένου αρχείου ή συσκευής.

Για να μπορούμε να εκτελούμε επεξεργασία αρχείων με τη C πρέπει να συμπεριλάβουμε μέσα στα προγράμματα μας το αρχείο κεφαλίδας <stdio.h>.

Η πρότυπη βιβλιοθήκη μας παρέχει πολλές συναρτήσεις για να διαβάζουμε και να γράφουμε δεδομένα σε αρχεία.

- Η συνάρτηση **fgetc**, όπως η **getchar**, διαβάζει ένα χαρακτήρα από ένα αρχείο. Η συνάρτηση **fgetc** παίρνει σαν όρισμα ένα **δείκτη** σε **FILE** για το αρχείο από όπου θα διαβάσουμε το χαρακτήρα. Η κλήση **fgetc (stdin)** διαβάζει ένα χαρακτήρα από το standard input (πληκτρολόγιο). Αυτό είναι το ίδιο με τη κλήση **getchar()**.
- Η συνάρτηση **fputc**, όπως η **putchar**, γράφει ένα χαρακτήρα σε ένα αρχείο. Η συνάρτηση **fputc**, παίρνει ένα χαρακτήρα για να γραφτεί και ένα **δείκτη** σε **FILE** στο οποίο ο χαρακτήρας θα γραφτεί. Η κλήση **fputc('a', stdout)** γράφει το **'a'** στο standard output (οθόνη). Αυτή η κλήση είναι η ίδια με τη κλήση **putchar('a')**.
- Επίσης μπορούμε να χρησιμοποιήσουμε τις συναρτήσεις **fgets** και **fputs** για να διαβάσουμε ή γράψουμε μια γραμμή κειμένου από και σε ένα αρχείο αντίστοιχα.

Δημιουργία Sequential file

Η **C** δεν επιβάλλει καμία δομή σε ένα αρχείο. Ως εκ τούτου, έννοιες όπως μια εγγραφή του αρχείου δεν υφίσταται σαν μέρος της **C**. Έτσι, ο προγραμματιστής πρέπει να παρέχει κάποια δομή αρχείου για να ικανοποιήσει τις απαιτήσεις για κάθε συγκεκριμένη εφαρμογή.

Το πιο κάτω πρόγραμμα δημιουργεί ένα απλό sequential-access αρχείο που μπορεί να χρησιμοποιηθεί για ένα σύστημα για να βοηθήσει μια εταιρεία να διατηρήσει ποσά που οφείλονται από πιστωτικούς πελάτες.

Για κάθε πελάτη το πρόγραμμα δέχεται ένα αριθμό λογαριασμού,το όνομα του πελάτη και το υπόλοιπο.

Τα δεδομένα που δίνονται για κάθε πελάτη αποτελούν μια εγγραφή (record) για τον πελάτη.

Ο αριθμός του λογαριασμού χρησιμοποιήται σαν κλειδί (key) σε αυτή την εφαρμογή. Το αρχείο θα δημιουργηθεί και να διατηρηθεί με μια σειρά με βάση τον αριθμό λογαριασμού.

Αυτό το πρόγραμμα θεωρεί ότι ο χρήστης θα περάσει τους πελάτες με σειρά βάση του αριθμού του λογαριασμού του κάθε ενός.

Σε ένα πιο σαφή σύστημα θα πρέπει να περιέχεται η δυνατότητα ταξινόμησης των δεδομένων έτσι ώστε ο χρήστης να μην χρειάζεται να βάλει τους λογαριασμούς με σειρά.

Παράδειγμα 122

```
1.  /* Example
2.  Create a sequential file */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int account;
8.      char name[ 30 ];
9.      double balance;
10.     FILE *cfPtr;   /* cfPtr = clients.dat file pointer */
11.
12.     if ( ( cfPtr = fopen( "clients.dat", "w" ) ) == NULL )
13.         printf( "File could not be opened\n" );
14.     else {
15.         printf( "Enter the account, name, and balance.\n" );
16.         printf( "Enter EOF to end input.\n" );
17.         printf( "? " );
18.         scanf( "%d%s%lf", &account, name, &balance );
19.
20.         while ( !feof( stdin ) ) {
21.             fprintf( cfPtr, "%d %s %.2f\n",  account, name, balance );
22.             printf( "? " );
23.             scanf( "%d%s%lf", &account, name, &balance );
24.         }
25.
26.         fclose( cfPtr );
27.     }
28.
29.     return 0;
30. }
```

ΕΞΟΔΟΣ:

```
Enter the account, name, and balance.
Enter EOF to end input,
? 100 Jones 24.98
? 200 Doe 345.67
? 300 White 0.00
? 400 Stone -42.16
? 500 Rich 224.62
```

Η ΓΡΑΜΜΗ 10 λέει ότι το **cfptr** είναι ένας δείκτης σε αρχείο. Κάθε ένα αρχείο στο πρόγραμμα πρέπει να έχει ένα ξεχωριστό δείκτη.

Η ΓΡΑΜΜΗ 12 - **if ((cfPtr = fopen("clients.dat", "w")) == NULL)** – ονόμαζει το αρχείο - **"clients.dat"** – για να χρησιμοποιηθεί από το πρόγραμμα και δημιουργεί μια γραμμή επικοινωνίας με το αρχείο.

Στο δείκτη **cfptr** δίνεται ένας δείκτης σε αρχείο που ανοίγεται με το **fopen**. Η συνάρτηση **fopen** παίρνει δύο ορίσματα, το όνομα του αρχειου και τη λειτουργία του αρχείου. Το **"w"** δηλώνει ότι το αρχείο ανοίγει για γράψιμο δεδομένων.

Εάν το αρχείο δεν υφίσταται η **fopen** το δημιουργεί. Εάν το αρχείο υπάρχει τότε τα δεδομένα του χάνονται χωρίς καμιά προειδοποίηση.

Η εντολή **if** χρησιμοποιείται για να καθορίσει αν ο δείκτης δείχνει σε τιμή **NULL**, που αυτό σημαίνει ότι το αρχείο δεν άνοιξε και παίρνουμε ένα μήνυμα λάθους, διαφορετικά η επεξεργασία εισόδου δεδομένων συνεχίζεται και αυτά γράφονται στο αρχείο.

Το πρόγραμμα καθοδηγεί το χρήστη να δώσει τα δεδομένα ή το end-of-file για να το τερματίσει.

Η ΓΡΑΜΜΗ 20 - **while (!feof(stdin))** – χρησιμοποιεί την συνάρτηση **feof** για να καθορίσει εάν το end-of-file έχει οριστεί για το αρχείο που εδώ είναι το **stdin** (πληκτρολόγιο). Το end-of-file πληροφορεί το πρόγραμμα ότι δεν υπάρχουν άλλα δεδομένα για επεξεργασία.

Η ΓΡΑΜΜΗ 21 – **fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);** - γράφει τα δεδομένα στο αρχείο **"clients.dat"**. Η συνάρτηση **fprintf** είναι η ίδια με την **printf** εκτός στο ότι η **fprintf** παίρνει ακόμα ένα όρισμα που είναι ο δείκτης στο αρχείο που θα γραφτούν τα δεδομένα.

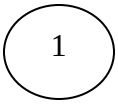
Η ΓΡΑΜΜΗ 26 – **fclose(cfPtr);** - κλείνει το αρχείο. Όπως έχετε προσέξει το όρισμα που παίρνει αυτή η συνάρτηση είναι επίσης ο δείκτης στο αρχείο αντί το ίδιο το αρχείο. Εάν δεν κλείσουμε το αρχείο ρητά μόνοι μας το λειτουργικό σύστημα θα το κάνει για μας. Αυτό είναι ένα παράδειγμα του "housekeeping" του συστήματος. Καλή πρακτική του προγραμματισμού είναι να κλείνουμε τα αρχεία μας ρητά.

Τέλος αρχείου

Σύστημα	**Συνδυασμός πληκτρολογίου**
UNIX/LINUX/MAC	<ctrl-d>
Microsoft windows	<ctrl-z> (κάποτε μπορεί να χρειάζεται το enter)
VAX/VMS	<ctrl-z>
Macintosh	<ctrl-d>

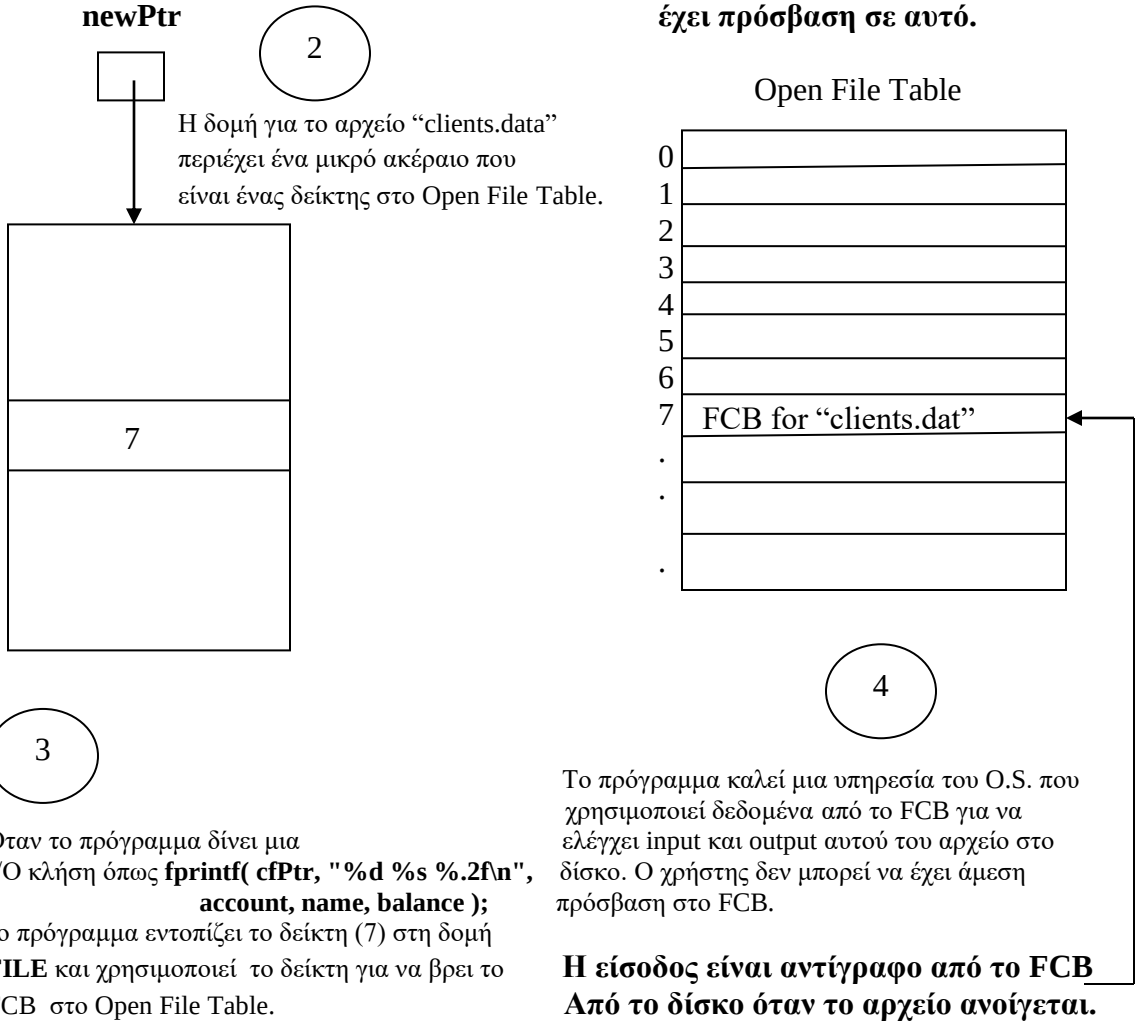
<u>Το ακόλουθο σχεδιάγραμμα παρουσιάζει την σχέση μεταξύ των δεικτών σε αρχεία, τη δομή του αρχείου και των FCB στη μνήμη</u>

Πρώτα από όλα ας εξηγήσουμε τι είναι το FCB (File Control Block); Είναι μια δομή συστήματος αρχείου μέσα στην οποία διατηρείται η κατάσταση ενός ανοιχτού αρχείου. Το διαχειρίζετε το λειτουργικό σύστημα αλλά βρίσκεται στο χώρο μνήμης του προγράμματος που το χρησιμοποιεί και όχι στο χώρο μνήμης του λειτουργικού συστήματος. Αυτό επιτρέπει σε ένα πρόγραμμα να έχει όσα αρχεία θέλει, νοουμένου ότι έχει αρκετή μνήμη για κάθε FCB του κάθε αρχείου.



cfPtr = fopen("clients.dat", "w"));
Η εντολή **fopen** επιστρέφει ένα δείκτη στη δομή του **FILE** που είναι ορισμένος μέσα στη **<stdio.h>**

Μόνο το λειτουργικό σύστημα έχει πρόσβαση σε αυτό.



Όταν το πρόγραμμα δίνει μια I/O κλήση όπως **fprintf(cfPtr, "%d %s %.2f\n", account, name, balance);** το πρόγραμμα εντοπίζει το δείκτη (7) στη δομή **FILE** και χρησιμοποιεί το δείκτη για να βρει το FCB στο Open File Table.

Το πρόγραμμα καλεί μια υπηρεσία του O.S. που χρησιμοποιεί δεδομένα από το FCB για να ελέγχει input και output αυτού του αρχείο στο δίσκο. Ο χρήστης δεν μπορεί να έχει άμεση πρόσβαση στο FCB.

Η είσοδος είναι αντίγραφο από το FCB Από το δίσκο όταν το αρχείο ανοίγεται.

Πως διαβάζουμε από ένα Sequential-access αρχείο

Τα δεδομένα αποθηκεύονται σε αρχεία έτσι ώστε να μπορούν να ανακτηθούν για επεξεργασία. Το παράδειγμα που ακολουθεί διαβάζει εγγραφές (records) από το αρχείο **"clients.dat"** που δημιουργήθηκε από το προηγούμενο πρόγραμμα που έχουμε δει.

Παράδειγμα 123

```
1.  /* Example
2.  Reading and printing a sequential file */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int account;
8.      char name[ 30 ];
9.      double balance;
10.     FILE *cfPtr;   /* cfPtr = clients.dat file pointer */
11.
12.     if ( ( cfPtr = fopen( "clients.dat", "r" ) ) == NULL )
13.         printf( "File could not be opened\n" );
14.     else {
15.         printf( "%-10s%-13s%s\n", "Account", "Name", "Balance" );
16.         fscanf( cfPtr, "%d%s%lf", &account, name, &balance );
17.
18.         while ( !feof( cfPtr ) ) {
19.             printf( "%-10d%-13s%7.2f\n", account, name, balance );
20.             fscanf( cfPtr, "%d%s%lf", &account, name, &balance );
21.         }
22.
23.         fclose( cfPtr );
24.     }
25.
26.     return 0;
27. }
```

ΕΞΟΔΟΣ:

```
Account     Name          Balance
100         Jones           24.98
200         Doe            345.67
300         White            0.00
400         Stone          -42.16
500         Rich           224.62
```

Η ΓΡΑΜΜΗ 10 δηλώνει ότι το **cfPtr** είναι ένας δείκτης σε **FILE**.

Η ΓΡΑΜΜΗ 12 προσπαθεί να ανοίξει το αρχείο **"clients.dat"** για να διαβάσουμε δεδομένα (**"r"**) και καθορίζει αν το αρχείο άνοιξε επιτυχώς (το **fopen** δεν επιστρέφει την τιμή **NULL**).

Η ΓΡΑΜΜΗ 16 διαβάζει μια εγγραφή από το αρχείο. Η εντολή **fscanf** είναι η ίδια με την εντολή **scanf** με διαφορά ότι η **fscanf** παίρνει σαν όρισμα ένα δείκτη στο αρχείο από όπου θέλουμε να διαβάσουμε δεδομένα. Μετά την εκτέλεση την πρώτη φορά αυτής της εντολής η μεταβλητή **account** θα πάρει την τιμή **100** η **name** θα πάρει την τιμή **Jones** και η **balance** την τιμή **24.98**. Κάθε φορά που το δεύτερο **fscanf** εκτελείται διαβάζει νέα εγγραφή μέχρι να διαβάσει το **end of file** όπου το αρχείο κλείνει και σταματά η εκτέλεση του προγράμματος.

Παράδειγμα 124

```
1.  /* Example
2.  Credit inquiry program */
3.  #include <stdio.h>
4.
5.  int main()
6.  {
7.      int request, account;
8.      double balance;
9.      char name[ 30 ];
10.     FILE *cfPtr;
11.
12.     if ( ( cfPtr = fopen( "clients.dat", "r" ) ) == NULL )
13.         printf( "File could not be opened\n" );
14.     else {
15.         printf( "Enter request\n"
16.                 " 1 - List accounts with zero balances\n"
17.                 " 2 - List accounts with credit balances\n"
18.                 " 3 - List accounts with debit balances\n"
19.                 " 4 - End of run\n? " );
20.         scanf( "%d", &request );
21.
22.         while ( request != 4 ) {
23.             fscanf( cfPtr, "%d%s%lf", &account, name,
24.                                             &balance );
25.
26.             switch ( request ) {
27.                 case 1:
28.                     printf( "\nAccounts with zero "
29.                                             "balances:\n" );
30.
31.                     while ( !feof( cfPtr ) ) {
32.
33.                         if ( balance == 0 )
34.                             printf( "%-10d%-13s%7.2f\n",
35.                                     account, name, balance );
36.
37.                         fscanf( cfPtr, "%d%s%lf",
38.                                     &account, name, &balance );
39.                     }
40.
41.                     break;
42.                 case 2:
43.                     printf( "\nAccounts with credit "
44.                                             "balances:\n" );
```

```
45.
46.                     while ( !feof( cfPtr ) ) {
47.
48.                         if ( balance < 0 )
49.                             printf( "%-10d%-13s%7.2f\n",
50.                                                 account, name, balance );
51.
52.                         fscanf( cfPtr, "%d%s%lf",
53.                                                 &account, name, &balance );
54.                     }
55.
56.                     break;
57.                 case 3:
58.                     printf( "\nAccounts with debit "
59.                                                 "balances:\n" );
60.
61.                     while ( !feof( cfPtr ) ) {
62.
63.                         if ( balance > 0 )
64.                             printf( "%-10d%-13s%7.2f\n",
65.                                                 account, name, balance );
66.
67.                         fscanf( cfPtr, "%d%s%lf",
68.                                                 &account, name, &balance );
69.                     }
70.
71.                     break;
72.             }
73.
74.             rewind( cfPtr );
75.             printf( "\n? " );
76.             scanf( "%d", &request );
77.         }
78.
79.         printf( "End of run.\n" );
80.         fclose( cfPtr );
81.     }
82.
83.     return 0;
84. }
```

ΕΞΟΔΟΣ:

```
Enter request
  1 – List accounts with zero balances
  2 – List accounts with credit balances
  3 – List accounts with debit balances
  4 – End of run
? 1

Accounts with zero balances:
300          White          0.00

? 2

Accounts with credit balances:
400          Stone          -42.16

? 3

Accounts with debit balances:
100          Jones           24.98
200          Doe            345.67
500          Rich           224.62
? 4
End of run.
```

Για να ανακτήσουμε δεδομένα **sequentially** από ένα αρχείο, το πρόγραμμα κανονικά ξεκινά από την αρχή του αρχείου και διαβάζει όλα τα δεδομένα διαδοχικά μέχρι να βρούμε τα επιθυμητά δεδομένα.

Υπάρχουν περιπτώσεις που θα θέλαμε να επεξεργαστούμε τα δεδομένα **sequentially** από ένα αρχείο αρκετές φορές (από την αρχή του αρχείου) κατά την εκτέλεση ενός προγράμματος. Η εντολή **rewind (cfPtr);** στη ΓΡΑΜΜΗ 74 είναι η αιτία το **file position pointer** (ο δείκτης που δείχνει στον αριθμό του επόμενου byte του αρχείου που θα διαβαστεί) να μετατοπιστεί στην αρχή του αρχείου (byte 0). Το **file position pointer** δεν είναι στην πραγματικότητα δείκτης αλλά ένας ακέραιος αριθμός που καθορίζει τη τοποθεσία του επόμενου byte μέσα στο αρχείο που θα διαβαστεί ή γραφτεί. Αυτό κάποτε λέγεται **file offset**.

Το παράδειγμα μας επιτρέπει σε κάποιο να πάρει μια λίστα πελατών που το υπόλοιπο τους είναι 0, που το υπόλοιπο τους είναι πλην, και που το υπόλοιπο τους είναι συν. Το πρόγραμμα εμφανίζει ένα μενού που μας επιτρέπει να δώσουμε μία από τις τρεις επιλογές που έχουμε, ανάλογα με τι θέλουμε να κάνουμε.

Σημειώστε ότι τα δεδομένα αυτού του **sequential** αρχείου δεν μετατρέπονται χωρίς τον κίνδυνο να καταστρέψουμε άλλα δεδομένα στο αρχείο. Για παράδειγμα, αν θέλουμε να αλλάξουμε το όνομα **White** σε **Worthington** αυτό μπορεί να δημιουργήσει προβλήματα με τα δεδομένα που ακολουθούν αυτή την εγγραφή για το λόγο ότι το νέο όνομα είναι πιο μεγάλο από το παλιό.

Το πρόβλημα εστιάζεται στο ότι μεσα στο μοντέλο μορφοποίησης για είσοδο/έξοδο όπου χρησιμοποιούμε τις εντολές **fprintf** και **fscanf** οι εγγραφές μπορεί να διαφέρουν σε μέγεθος. Για παράδειγμα, 7, 14, -117, 2074 και 27383 είναι όλα **int** αποθηκευμένα στον ίδιο αριθμό bytes αλλά τυπώνονται ή αποθηκεύονται στο δίσκο σαν διαφορετικού μεγέθους πεδία.

Ως εκ τούτου, αρχεία με πρόσβαση **sequential** με τις εντολές **fprintf** και **fscanf** συνήθως δεν χρησιμοποιούνται για μετατροπή εγγραφών.

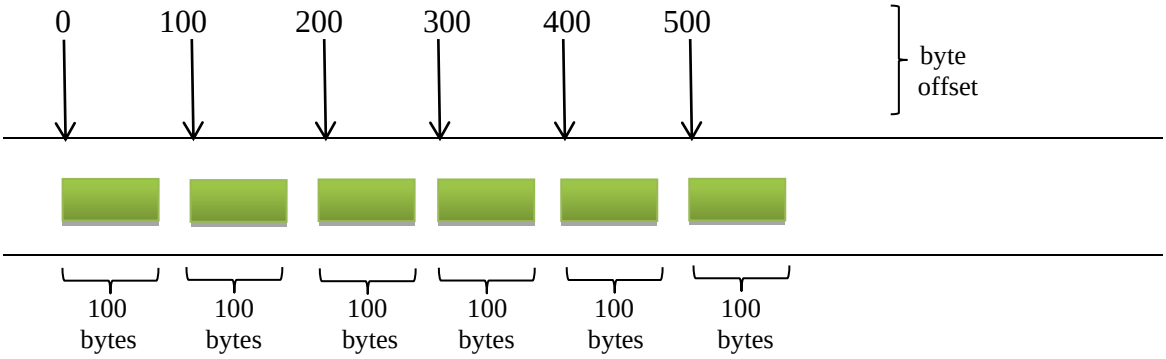
Το πιο πάνω πρόβλημα λύεται με προγραμματισμό αντιγράφοντας εγγραφές σε κάποιο άλλο αρχείο.

Αρχεία Τυχαίας Προσπέλασης (Random Access Files)

Όπως έχουμε δηλώσει προηγουμένως, οι εγγραφές σε ένα αρχείο που δημιουργούνται με την εντολή **fprintf** δεν είναι αναγκαία του ιδίου μεγέθους. Εντούτοις, οι εγγραφές ενός αρχείου **τυχαίας προσπέλασης** είναι κανονικά του ιδίου σταθερού μεγέθους και μπορούμε να έχουμε άμεση πρόσβαση (direct access) πολύ γρήγορα σε αυτά, χωρίς να τα αναζητήσουμε μέσω άλλων εγγραφών. Αυτό, κάνει αυτού του είδους αρχεία κατάλληλα για συστήματα όπως, airline reservation, banking, point-of-sale και άλλα διάφορα συστήματα που χρειάζονται γρήγορη πρόσβαση σε συγκεκριμένα δεδομένα. Υπάρχουν και άλλοι τρόποι εφαρμογής αρχείων τυχαίας προσπέλασης αλλά για αυτό το μάθημα θα μείνουμε μέχρι αυτή την προσέγγιση.

Χρησιμοποιώντας εγγραφές του ιδίου σταθερού μεγέθους ευκολύνουμε ένα πρόγραμμα να υπολογίζει την ακριβή θέση μέσα στο αρχείο οποιασδήποτε εγγραφής σε σχέση με την αρχή του αρχείου (ξέροντας το μέγεθος της εγγραφής και την εγγραφή κλειδί).

Ας δούμε το ακόλουθο σχεδιάγραμμα για να καταλάβουμε πως οι εγγραφές (π.χ. εγγραφές μεγέθους 100 bytes) συνθέτουν ένα αρχείο **random-access (τυχαίας πρόσβασης)**.



- Δεδομένα μπορούν να εισαχθούν στο αρχείο χωρίς να καταστρέφονται άλλα δεδομένα μέσα στο αρχείο.
- Δεδομένα που αποθηκεύτηκαν από πριν μπορούν να τροποποιηθούν ή να διαγραφούν χωρίς να ξαναγράψουμε ολόκληρο το αρχείο.

Δημιουργία αρχείου random-access (τυχαίας πρόσβασης)

Τώρα θα δούμε πως δημιουργούμε ένα αρχείο τυχαίας πρόσβασης, πως βάζουμε δεδομένα σε αυτό, πως διαβάζουμε δεδομένα (sequentially ή randomly), ενημερώνουμε (update) το αρχείο και πως διαγράφουμε δεδομένα που δεν χρειαζόμαστε από αυτό.

Η συνάρτηση **fwrite** μεταφέρει ένα καθορισμένο αριθμό bytes, ξεκινώντας από κάποια συγκεκριμένη θέση της μνήμης στο αρχείο. Τα δεδομένα γράφονται στην θέση του αρχείου που υποδεικνύεται από το **file position pointer**.

Η συνάρτηση **fread** μεταφέρει ένα καθορισμένο αριθμό bytes από τη θέση του αρχείου που καθορίζεται από το **file position pointer** σε μια περιοχή της μνήμης αρχίζοντας από τη καθορισμένη διεύθυνση της μνήμης.

Τώρα αν θέλουμε να γράψουμε ένα ακέραιο (int) σε ένα αρχείο, αντί να χρησιμοποιήσουμε

fprintf(fPtr, “%d”, number);

το οποίο μπορεί να γράψει από 1 ψηφίο μέχρι 11 ψηφία (1 για το πρόσημο) και το κάθε ένα ψηφίο απαιτεί 1 byte για ένα 4-byte ακέραιο, μπορούμε να χρησιμοποιήσουμε

fwrite(&number, sizeof(int), 1, fPtr);

το οποίο πάντα γράφει 4-bytes από τη μεταβλητή **number** στο αρχείο που αντιπροσωπεύεται από το **fPtr**. Μετά, η εντολή **fread** μπορεί να χρησιμοποιηθεί για να διαβάσουμε αυτά τα 4 bytes.

Οι συναρτήσεις **fwrite** και **fread** είναι ικανές να διαβάζουν και να γράφουν πίνακες δεδομένων (arrays) από και στο δίσκο. Το τρίτο όρισμα, **1**, του προηγούμενου παραδείγματος είναι ο αριθμός των στοιχείων που θα γραφτούν στο δίσκο. Σε αυτό το παράδειγμα γράφουμε μόνο ένα στοιχείο. Αυτό το τρίτο όρισμα μπορεί να χρησιμοποιηθεί και με την εντολή **fread**. Τα προγράμματα που επεξεργάζονται αρχεία

σπάνια γράφουν ένα απλό πεδίο στο αρχείο. Συνήθως, γράφουμε ένα ολόκληρο **struct** κάθε μια φορά.
Θεωρείστε το ακόλουθο πρόβλημα:

Δημιουργήστε ένα πιστωτικό σύστημα το οποίο αποθηκεύει μέχρι και 100 σταθερού μεγέθους εγγραφές. Η κάθε εγγραφή πρέπει να αποτελείται από ένα αριθμό λογαριασμού ο οποίος θα χρησιμοποιείται σαν κλειδί, ένα επίθετο, ένα όνομα και ένα υπόλοιπο. Το πρόγραμμα πρέπει να είναι ικανό να ενημερώνει ένα λογαριασμό, να προσθέτει ένα νέο λογαριασμό, να διαγράφει ένα λογαριασμό και να δημιουργεί μια μορφοποιημένη λίστα όλων των λογαριασμών σε ένα αρχείο κειμένου για εκτύπωση. Χρησιμοποιείστε αρχείο τυχαίας προσπέλασης.

1. Δημιουργία αρχείου Τυχαίας Προσπέλασης

Το ακόλουθο πρόγραμμα μας δείχνει πώς να ανοίγουμε ένα αρχείο τυχαίας προσπέλασης, πως ορίζουμε μιαν εγγραφή χρησιμοποιώντας το **struct**, πως γράφουμε δεδομένα στο δίσκο και πως κλείνουμε το αρχείο.

Αυτό το πρόγραμμα δημιουργεί 100 κενές εγγραφές για το αρχείο **“credit.dat”** χρησιμοποιώντας την εντολή **fwrite**. Το κάθε κενό **struct** περιέχει **0** για το **account number**, **NULL** για το **last name**, **NULL** για το **first name** και **0.0** για το **balance**.

Το αρχείο προετοιμάζεται με αυτό τον τρόπο για να δημιουργήσουμε χώρο στο δίσκο στον οποίο το αρχείο θα αποθηκευτεί και για να μπορούμε να προσδιορίσουμε αν μια εγγραφή περιέχει δεδομένα.

Η ΓΡΑΜΜΗ 23 γράφει ένα block (συγκεκριμένο αριθμό bytes) δεδομένων στο αρχείο. Αυτή η γραμμή είναι η αιτία ώστε η δομή **&blankClient** του μεγέθους **sizeof(struct clientData)** να γραφτεί στο αρχείο που δείχνεται από το **cfPtr**.

Ο τελεστής **sizeof** χρησιμοποιείται για να μας επιστρέψει το μέγεθος σε bytes του αντικειμένου που βρίσκεται μέσα στις παρενθέσεις. Μπορεί να χρησιμοποιηθεί για οποιοδήποτε τύπο δεδομένων. Για παράδειγμα **sizeof(int)** θα μας επιστρέψει **2** ή **4** ανάλογα σε πιο Η/Υ εργαζόμαστε.

Η συνάρτηση **fwrite** χρησιμοποιείται για να γράφουμε διάφορα στοιχεία ενός πίνακα αντικειμένων. Για να γράψουμε αυτά τα στοιχεία ο προγραμματιστής χρησιμοποιεί ένα δείκτη στο πίνακα σαν το πρώτο όρισμα στη κλήση της **fwrite** και καθορίζει τον αριθμό των στοιχείων για να γραφτούν με το τρίτο όρισμα. Στο παράδειγμα μας το **fwrite** χρησιμοποιείται για να γράψουμε ένα μοναδικό αντικείμενο το οποίο δεν είναι στοιχείο πίνακα. Για να γράψουμε ένα αντικείμενο είναι σαν να γράφουμε ένα στοιχείο του πίνακα, και ως εκ τούτου χρησιμοποιούμε το **1**.

Παράδειγμα 125 (Δημιουργία αρχείου τυχαίας προσπέλασης διαδοχικά (sequentially)

```
1.  /* Example
2.  Creating a randomly accessed file sequentially */
3.  #include <stdio.h>
4.
5.  struct clientData {
6.      int acctNum;
7.      char lastName[ 15 ];
8.      char firstName[ 10 ];
9.      float balance;
10. };
11.
12. int main()
13. {
14.     int i;
15.     struct clientData blankClient = { 0, "", "", 0.0 };
16.     FILE *cfPtr;
17.
18.     if ( ( cfPtr = fopen( "credit.dat", "w" ) ) == NULL )
19.         printf( "File could not be opened.\n" );
20.     else {
21.
22.             for ( i = 1; i <= 100; i++ )
23.                 fwrite( &blankClient,
24.                                 sizeof( struct clientData ), 1, cfPtr );
25.
26.             fclose ( cfPtr );
27.     }
28.
29.     return 0;
30. }
```

Το balance μπορεί να δηλωθεί και σαν double αλλά να διαβάζεται με %ld διαφορετικά υπάρχουν compilers που δεν διαβάζουν σωστά το αρχείο.

2. Πως γράφουμε τυχαίως δεδομένα σε ένα αρχείο Τυχαίας Προσπέλασης

Το ακόλουθο πρόγραμμα γράφει δεδομένα στο αρχείο **"credit.dat"**. Χρησιμοποιεί τον συνδυασμό των συναρτήσεων **fseek** και **fwrite** για να αποθηκεύσει δεδομένα σε συγκεκριμένες θέσεις στο αρχείο. Η συνάρτηση **fseek** τοποθετεί το δείκτη του αρχείου σε συγκεκριμένη θέση στο αρχείο και μετά η **fwrite** γράφει τα δεδομένα.

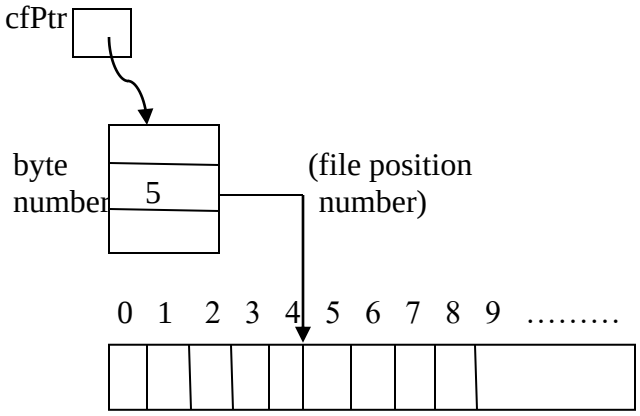
Παράδειγμα 126

```
1.  /* Example
2.  Writing to a random access file */
3.  #include <stdio.h>
4.  struct clientData {
5.      int acctNum;
6.      char lastName[ 15 ];
7.      char firstName[ 10 ];
8.      float balance;
9.  };
10. int main()
11. {
12.     FILE *cfPtr;
13.     struct clientData client = { 0, "", "", 0.0 };
14.
15.     if ( ( cfPtr = fopen( "credit.dat", "r+" ) ) == NULL )
16.         printf( "File could not be opened.\n" );
17.     else {
18.         printf( "Enter account number"
19.                                 " ( 1 to 100, 0 to end input )\n? " );
20.         scanf( "%d", &client.acctNum );
21.
22.         while ( client.acctNum != 0 ) {
23.             printf( "Enter lastname, firstname, balance\n? " );
24.             fscanf( stdin, "%s%s%f", client.lastName,
25.                                 client.firstName, &client.balance );
26.             fseek( cfPtr, ( client.acctNum - 1 ) *
27.                             sizeof( struct clientData ), SEEK_SET );
28.             fwrite( &client, sizeof( struct clientData ), 1,
29.                                                     cfPtr );
30.             printf( "Enter account number\n? " );
31.             scanf( "%d", &client.acctNum );
32.         }
33.
34.         fclose( cfPtr );
35.     }
36.     return 0;
37. }
```

ΕΞΟΔΟΣ:

```
Enter account number (1 to 100, 0 to end input)
? 37
Enter lastname, firstname, balance
? Barker Doug 0.00
Enter account number
? 29
Enter lastname, firstname, balance
? Brown Nancy -24.54
Enter account number
? 96
Enter lastname, firstname, balance
? Stone Sam 34.98
Enter account number
? 88
Enter lastname, firstname, balance
? Smith Dave 258.34
Enter account number
? 33
Enter lastname, firstname, balance
? Dunn Stacey 314.33
Enter account number
? 0
```

Η ΓΡΑΜΜΗ 26 τοποθετεί το δείκτη του αρχείου που αναφέρεται από το **cfPtr** στη θέση του byte που υπολογίζεται από **(accountNum – 1) * sizeof (struct clientData);** Η τιμή αυτής της παράστασης ονομάζεται **offset** ή **displacement**. Επειδή το account number είναι μεταξύ 1 και 100 και η τοποθεσία του byte ξεκινά από το 0 αφαιρούμε 1 από το account number όταν υπολογίζουμε την τοποθεσία του byte. Η σταθερά **SEEK_SET** μας δείχνει ότι ο δείκτης του αρχείου τοποθετείται με σχέση με την αρχή του αρχείου από την τιμή του offset.



3. Πως διαβάζουμε τυχαίως δεδομένα από ένα αρχείο Τυχαίας Προσπέλασης

Η συνάρτηση **fread** διαβάζει ένα συγκεκριμένο αριθμό bytes από το αρχείο στη μνήμη. Για παράδειγμα η εντολή,

fread(&client, sizeof(struct clientData), 1, cfPtr);

στις ΓΡΑΜΜΕΣ 24-25 του ακόλουθου προγράμματος διαβάζουν τον αριθμό των bytes που καθορίζεται από το **sizeof(struct clientData)** από το αρχείο που αναφέρεται από το **cfPtr** και αποθηκεύει τα δεδομένα στη δομή **client**.

Τα bytes διαβάζονται από τη τοποθεσία του αρχείου που προσδιορίζεται από το **file position pointer**.

Η συνάρτηση **fread** μπορεί να χρησιμοποιηθεί για να διαβάζουμε αρκετά στοιχεία πινάκων σταθερού μεγέθους παρέχοντας ένα δείκτη στον πίνακα τον οποίο τα στοιχεία θα αποθηκευτούν και δείχνοντας τον αριθμό των στοιχείων που θα διαβαστούν. Τον αριθμό των στοιχείων τον παρέχουμε με το τρίτο όρισμα της εντολής **fread**.

Το ακόλουθο πρόγραμμα διαβάζει διαδοχικά (sequentially) την κάθε εγγραφή του αρχείου **"credit.dat"**, καθορίζει εάν η εγγραφή περιέχει δεδομένα και τυπώνει μορφοποιημένα δεδομένα για τις εγγραφές που περιέχουν δεδομένα.

Η συνάρτηση **feof** καθορίζει πότε φτάνουμε στο τέλος του αρχείου και η συνάρτηση **fread** μεταφέρει δεδομένα από το δίσκο στη δομή **client** του τύπου **clientData**.

Παράδειγμα 127

```
1.  /* Example
2.  Reading a random access file sequentially */
3.  #include <stdio.h>
4.
5.  struct clientData {
6.      int acctNum;
7.      char lastName[ 15 ];
8.      char firstName[ 10 ];
9.      float balance;
10. };
11.
12. int main()
13. {
14.     FILE *cfPtr;
15.     struct clientData client = { 0, "", "", 0.0 };
16.
17.     if ( ( cfPtr = fopen( "credit.dat", "r" ) ) == NULL )
18.         printf( "File could not be opened.\n" );
19.     else {
20.             printf( "%-6s%-16s%-11s%10s\n", "Acct", "Last Name",
21.                                             "First Name", "Balance" );
22.
23.             while ( !feof( cfPtr ) ) {
24.                 fread( &client, sizeof( struct clientData ), 1,
25.                                                         cfPtr );
26.                 if ( client.acctNum != 0 )
27.                     printf( "%-6d%-16s%-11s%10.2f\n",
28.                 client.acctNum, client.lastName,
29.                 client.firstName, client.balance );
30.             }
31.             fclose( cfPtr );
32.     }
33.     return 0;
34. }
```

ΕΞΟΔΟΣ:

```
Acct  Last Name       First name  Balance
29    Brown           Nancy         -24.54
33    Dunn            Stacey        314.33
37    Barker          Doug            0.00
88    Smith           Dave          258.34
96    Stone           Sam            34.98
```

Παράδειγμα 128 (Case Study)

Τώρα ας παρουσιάσουμε ένα πλούσιο πρόγραμμα επεξεργασίας συναλλαγών που χρησιμοποιεί αρχεία τυχαίας προσπέλασης.

Αυτό το πρόγραμμα

- διατηρεί πληροφορίες τραπεζιτικών λογαριασμών,
- ενημερώνει υφιστάμενους λογαριασμούς,
- προσθέτει νέους λαγαριασμούς,
- διαγράφει λογαριασμούς και
- αποθηκεύει μια λίστα τρεχούμενων λογαριασμών σε ένα αρχείο (text) για εκτύπωση.

Θεωρούμε ότι το πρόγραμμα του **παραδείγματος 126** εκτελέστηκε για να δημιουργηθεί το αρχείο **credit.dat**.

```
1.  /* Example
2.  This program reads a random access file sequentially,
3.  updates data already written to the file, creates new
4.  data to be placed in the file, and deletes data
5.  already in the file.                          */
6.  #include <stdio.h>
7.
8.  struct clientData {
9.      int acctNum;
10.     char lastName[ 15 ];
11.     char firstName[ 10 ];
12.     float balance;
13. };
14.
15. int enterChoice( void );
16. void textFile( FILE * );
17. void updateRecord( FILE * );
18. void newRecord( FILE * );
19. void deleteRecord( FILE * );
20.
21. int main()
22. {
23.     FILE *cfPtr;
24.     int choice;
25.
26.     if ( ( cfPtr = fopen( "credit.dat", "r+" ) ) == NULL )
27.         printf( "File could not be opened.\n" );
28.     else {
29.
30.             while ( ( choice = enterChoice() ) != 5 ) {
```

```
31.
32.                 switch ( choice ) {
33.                     case 1:
34.                         textFile( cfPtr );
35.                         break;
36.                     case 2:
37.                         updateRecord( cfPtr );
38.                         break;
39.                     case 3:
40.                         newRecord( cfPtr );
41.                         break;
42.                     case 4:
43.                         deleteRecord( cfPtr );
44.                         break;
45.               }
46.             }
47.
48.             fclose( cfPtr );
49.     }
50.
51.     return 0;
52. }
53.
54. void textFile( FILE *readPtr )
55. {
56.     FILE *writePtr;
57.     struct clientData client = { 0, "", "", 0.0 };
58.
59.     if ( ( writePtr = fopen( "accounts.txt", "w" ) ) == NULL )
60.         printf( "File could not be opened.\n" );
61.     else {
62.             rewind( readPtr );
63.             fprintf( writePtr, "%-6s%-16s%-11s%10s\n",
64.                         "Acct", "Last Name", "First Name","Balance" );
65.
66.             while ( !feof( readPtr ) ) {
67.                 fread( &client, sizeof( struct clientData ), 1,
68.                                                             readPtr );
69.
70.                 if ( client.acctNum != 0 )
71.                     fprintf( writePtr, "%-6d%-16s%-11s%10.2f\n",
72.                                         client.acctNum, client.lastName,
73.                                         client.firstName, client.balance );
74.              }
75.
76.              fclose( writePtr );
```

```
77.      }
78.
79.  }
80.
81.  void updateRecord( FILE *fPtr )
82.  {
83.      int account;
84.      double transaction;
85.      struct clientData client = { 0, "", "", 0.0 };
86.
87.      printf( "Enter account to update ( 1 - 100 ): " );
88.      scanf( "%d", &account );
89.      fseek( fPtr,
90.                      ( account - 1 ) * sizeof( struct clientData ),
91.                                                      SEEK_SET );
92.      fread( &client, sizeof( struct clientData ), 1, fPtr );
93.
94.      if ( client.acctNum == 0 )
95.          printf( "Acount #%d has no information.\n", account );
96.      else {
97.              printf( "%-6d%-16s%-11s%10.2f\n\n",
98.                      client.acctNum, client.lastName,
99.                      client.firstName, client.balance );
100.             printf( "Enter charge ( + ) or payment ( - ): " );
101.             scanf( "%lf", &transaction );
102.             client.balance += transaction;
103.             printf( "%-6d%-16s%-11s%10.2f\n",
104.                      client.acctNum, client.lastName,
105.                      client.firstName, client.balance );
106.             fseek( fPtr,
107.                      ( account - 1 ) * sizeof( struct clientData ),
108.                                                      SEEK_SET );
109.             fwrite( &client, sizeof( struct clientData ), 1,
110.                                                      fPtr );
111.     }
112. }
113.
114. void deleteRecord( FILE *fPtr )
115. {
116.     struct clientData client,
117.          blankClient = { 0, "", "", 0 };
118.          int accountNum;
119.
120.     printf( "Enter account number to "
121.                              "delete ( 1 - 100 ): " );
122.     scanf( "%d", &accountNum );
```

```
123.     fseek( fPtr,
124.              ( accountNum - 1 ) * sizeof( struct clientData ),
125.                                                  SEEK_SET );
126.     fread( &client, sizeof( struct clientData ), 1, fPtr );
127.
128.     if ( client.acctNum == 0 )
129.        printf( "Account %d does not exist.\n", accountNum );
130.     else {
131.              fseek( fPtr,
132.                   ( accountNum - 1 ) * sizeof( struct clientData ),
133.                                                   SEEK_SET );
134.              fwrite( &blankClient,
135.                        sizeof( struct clientData ), 1, fPtr );
136.     }
137. }
138.
139. void newRecord( FILE *fPtr )
140. {
141.     struct clientData client = { 0, "", "", 0.0 };
142.     int accountNum;
143.     printf( "Enter new account number ( 1 - 100 ): " );
144.     scanf( "%d", &accountNum );
145.     fseek( fPtr,
146.              ( accountNum - 1 ) * sizeof( struct clientData ),
147.                                                   SEEK_SET );
148.     fread( &client, sizeof( struct clientData ), 1, fPtr );
149.
150.     if ( client.acctNum != 0 )
151.        printf( "Account #%d already contains information.\n",
152.                                    client.acctNum );
153.     else {
154.              printf( "Enter lastname, firstname, balance\n? " );
155.              scanf( "%s%s%lf", &client.lastName, &client.firstName,
156.                                    &client.balance );
157.              client.acctNum = accountNum;
158.              fseek( fPtr, ( client.acctNum - 1 ) *
159.                              sizeof( struct clientData ), SEEK_SET );
160.              fwrite( &client,
161.                              sizeof( struct clientData ), 1, fPtr );
162.     }
163. }
164.
165. int enterChoice( void )
166. {
167.     int menuChoice;
168.
```

```
169.    printf( "\nEnter your choice\n"
170.            "1 - store a formatted text file of acounts called\n"
171.            "    \"accounts.txt\" for printing\n"
172.            "2 - update an account\n"
173.            "3 - add a new account\n"
174.            "4 - delete an account\n"
175.            "5 - end program\n? " );
176.    scanf( "%d", &menuChoice );
177.    return menuChoice;
178. }
```

Το πρόγραμμα έχει 5 επιλογές.

Η **επιλογή 1** καλεί την συνάρτηση **textfile** για να αποθηκεύσει μια μορφοποιημένη λίστα όλων των λογαριασμών στο αρχείο (text) **accounts.txt** που μπορεί να εκτυπωθεί μετέπειτα. Η συνάρτηση χρησιμοποιεί το **fread** και τις τεχνικές για sequential αρχεία που είδαμε στο **παράδειγμα 96**. Μετά την **επιλογή 1** το αρχείο περιέχει:

```
Acct  Last Name      First name      Balance
29    Brown          Nancy            -24.54
33    Dunn           Stacey           314.33
37    Barker         Doug               0.00
88    Smith          Dave             258.34
96    Stone          Sam               34.98
```

Η **επιλογή 2** καλεί την συνάρτηση **updateRecord** για να ενημερώσει ένα λογαριασμό. Η συνάρτηση ενημερώνει μόνο κάποιο λογαριασμό που υπάρχει, έτσι η συνάρτηση πρώτα ελέγχει εάν ο λογαριασμός που δόθηκε από το χρήστη δεν είναι κενός. Εάν ο λογαριασμός δεν περιέχει δεδομένα ένα μήνυμα λάθους δίδεται στο χρήστη. Εάν ο λογαριασμός υφίσταται τότε το πρόγραμμα συνεχίζει, για παράδειγμα, ως εξής:

```
Enter account to update (1 – 100): 37
37  Barker      Doug        0.00

Enter charge (+) or payment (-): +87.99
```

Η **επιλογή 3** καλεί την συνάρτηση **newRecord** για να προσθέσει νέο λογαριασμό στο αρχείο. Εάν ο χρήστης δώσει έναν αριθμό λογαριασμού που υπάρχει στο αρχείο η συνάρτηση δίδει ένα μήνυμα λάθους. Αυτή η συνάρτηση χρησιμοποιεί την ίδια διαδικασία για πρόσθεση νέας εγγραφής όπως το ***παράδειγμα 95***. Μια τυπική έξοδος αυτής της επιλογής είναι η ακόλουθη:

```
Enter new account number (1 – 100): 22
Enter lastname, firstname, balance
? Johnston Sarah 247.45
```

Η **επιλογή 4** καλεί την συνάρτηση **deleteRecord** για να διαγράψει μια εγγραφή από το αρχείο. Αυτό επιτυγχάνεται ζητώντας από το χρήστη για τον αριθμό λογαριασμού που θέλει να διαγράψει και μετά ξανά αρχικοποιεί την εγγραφή. Εάν η εγγραφή δεν υφίσταται η συνάρτηση δίνει ένα μήνυμα λάθους στο χρήστη.

Η **επιλογή 5** τερματίζει την εκτέλεση του προγράμματος.